上海交通大學
SHANGHAI JIAO TONG UNIVERSITY



Software Testing

Hufei

2015-09

Course information

- Credits: 3
- 2~4 classes /week, 16 weeks
- Textbook:

《SoftwareTesting》，A Craftsman's Approach, Fourth Edition，Paul C. Jorgensen，2014

《软件测试:第3版》，人民邮电出版社，2011

《软件测试：第2版》， 机械工业出版社，2002

- Reference book:

① "Foundation of software testing", Aditya P. Mathur, 机械工业出版社, 2011年

② 《软件测试 原理与实践》，srinivasan Desikan 韩柯 李娜等译，机械工业出版社 2009年

- The grade of the course will be determined as follows:

  Projects 1~4：40%,

  Usual test ： 10%

  Final exam: 50%

Contents of the course

1. The purpose of Software testing

2. Theory and methods of software test

white-box test、black-box test、walk through、formal verification 、clean room、fault level；

3. Tools and environment of software test

Eclipse、JUnit and EMMA

4. Functional test

Boundary testing, Equivalence partitioning, Decision table testing, testing efficiency



5. Standards of software quality and software documentation

ISO9000, IEEE, Chinese national standards

6.Structure test

Control-flow testing, data-flow testing

7. Web test and selenium

8.Software Test in software development process

Integrate test、system test 、accept test、
Internationalization test

9.Interactive Testing（Correlation Test）



10. **Performance test and LoadRunner**

11. **Regression Test**

    concept and seminar

12. **Object-Oriented Test**

    basic concept and methods

13. **Mutation test**

    basic concept and methods

14. **Test Adequacy Measurement and Enhancement Control**

- Class Prerequisites：**Java, Software Engineering**
- Project requirement: **4 students as a test group, should complete 4 projects as follows;**

- project 1: **Black-box test-case generation - during week 3~6; take subject application with requirements document; create black-box test cases； execute test； report bugs in a bug tracking tool.**

- **Project 2: White-box test-case generation and coverage - during week 7~10; use instrument; evaluate coverage; create additional white-box test cases to satisfy some coverage criteria.**

- **Project 3: GUI test :during week 11~14; Select a software GUI with at least 2 level , Use selenium, to complete GUI test.**

- **Project 4: Performance test: during week 15~17; select a software of web server, use-LoadRunner, to complete performance test.**

Faculty information:

胡　飞: 软件大楼　1310　　室
电子邮箱: hufei@sjtu.edu.cn

Co-Teacher: 姚建国，软件大楼　1110室
电子邮箱：jianguo.yao@sjtu.edu.cn

TA：舒映、王臻
软件大楼 1411室
电子邮箱：

Office time: Wednesday afternoon 14:00~16:30



Chapter 1

Introduction of software testing



1.1 Importance of software testing

1.2 Concepts and Glossary

1.2.1 Goals of testing

1.2.2 Facts About Testing

1.2.3 Glossary(术语)

1.2.4 The basic concepts of software test

1.2.5 Fault Taxonomies



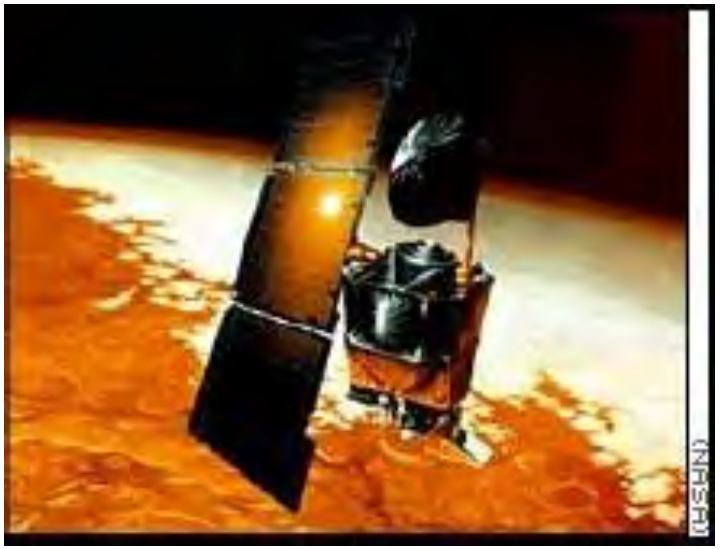
1.1 Importance of software testing

Mars Climate Orbiter

- Purpose: to relay signals from the Mars Polar Lander once it reached the surface of the planet
- Disaster: smashed into the planet instead of reaching a safe orbit
- Why: Software bug - failure to convert English measures to metric values

• $165M

1mile = 1609M





Shooting Down of Airbus 300

- 1988
- US Vicennes shot down Airbus 300
- Mistook airbus 300 for a F-14
- 290 people dead
- Why: Software bug - cryptic and misleading output displayed by the tracking software



THERAC-25 Radiation Therapy

- THERAC-25, a computer controlled radiation-therapy machine
- 1986: two cancer patients at the East Texas Cancer Center in Tyler received fatal radiation overdoses
- Why: Software bug - mishandled race condition

(i.e., miscoordination between concurrent tasks)



London Ambulance Service

- **London Ambulance Service Computer Aided Dispatch (LASCAD)**
- **Purpose: automate many of the human-intensive processes of manual dispatch systems associated with ambulance services in the UK**

– functions: Call taking

- **Failure of the London Ambulance Service on 26 and 27 November 1992**



"Nice of You to Turn Up"

- Load increased
- Emergencies accumulated
- System made incorrect allocations

– more than one ambulance being sent to the same incident

– the closest vehicle was not chosen for the emergency

- At 23:00 on October 28 the LAS eventually instigated a backup procedure, after the death of at least 20 patients

More…

"Software and its Impact: A Quantitative Assessment," by B.W. Boehm, Datamation, 19(5), 48-59 (1973)

– Errors in medical software have caused deaths

More…

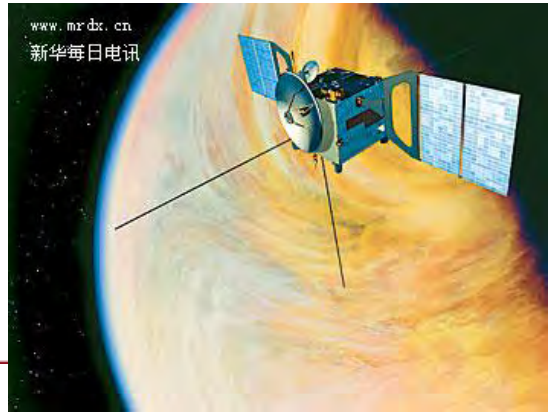
- *"The development of software for ballistic-missile defense," by H. Lin, Scientific American, vol. 253, no. 6 (Dec. 1985), p. 48*

– *An Air New Zealand airliner crashed into an Antarctic mountain*

– *North American Aerospace Defense Command reported that the U.S. was under missile attack; traced to faulty computer software - generated incorrect signals*

– *Manned space capsule Gemini V missed its landing point by 100 miles; software ignored the motion of the earth around the sun*





More…

- "Ghost in the Machine," Time Magazine, Jan. 29, 1990. p. 58
  - ***Dallas/Fort Worth air-traffic system began spitting out gibberish in the Fall of 1989 and controllers had to track planes on paper***

More…

- Software Reliability: Principles & Practice, p. 25, by G. J. Myers

– *Apollo 8 spacecraft erased part of the computer's memory*

– *Eighteen errors were detected during the 10-day flight of Apollo 14*

– *An error in a single FORTRAN statement resulted in the loss of the first American probe to Venus(金星)*

More…

- ACM SIGSOFT Software Engineering Notes, vol. 6, no. 2
  - *F-18 fighter plane crashed due to a missing exception condition*
- ACM SIGSOFT Software Engineering Notes, vol. 9, no. 5
  - *F-14 fighter plane was lost to uncontrollable spin, traced to tactical(战术) software*

- 1999年千年虫Bug　5000亿

为了节省计算机的存储空间，软件经常将存储日期用两位阿拉伯数字表示，即用“99”来表示“1999”，还用“00”来代替“1900”而不是2000，所以到2000年的时候，软件漏洞就表现出来了。

More …

- 2011年7月23日晚，在温州发生动车追尾脱轨坠落事故，40人死亡。
- 原因：列车控制中心设备存在严重设计缺陷、上道使用审查把关不严、雷击导致设备故障。

- 2012年1月，中国返乡旅客导致铁路网上售票系统瘫痪
- 原因：春运时，数以百万、千万乃至亿计的旅客需求，使意在方便旅客购票的铁路网络售票系统无法正常工作。



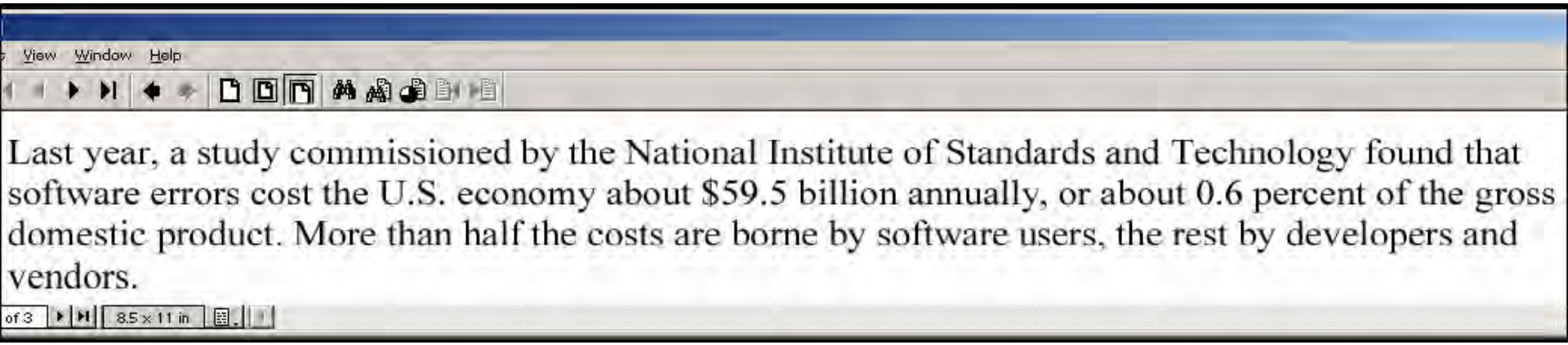
- and many many more ....



What's your experience?

Economic Impact

- **NIST study**
  - **On CNN.com - April 27, 2003**



Expectation

- Can't we expect software to execute correctly?
- Carefully made programs
  - *5 faults/1000 LOC*
  - *1M LOC will have 5000 faults*
- Windows XP has 45M LOC
  - *How many faults?*
  - *45 x 5000 = 225,000*
- Win 8 ： 25G~30G
- Why not remove the faults?

```
*** STOP: 0x0000000A (0x00000000,0x00000002,0x00000000,8038c240)
IRQL_NOT_LESS_OR_EQUAL*** Address 8038c240 has base at 8038c000 - Ntfs.SYS

CPUID:Genuine Intel 6.3.3 irql:1f SYSVER 0xf0000565

Dll Base DateStmp - Name                    Dll Base DateStmp - Name
80100000 336546bf - ntoskrnl.exe            80010000 33247f88 - hal.dll
80000100 334d3a53 - atapi.sys               80007000 33248043 - SCSIPORT.SYS
802aa000 33013e6b - epst.mpd                802b5000 336016a2 - Disk.sys
802b9000 336015af - CLASS2.SYS              8038c000 3356d637 - Ntfs.sys
802bd000 33d844be - Siwvid.sys              803e4000 33d84553 - NTice.sys
f9318000 31ec6c8d - Floppy.SYS              f95c9000 31ec6c99 - Null.SYS
f9468000 31ed868b - KSecDD.SYS              f95ca000 335e60cf - Beep.SYS
f9358000 335bc82a - i8042prt.sys            f9474000 3324806f - mouclass.sys
f947c000 31ec6c94 - kbdclass.sys            f95cb000 3373c39d - ctrl2cap.SYS
f9370000 33248011 - VIDEOPORT.SYS           fe9d7000 3370e7b9 - ati.sys
f9490000 31ec6c6d - vga.sys                 f93b0000 332480dd - Msfs.SYS
f90f0000 332480d0 - Npfs.SYS                fe957000 3356da41 - NDIS.SYS
a0000000 335157ac - win32k.sys              fe914000 334ea144 - ati.dll
fe0c9000 335bd30e - Fastfat.SYS             fe110000 31ec7c9b - Parport.SYS
fe108000 31ec6c9b - Parallel.SYS            f95b4000 31ec6c9d - ParVdm.SYS
f9050000 332480ab - Serial.SYS

Address  dword dump Build [1314]                                    - Name
801afc24 80149905 80149905 ff8e6b8c 80129c2c ff8e6b94 8025c000 - Ntfs.SYS
801afc2c 80129c2c 80129c2c ff8e6b94 00000000 ff8e6b94 80100000 - ntoskrnl.exe
801afc34 801240f2 80124f02 ff8e6df4 ff8e6f60 ff8e6c58 80100000 - ntoskrnl.exe
801afc54 80124f16 80124f16 ff8e6f60 ff8e6c3c 8015ac7e 80100000 - ntoskrnl.exe
801afc64 8015ac7e 8015ac7e ff8e6df4 ff8e6f60 ff8e6c58 80100000 - ntoskrnl.exe
801afc70 80129bda 80129bda 00000000 80088000 80106fc0 80100000 - ntoskrnl.exe

Restart and set the recovery options in the system control panel
or the /CRASHDEBUG system start option. If this message reappears,
contact your system administrator or technical support group.
```

Joke?

"If the automobile industry had developed like the software industry, Who dare to driving the cars that with too many problems".

Yeah, and if cars were like software, they would crash twice a day for no reason, and when you called for service, they'd tell you to reinstall the engine."

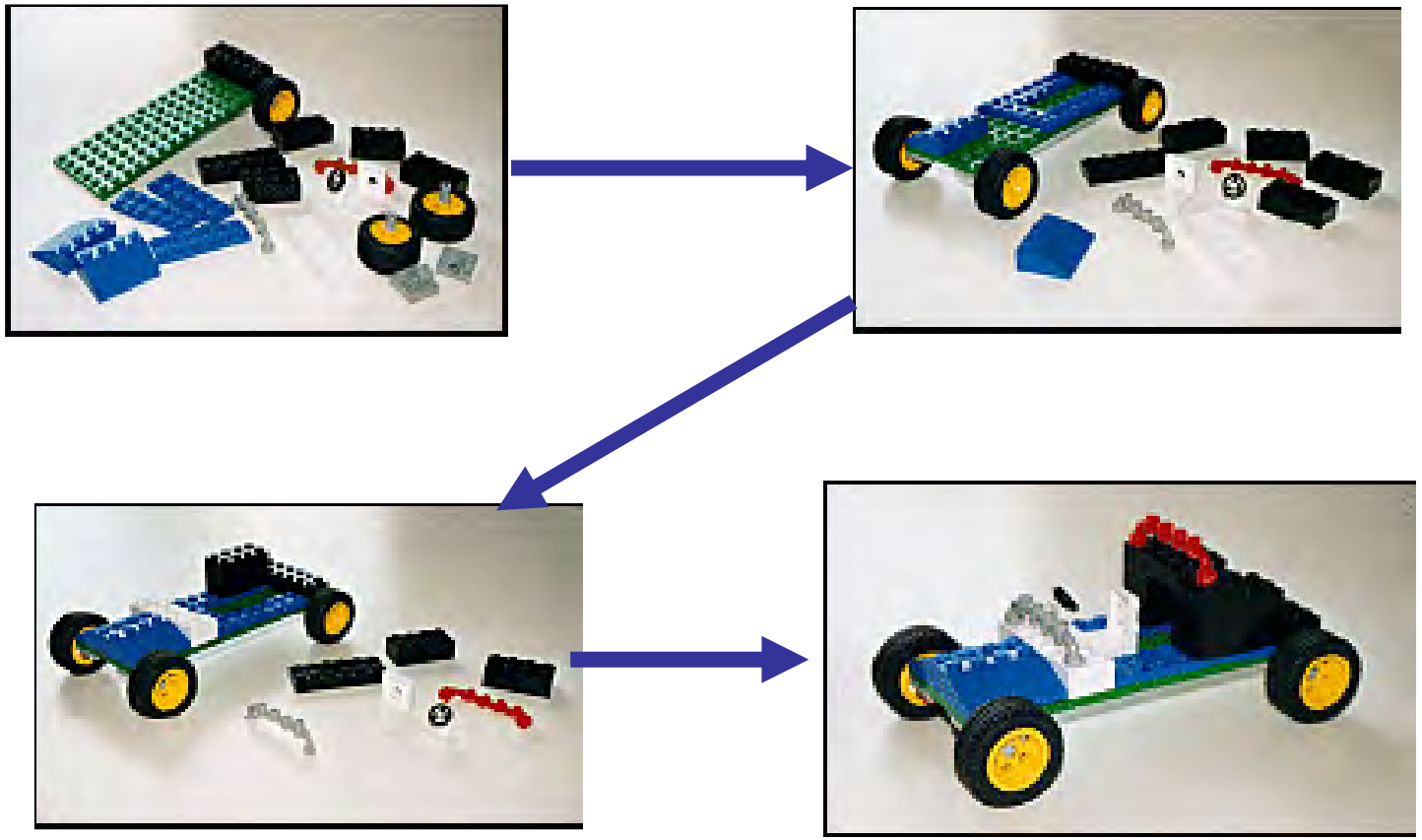




How Cars Are Engineered (A Simple View)

- User requirements
  - *Engine power, all-wheel, seating, comfort, MP3 player!!*
- Detailed design
  - *Blueprints, design documents*
- Verify design
  - *Simulation, prototyping*
- Develop parts (components)
  - *Test each component*
  - *Components may be reused*
  - *Mass produced*
- Assemble the car
  - *Test the car (Front/side crash tests, Stability tests)*
  - *Usability testing (Feedback from drivers/passengers)*

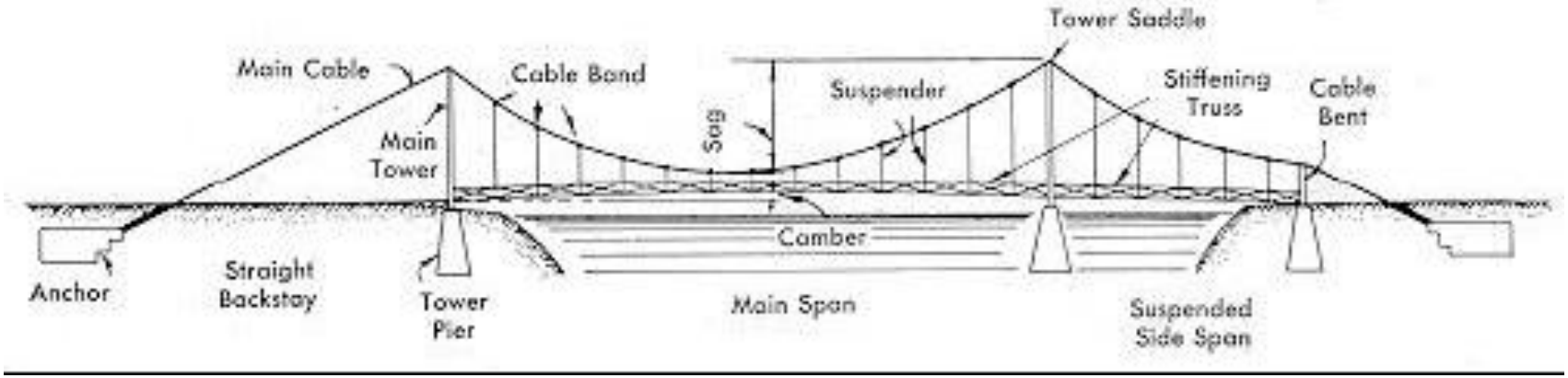
How Cars Are REALLY Engineered (A Detailed View)

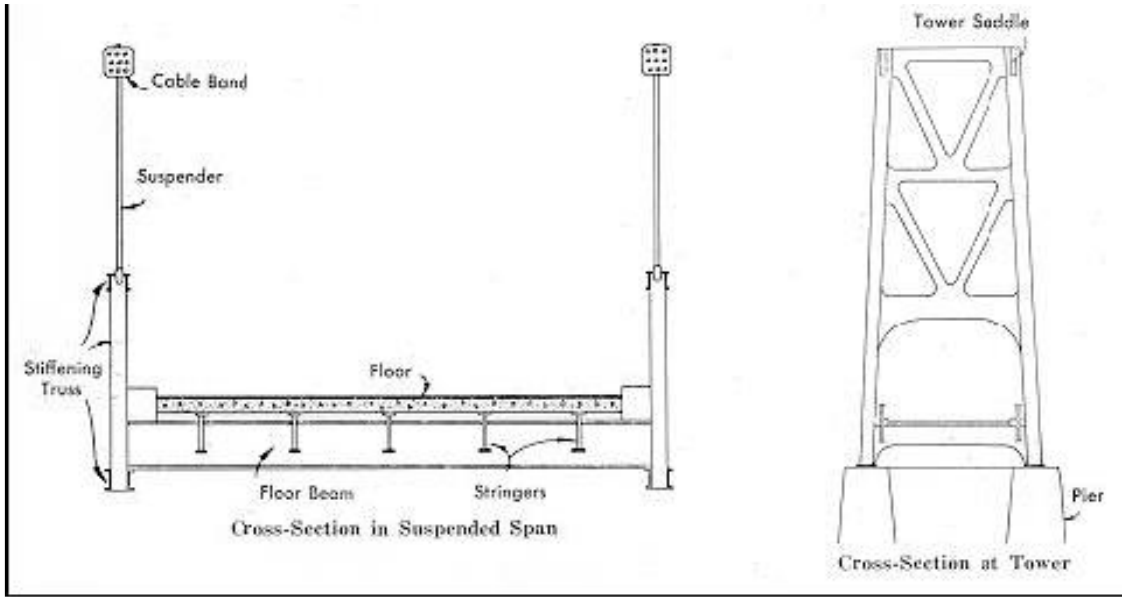


But Seriously

- Features of many LEGO parts
  - *Modularity*
  - *Reusability*
- **Each part can be used in different places and ways**
  - *Flexibility of design*
  - *Compatibility*
- **With other LEGO sets**
- Building-blocks

Similar Techniques Used by Builders: Bridges





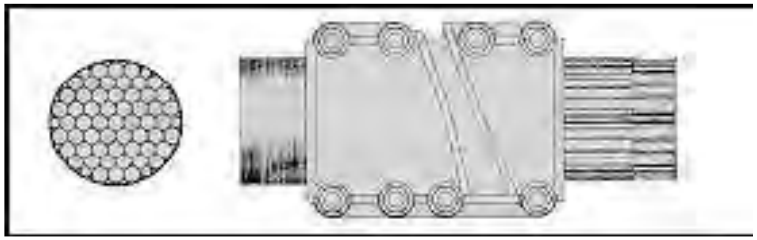
Detailed Design and Specifications

Galvanized Bridge Wire for Parallel Wire Bridge Cables. Recommended diameter .196 inch.
Galvanized Bridge Strand--consists of several bridge wires, of various diameters twisted together.
Galvanized Bridge Rope--consists of six strands twisted around a strand core.





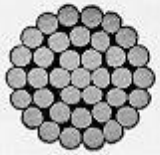
Parallel Wire Cable



Detail of Main Cable and Cable Band. The wrapping wire is omitted at the right for clarity. Note the closed construction and aluminum fillers.

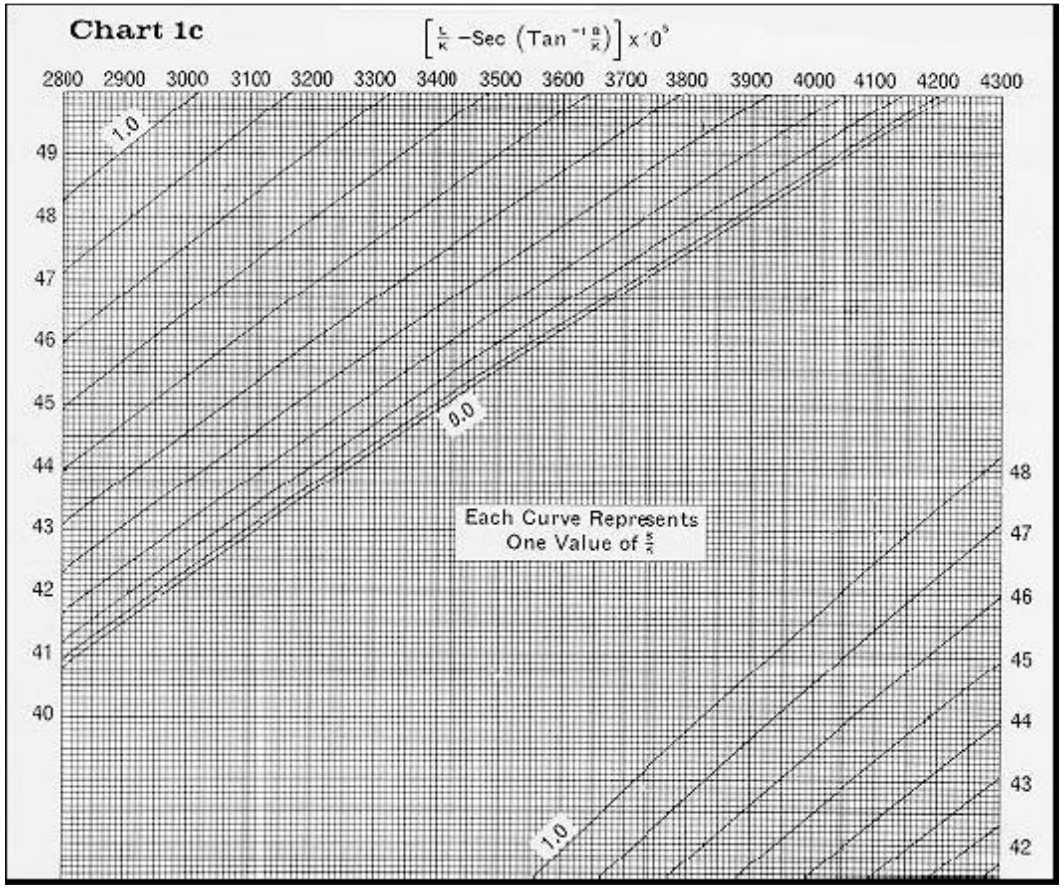
More Detailed Design and Specifications

STANDARD GALVANIZED STEEL BRIDGE STRAND



Made from Standard Galvanized Bridge Wire

Nominal Strand Diameter[3]	Approx. Weight Lb./Ft.[4]	Approx. Gross Metallic Area incl. Galv., Sq. In.[4]	Min. Breaking Strength in Tons of 2,000 lb.
1/2	0.52	.149	15.0
9/16	0.66	.190	19.0
5/8	0.82	.233	24.0
11/16	0.99	.284	29.0
3/4	1.18	.336	34.0
13/16	1.39	.396	40.0
7/8	1.61	.467	46.0
15/16	1.85	.527	54.0
1	2.00	.577	61.0
1 1/16	2.30	.653	69.0
1 1/8	2.61	.751	78.0
1 3/16	2.92	.843	86.0
1 1/4	3.22	.931	96.0
1 5/16	3.58	1.04	106.0
1 3/8	3.89	1.13	116.0
1 7/16	4.29	1.24	126.0
1 1/2	4.70	1.36	138.0
1 9/16	5.11	1.48	150.0





They Make Mistakes Too!

- Even good design cannot guarantee a perfect product
- Need testing of all products including software

How get high quality software

Two ways

① Process management

CMM,PSP,TSP

Engineering process

② Testing

Example

Matrix operation

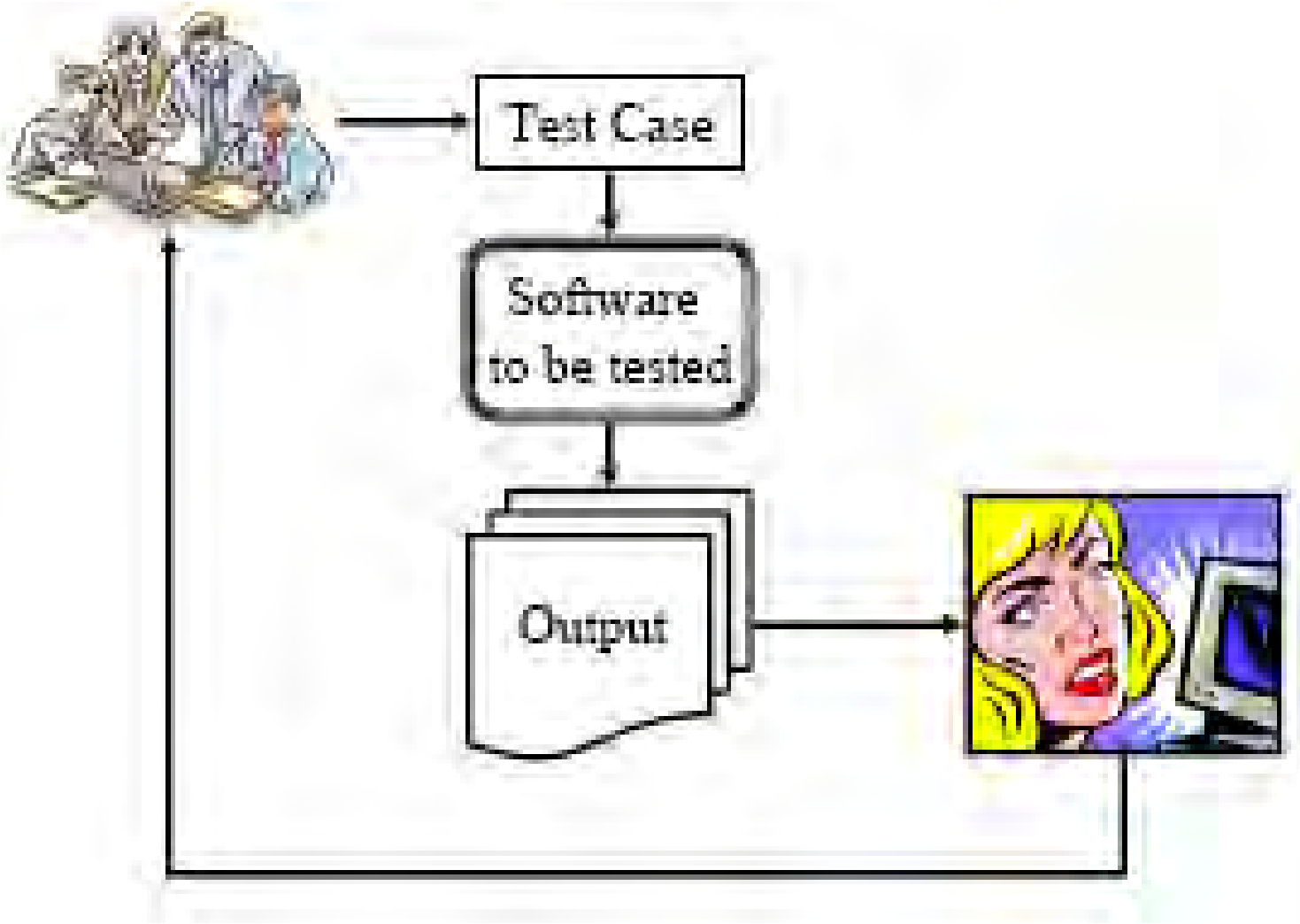
A+B, AXB

You could easily implement the algorithm.

But, is that correct ?

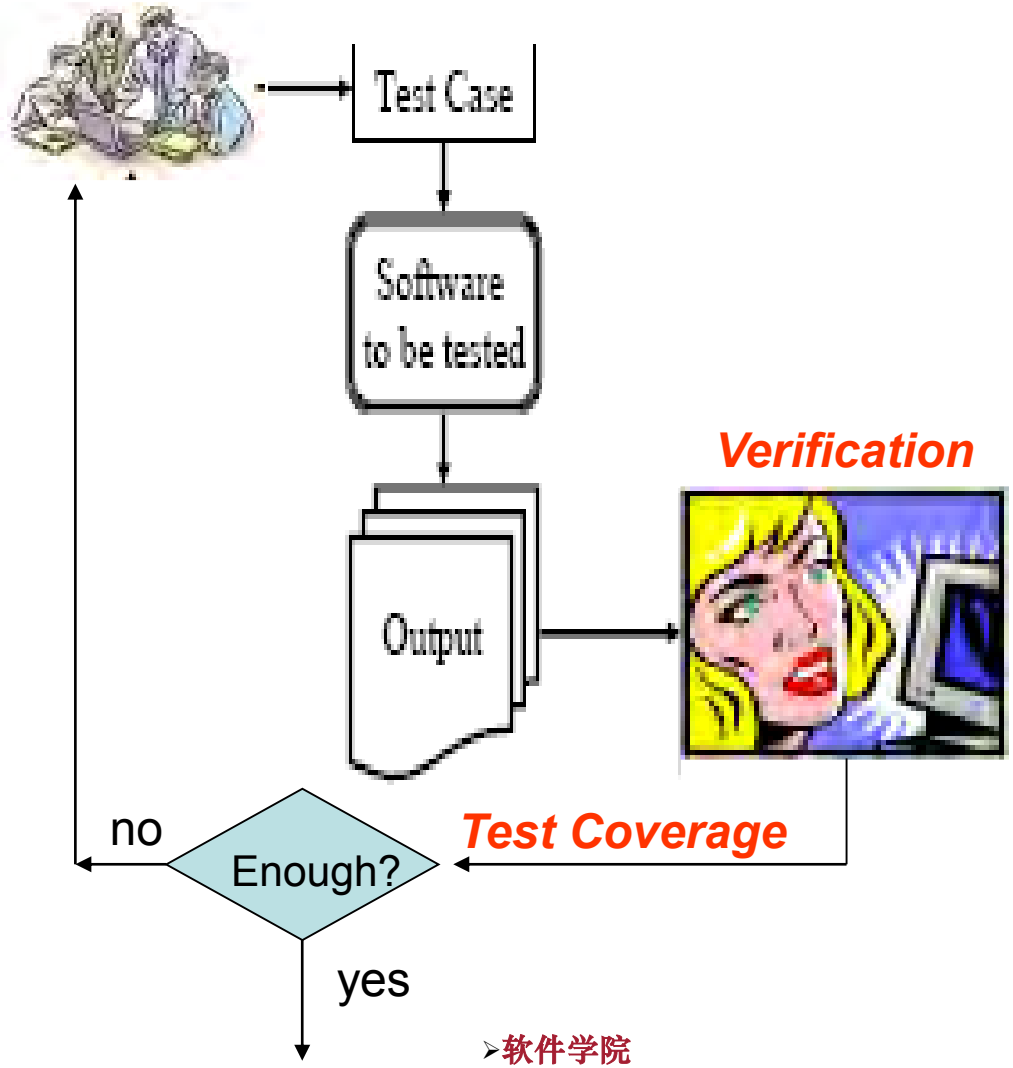
Have you thought all possibilities of Matrix ?

Software test

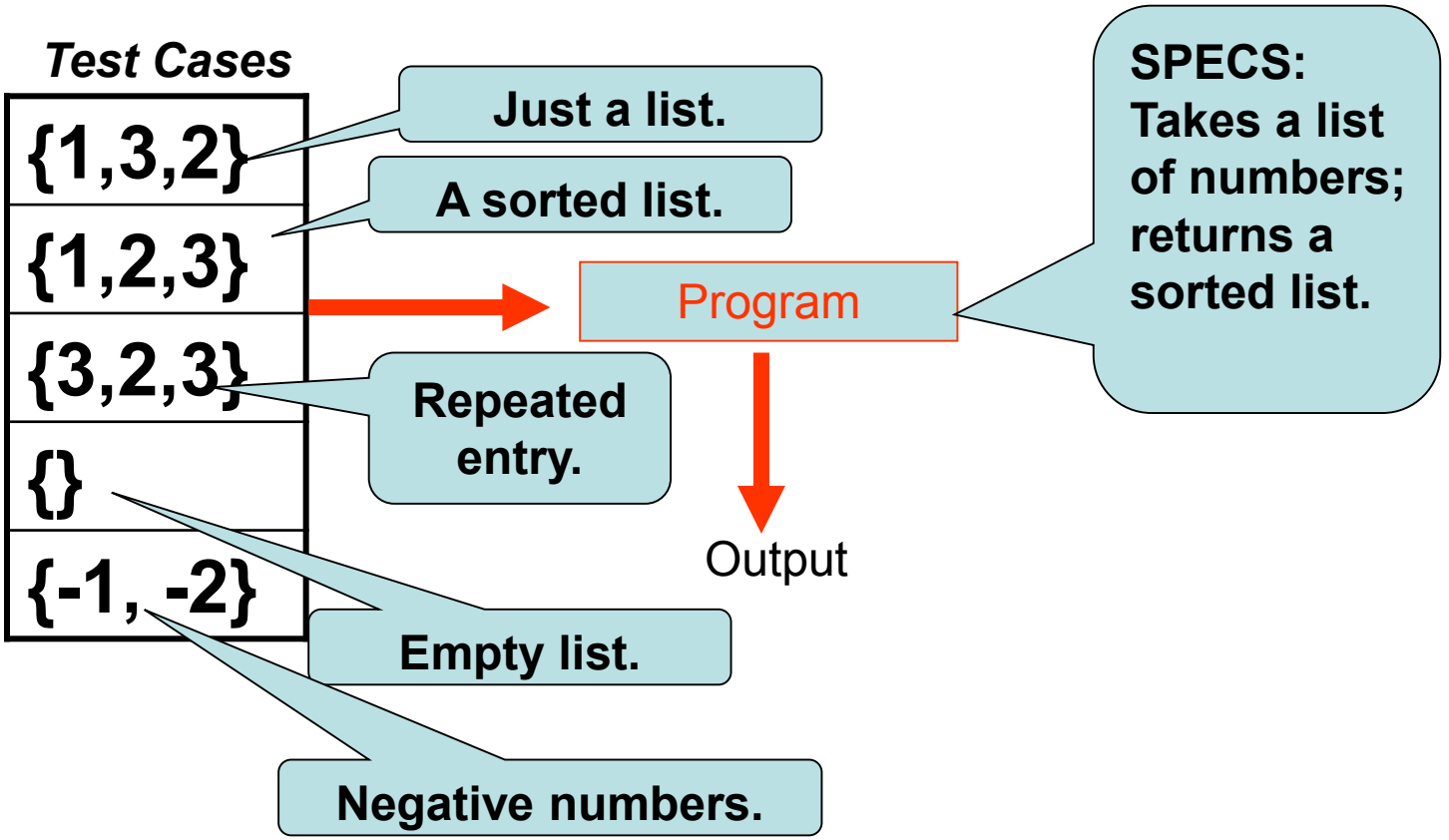


When to Stop?

Test Case Generation

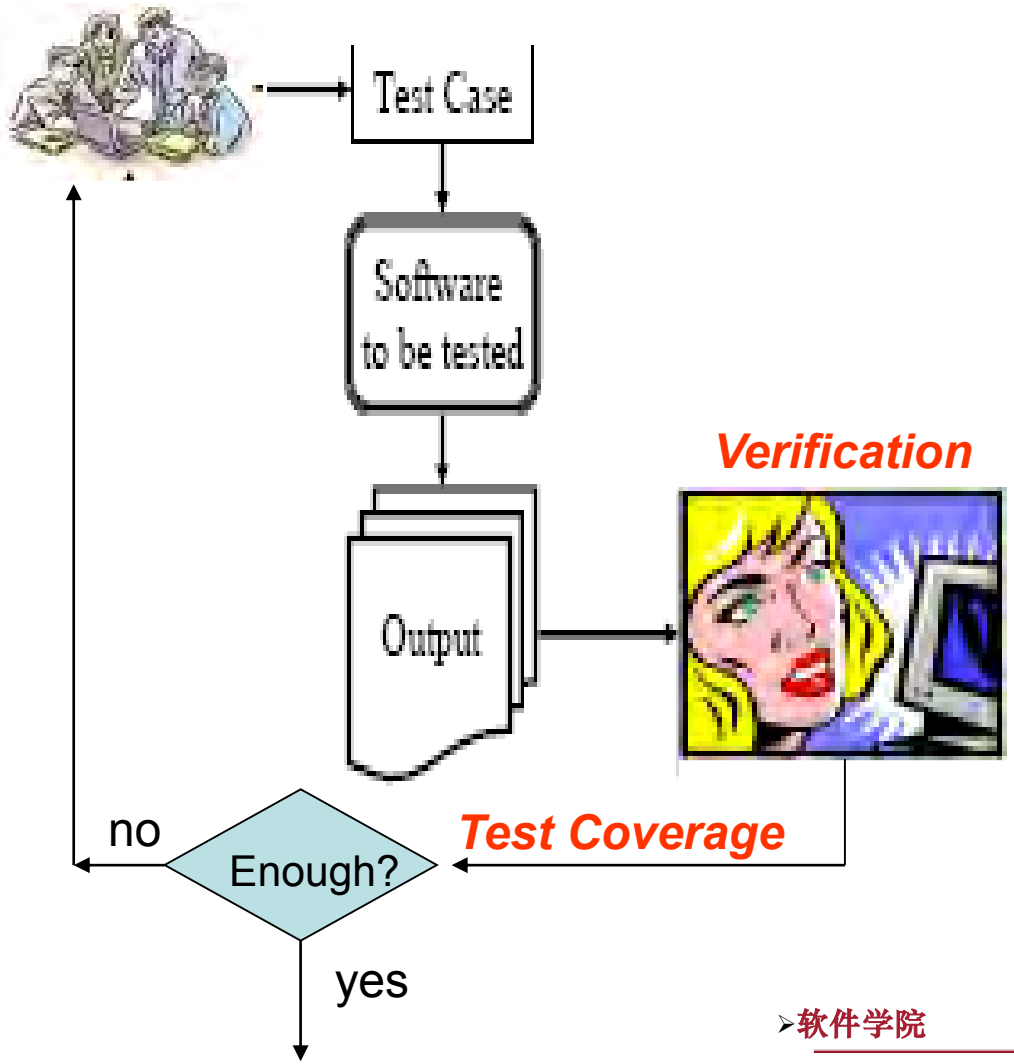


A Real Testing Example

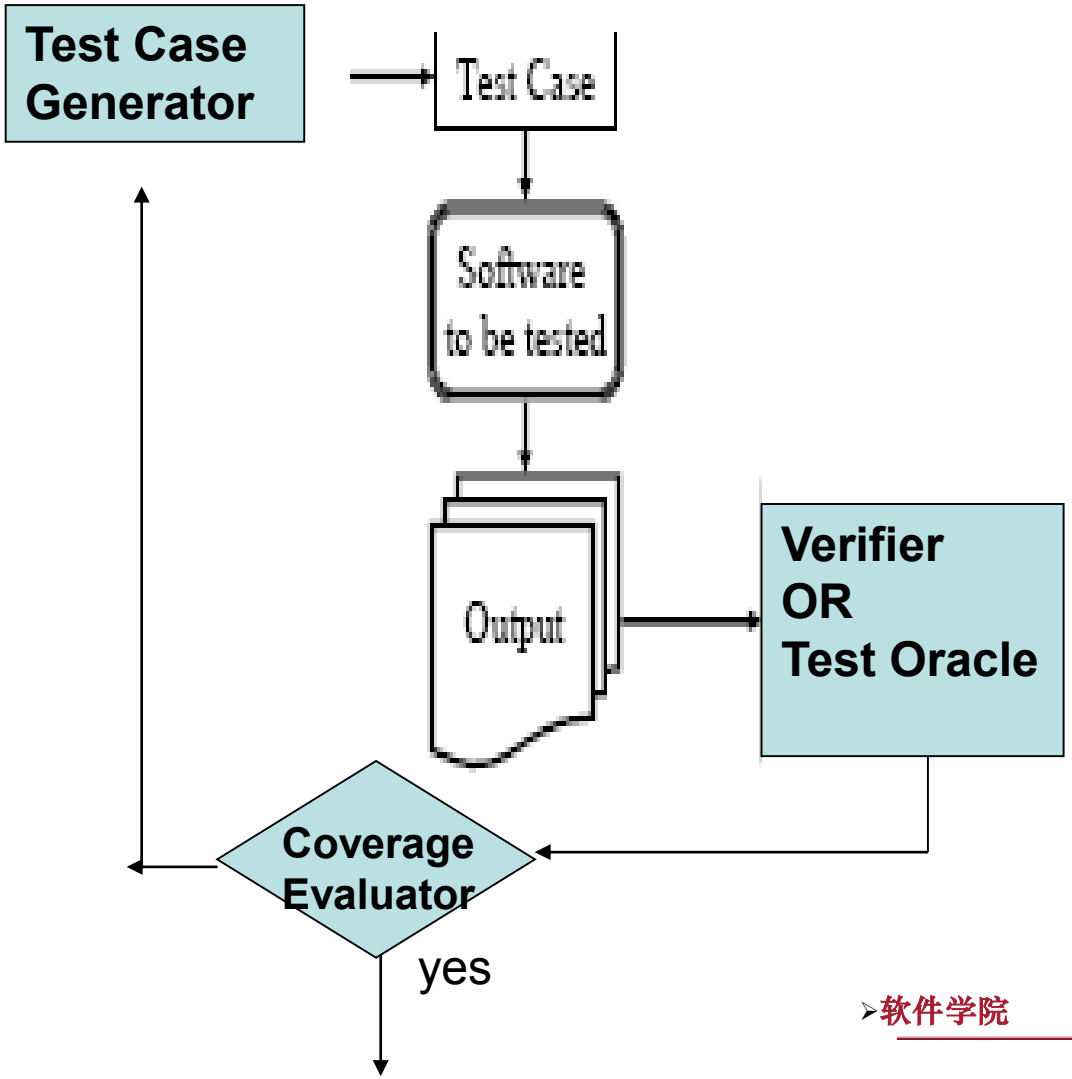


Automated Testing

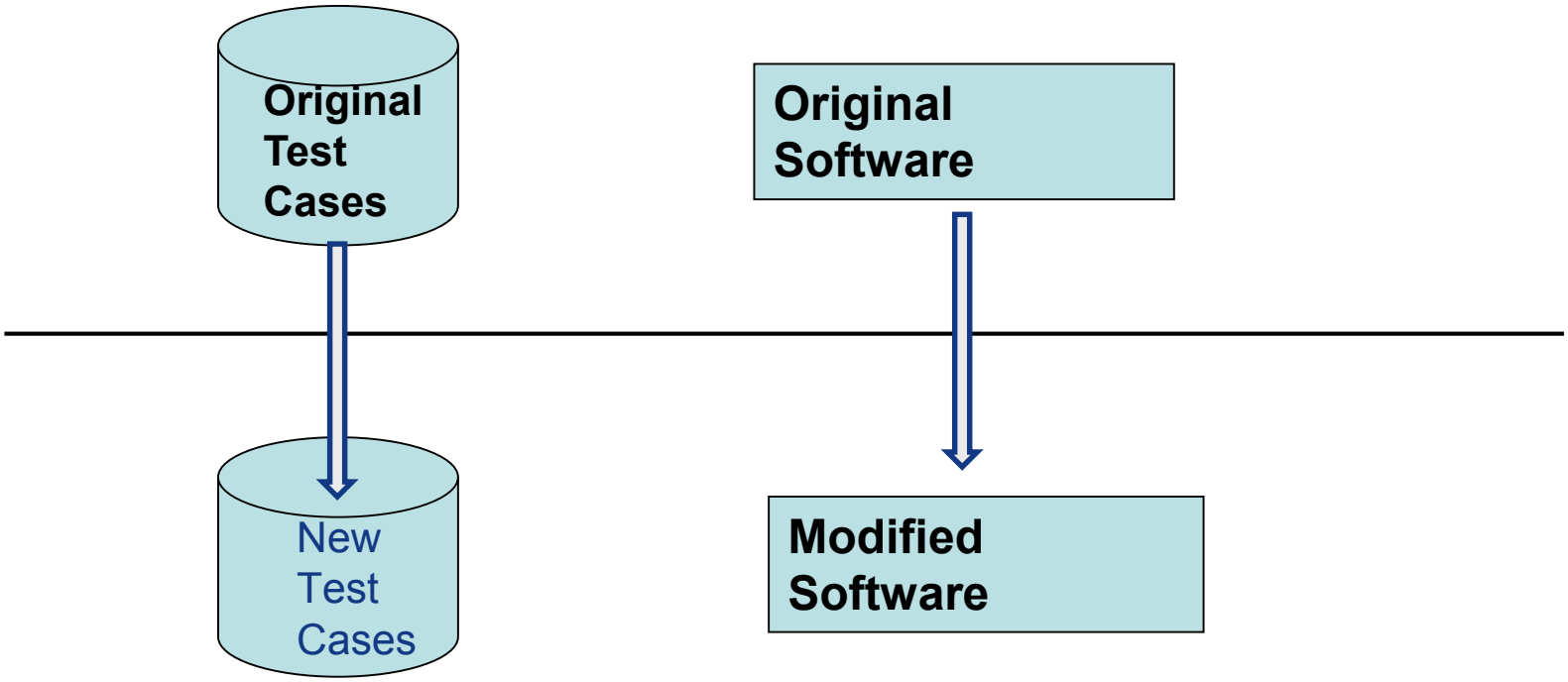
Test Case Generation



Automated Testing



Testing the New Version



How to generate new test case?

Discussion

- **Different Software Types**
  - ***Object-Oriented***
  - ***Component-Based***
  - ***Concurrent***
  - ***Distributed***
  - ***Graphical-User Interfaces***
  - ***Web***
- **Different goals of testing**
  - *function*
  - ***Performance***
  - ***Security***
  - ***Usability***
  - ***Correctness***
  - ***…***



Q&A



1.2 *Concepts and Glossary*

1.2.1 Goals of testing

- Testing is a process of executing a program with the intent of finding an error.
- A good test is one that has a high probability of finding an as yet undiscovered error.
- A successful test is one that uncovers an as yet undiscovered error.
- The objective is to design tests that systematically uncover different classes of errors and do so with a minimum amount of time and effort.

- Demonstrate that software functions appear to be working according to specification.
- That performance requirements appear to have been met.
- Data collected during testing provides a good indication of software reliability and some indication of software quality.

Testing cannot show the absence of defects, it can only show that software defects are present.

1.2.2 Facts About Testing

- Question “does program P obey specification S” is undecidable!
- Every testing technique embodies some compromise between accuracy and computational cost

- Facts
  - *Inaccuracy is not a limitation of the technique, It is theoretically impossible to devise a completely accurate technique*
  - *Every practical technique must sacrifice accuracy in some way*

Why?

- The current development methods of software are:
  - Depend on people's intelligence
  - Specification is informal or even chaos
  - Software development tools are not self-contained
  - Software development process is elective and without standards on detail

在当前的软件开发方法学指导下开发软件，软件测试仍是保证软件质量和可靠性的重要手段。

Cost/benefit

- Testing takes more than 50% of the total cost of software development
  - *More for critical software*
- Software quality will become the dominant success criterion

The understanding of software test

- Exhausted test is impossible to realized.
- Software test can not ensure to uncover all errors of a software.
- Software test can not instead of other quality insurance methods

软件测试：巨大的效益和艰苦的工作并存



Some examples

1 Exception

```
if(state = = 0)
{   …
 }
if(state = = 1)
{…} ;
```

2 Data overflow

```
Int count( int &a, int month_num)
{ int  month_income = 0, i;
        for( i = 0; i< month_num; i++)
        month_income+=a[i];
        return month_income;
}

//1000<a[i]<10000
```



3 Beyond array boundary

```
void input(…)
{
…
  strcopy( stu_name,name)
…
}
//char stu_name[20];
```

4 Data is inaccurate

Every data type is limited by it's bits.

Such as float variable accurate is 7 bit, so 123456799.00 is equal to 123456711.00.

1.2.3 Glossary(术语)

- Error (ISO)(错误)
  - *A good synonym is mistake. Errors tend to propagate;*
- Fault(故障)
  - *A fault is the result of an error. Defect (see the ISTQB Glossary) is a good synonym for fault.*
- Failure (IEEE)(失效)
  - *The inability of a system or component to perform its required functions within specified performance requirements.*
- Incident(事件)

  —When a failure occurs, it may or may not be readily apparent to the user (or customer or tester). An incident is the symptom associated with a failure that alerts the user to the occurrence of a failure.

Glossary

- Exception (IEEE)(例外/越界)

– *An event that causes suspension of normal program operation. Types include addressing exception, data exception, operation exception, overflow exception, protection exception, underflow exception*

- Anomaly (IEEE)(异常)

– *Anything observed in the documentation or operation of software，that deviates from expectations based on previously verified software products or reference documents*

About “bug”

- Bug的起源
- Grace Hopper (1906--1992) （Grace Murray Hopper ） 计算机科学先驱，美国海军准将。
- 她还有一个重要的荣誉，就是抓住了计算机历史上第一个bug。这只真正的臭虫，如今还荣幸地存放在计算机档案馆里供人瞻仰。

1.2.4 The basic concepts of software test

- Basic principle of software test (**Myers** principle)
- Independent principle
- Completive(**完全性**) principle
- The types of software test

Basic principle of software test (Myers principle）

- Early test and continual test
- Avoid test software by programmer him/her self
- Test case = input data + expected result
- Test case should include expected（valid）input data and unexpected（invalid）input data
- Test program = what it should to do + what it should not do
- The probability of exist errors are direct proportion to the errors have being uncovered.
- Check each test result roundly
- Software test is an most innovational work

Independent principle (start)

- Software test should be execute by the group which is independent from development group.

交互测试

第三方测试

Completive principle

- Complete all test cases according to the test plan
- Use stated test case generate methods.
- Reach expected object (such as coverage criteria)
- Draw a curve for analyze, which is show the errors for different time period.
- Should uncover an expected number of errors.

The types of software test

—For process:

Walk through → Unit test → Integrate test → System test

—For technique :

Execution-based Verification/ Non-execution based Verification,

—For object:

Functional test, Performance test, Reliability test, Security test, Intensity(强度) test, Regression test

Steps of software test



- Design test plan
- Test case generate
- Establish test environment
- Test execute
- Evaluate test result
- Complete test documentation : test plan, test explain and test report

Test Phases

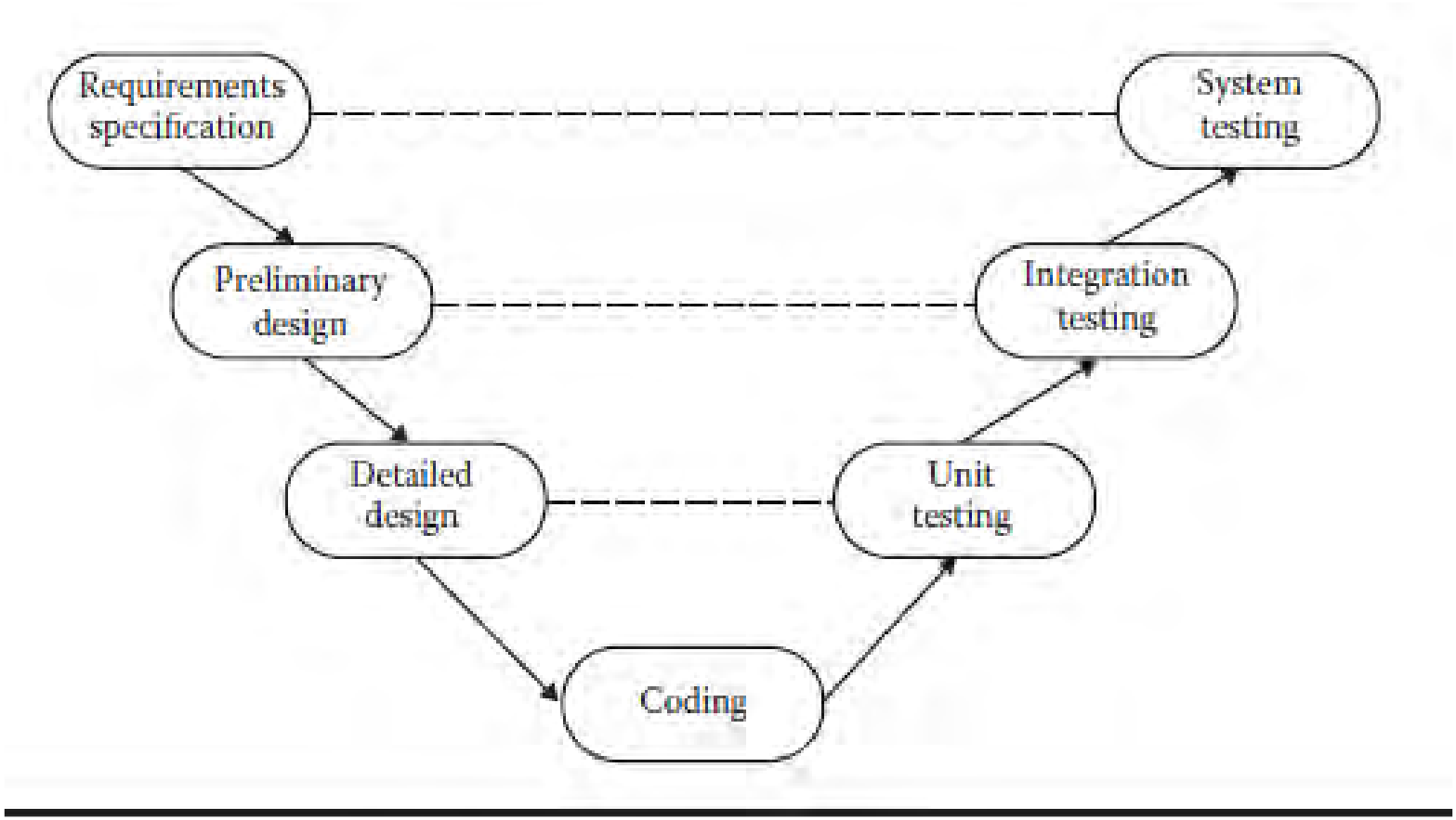
a. Walk Through（*Non-execution based Verification*）

b. Unit Test ;

c. Integrate Test;

d. System Test;

e. Acceptable Test ;


Requirements specification
System testing
Preliminary design
Integration testing
Detailed design
Unit testing
Coding

1.2.5 Fault Taxonomies

(read by students)

Table 1.1 Input/Output Faults

Type	*Instances*
Input	Correct input not accepted
	Incorrect input accepted
	Description wrong or missing
	Parameters wrong or missing
Output	Wrong format
	Wrong result
	Correct result at wrong time (too early, too late)
	Incomplete or missing result
	Spurious result
	Spelling/grammar
	Cosmetic

Table 1.2 Logic Faults

Missing case(s)
Duplicate case(s)
Extreme condition neglected
Misinterpretation
Missing condition
Extraneous condition(s)
Test of wrong variable
Incorrect loop iteration
Wrong operator (e.g., < instead of ≤)

Table 1.3 Computation Faults

Incorrect algorithm
Missing computation
Incorrect operand
Incorrect operation
Parenthesis error
Insufficient precision (round-off, truncation)
Wrong built-in function

Table 1.4 Interface Faults

Incorrect interrupt handling
I/O timing
Call to wrong procedure
Call to nonexistent procedure
Parameter mismatch (type, number)
Incompatible types
Superfluous inclusion

Table 1.5 Data Faults

Incorrect initialization
Incorrect storage/access
Wrong flag/index value
Incorrect packing/unpacking
Wrong variable used
Wrong data reference
Scaling or units error
Incorrect data dimension
Incorrect subscript
Incorrect type
Incorrect data scope
Sensor data out of limits
Off by one
Inconsistent data

Home works

- Read part one of textbook before week 4;
- Consist test group before week 4;
- Search some Java program suit for test before week 3;

Example: Calendar, Note, Calculate(simple model),small games, servers ……



Questions ?


SHANGHAI JIAO TONG UNIVERSITY
1896

- 1945 年 9 月 9 日，发生了一桩对计算机界而言非常重要的轶事。那天因为天气很热，他们都把窗户给打开了，然后飞进来一只蛾，结果那只蛾被打死在一支继电器里面，造成电路不通，让机器当机，使他们没办法算出他们要的结果。经过了近一天的检查，Hopper 找到了那只蛾，她用她的发夹去把那只蛾给弄出来，还把那只蛾的尸体贴在她的管理日志上，上面写着：「就是这个 bug，害我们今天的工作无法完成。」
- 这个消息传开之后，那个实验室里的人每逢老板询问为何还没做出结果时，都把过错推给 bug。尔后，在计算机界中，遇到程序中有错，就称之为 bug。除错叫做 debug。

return

Chapter 2

Prerequisites Mathematics knowledge of software test



2.1 Set Theory

2.2 Functions

2.3 Relations

2.4 Propositional Logic

2.5 Probability Theory

2.6 Graph Theory

2.7 Finite State Machines

2.8 Petri Net

2.9

2.1 Set Theory

- **Set Definition**

  A={elements list }; B={elements list };

  $a \in A$, $b \notin B$;

- **Set Operations:**

$A \cap B = \{x: x \in A \wedge x \in B\}$;

$A \cup B = \{x: x \in A \vee x \in B\}$.

$A' = \{x: x \notin A\}$;

$A-B=\{x \in A \wedge x \notin B\}$;

$A \oplus B=(A \cup B)-(A \cap B)$;

$A \times B=\{\langle x, y\rangle : x \in A \wedge y \in B\}$

- **Set Relations**

  A is a subset of B: $A \subseteq B$;

  A is a proper subset of B,: $A \subset B$;

  A=B: iff $A \subseteq B \wedge B \subseteq A$;

- **Set Partitions**

  Given a set A, and a set of subsets A_1, A_2, …, An of A, the subsets are a partition of A iff:

  $A_1 \cup A_2 \cup \cdots \cup A_n = A$,

  and $i \neq j \Rightarrow A_i \cap A_j = \varnothing$.

2.2 Functions

- **Definition**

Given sets A and B, a function f is a subset of A × B such that for

$$a_i, a_j \in A, \quad b_i, b_j \in B, \text{ and } f(a_i)=b_i, \quad f(a_j)=b_j,$$

$$\text{If } b_i \neq b_j \Rightarrow a_i \neq a_j$$

or $f: A \rightarrow B$

$f \subseteq A \times B$

- **Function Types**

f is a function from A onto B iff f(A) = B (上函数)

f is a function from A into B iff f(A) ⊂ B (note the proper subset here! 中函数)



f is a one-to-one function from A to B iff,
for all ai, aj $\in$ A, ai $\neq$ aj $\Rightarrow$ f(ai) $\neq$ f(aj)

f is a many-to-one function from A to B iff,
there exists ai, aj $\in$ A, ai $\neq$ aj such that f(ai) = f(aj).

2.3 Relations

- **Definition:**

Given two sets A and B, a relation R is a subset of the Cartesian product $A \times B$.

A relation $R \subseteq A \times A$ is

Reflexive： iff for all $a \in A$, $\langle a, a \rangle \in R$

Symmetric ： iff $\langle a, b \rangle \in R \Rightarrow \langle b, a \rangle \in R$

Antisymmetric： $\langle a, b \rangle, \langle b, a \rangle \in R \Rightarrow a = b$

Transitive： iff $\langle a, b \rangle, \langle b, c \rangle \in R \Rightarrow \langle a, c \rangle \in R$：

A relation $R \subseteq A \times A$ is an ordering relation if R is reflexive, antisymmetric, and transitive.

Fore xample: $\geqslant$

A relation $R \subseteq A \times A$ is an equivalence relation if R is reflexive, symmetric, and transitive.

Fore xample:

(classmates, roommates)

2.4 Propositional Logic

- **Definition:**

A proposition is a sentence that is either true or false.

For example：6>2、Earth is a star；

but：water is cold, The mountain is very high, are not propositions.

- **Logical Operators**

and ： $\wedge$

or ： $\vee$

not ： $\neg$

exclusive–or ： $\oplus$

IF-THEN： $\rightarrow$

p	q	$p \oplus q$	$p \rightarrow q$
T	T	F	T
T	F	T	F
F	T	T	T
F	F	F	T

2.5 Probability Theory

- **Definition**

The probability that a proposition p is true, denoted Pr(p), is

$$|T(p)|/|U|.$$

T(p), is the set of all elements in the universe U for which p is true.

For example：一个袋子中有10个红球，10个兰球；从中任取一个球，是红球的几率？

再取一个是红球的几率？

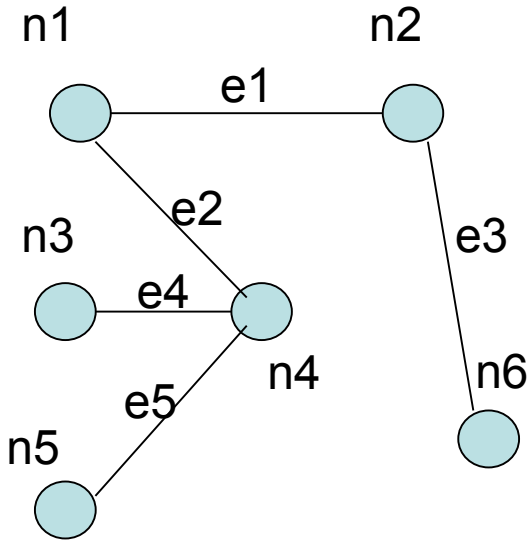
连续2个是红球的几率？

2.6 Graph Theory

- **Definition**

A graph G = (V, E) is composed of a finite (and nonempty) set V of nodes and a set E of unordered pairs of nodes.

Example 1：



V={n1,n2,n3,n4,n5};

E={e1,e2,e3,e4,e5}

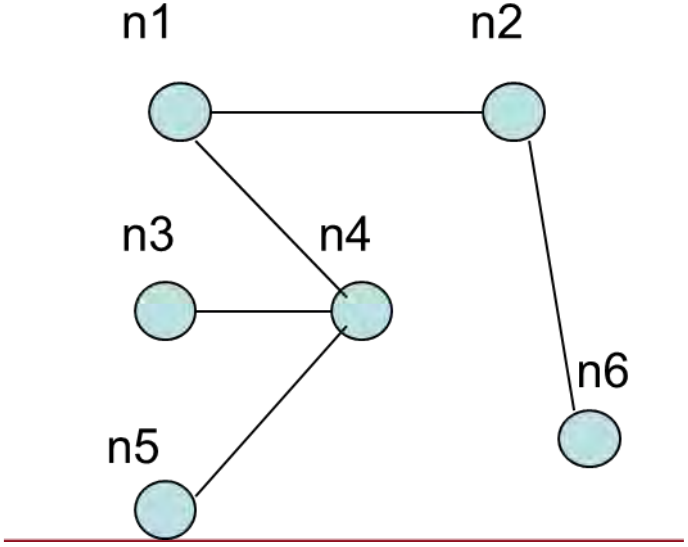
={(n1,n2),(n1,n4),(n2,n6),(n3,n4),(n4,n5)};

- degree of a node: the number of edges that have that node as an endpoint.

deg(n1)=2, deg(n2)=2,…deg(n6)=1

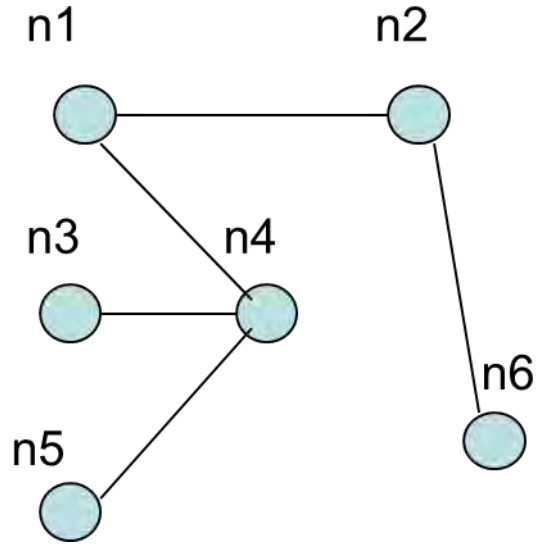
- **Incidence Matrices**：Incidence Matrices：with m nodes and n edges is an m × n matrix, where the element in row i, column j is a 1 if and only if node i is an endpoint of edge j; otherwise, the element is 0.

	e1	e2	e3	e4	e5
n1	1	1	0	0	0
n2	1	0	1	0	0
n3	0	0	0	1	0
n4	0	1	0	1	1
n5	0	0	0	0	1
n6	0	0	1	0	0



- **Adjacency Matrices**: The adjacency matrix of a graph G = (V, E) with m nodes is an m × m matrix, where the element in row i, column j is a 1 if and only if an edge exists between node i and node j; otherwise, the element is 0.

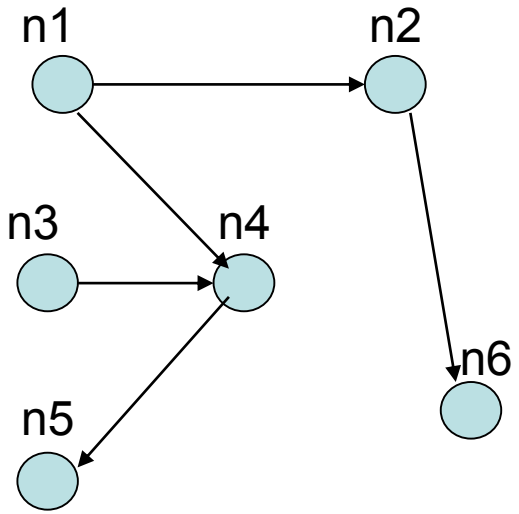
	n1	n2	n3	n4	n5	n6
n1	0	1	0	1	0	0
n2	1	0	0	0	0	1
n3	0	0	0	1	0	0
n4	1	0	1	0	1	0
n5	0	0	0	1	0	0
n6	0	1	0	0	0	0



- Path：a sequence of edges such that for any adjacent pair of edges ei, ej in the sequence, the edges share a common (node) endpoint.
- Connectedness:Two nodes are connected if and only if they are in the same path.
- component of a graph: a maximal set of connected nodes.
- Directed Graphs
- Indegrees
- Outdegree

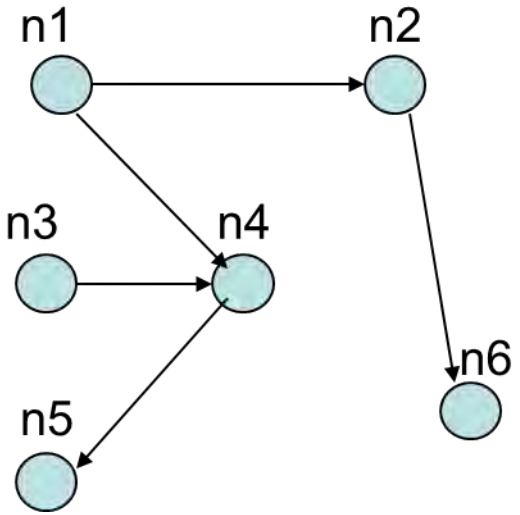
indeg(n1) = 0 outdeg(n1) = 2

indeg(n2) = 1 outdeg(n2) = 1



Adjacency Matrix of a Directed Graph

	n1	n2	n3	n4	n5	n6
n1	0	1	0	1	0	0
n2	0	0	0	0	0	1
n3	0	0	0	1	0	0
n4	0	0	0	0	1	0
n5	0	0	0	0	0	0
n6	0	0	0	0	0	0



• Paths and Semipaths

(directed) path： a sequence of edges such that, for any adjacent pair of edges ei, ej in the sequence, the terminal node of the first edge is the initial node of the second edge.

Cycle：a directed path that begins and ends at the same node.

Chain：a sequence of nodes such that each interior node has indegree = 1 and outdegree = 1.

(directed) semipath：a sequence of edges such that for at least one adjacent pair of edges ei, ej in the sequence, the initial node of the first edge is the initial node of the second edge or the terminal node of the first edge is the terminal node of the second edge.

For example:

A path from n1 to n6； A semipath between n1 and n3；

A semipath between n2 and n4； A semipath between n5 and n6



- **n-Connectedness**

0-connected: iff no path exists between ni and nj.

1-connected: iff a semipath but no path exists between ni and nj.

2-connected: iff a path exists between ni and nj.

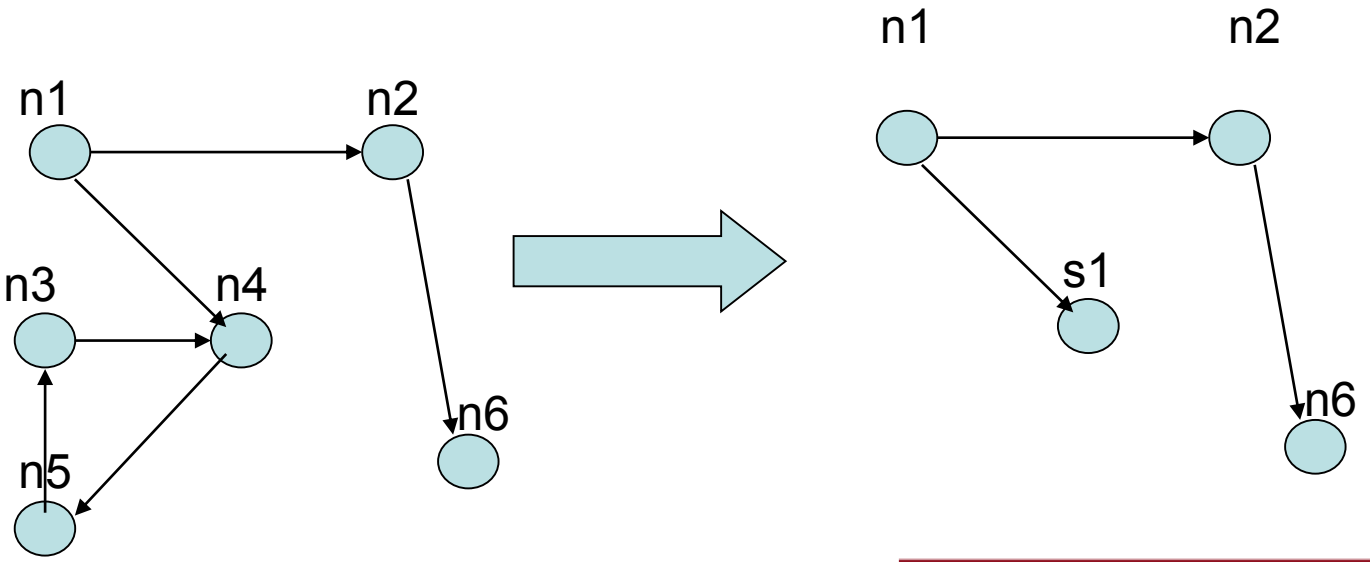
3-connected: iff a path goes from ni to nj and a path goes from nj to ni.

- **Strong Components**

A strong component of a directed graph is a maximal set of 3-connected nodes.

For example:

Strong Components are {n3,n4,n5}, we could removing loops and isolated nodes.





The large number of execution paths comes from nested loops. Condensation graphs eliminate loops (or at least condense them down to a single node); therefore, we can use this as a strategy to simplify situations that otherwise are computationally untenable.

- **Program Graphs（omit）**

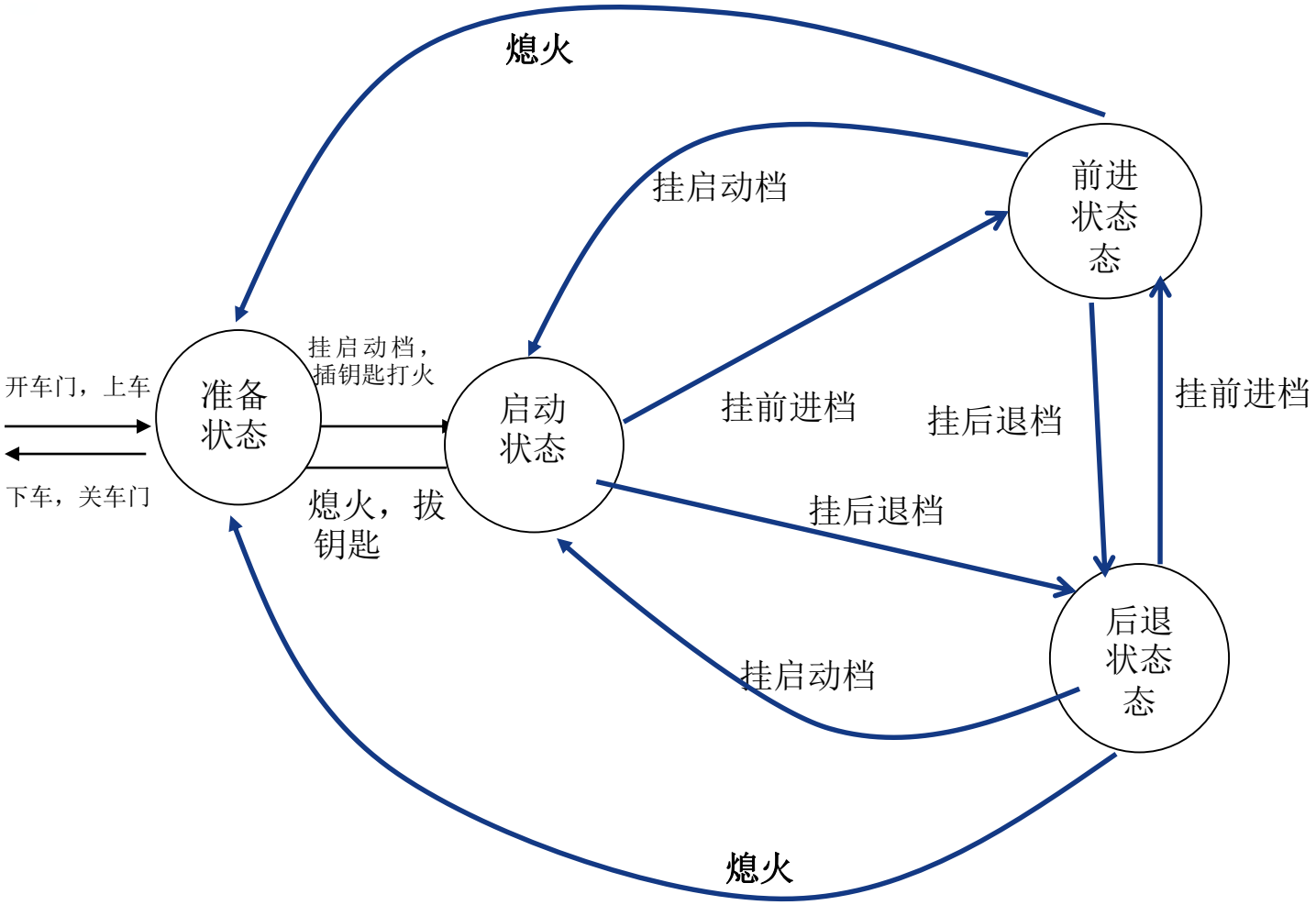
2.7 Finite State Machines

有限（穷）状态机是形式化描述的有效方法，在计算机调度、管理中得到了广泛的应用。

我们通过一个简单的例子来说明有穷状态机的基本概念。

司机驾驶汽车，有以下状态。打开车门；车挡在启动档，插钥匙，打火发动汽车；挂前进档，汽车前行；挂后退档，汽车后退；挂启动档，熄火，拔钥匙；下车关门。

当然，油门和刹车也是两个重要的动作，一个是为了提速，一个是为了减速，但它们都不影响汽车的状态，这里予以简化。空档是一个过渡状态，为了简洁，也予以简化。


熄火
挂启动档
前进状态态
挂启动档，插钥匙打火
开车门，上车
准备状态
启动状态
挂前进档
挂后退档
挂前进档
下车，关车门
熄火，拔钥匙
挂后退档
后退状态态
挂启动档
熄火

状态转换图在实际设计中往往被状态转换表所代替，下表是汽车驾驶的状态转换表。

当前状态 / 状态 / 执行动作	准备状态	启动状态	前进状态	后退状态
（挂启动档）打火	启动状态	启动状态	无效	无效
挂前进档	准备状态	前进状态		前进状态
挂后退档	准备状态	后退状态	后退状态	
挂启动挡	准备状态	启动状态	启动状态	启动状态
熄火	准备状态	准备状态	准备状态	准备状态

汽车驾驶的状态转换表

从上面的例子我们可以看到，一个有穷状态机包括以下5个部分：状态集J、输入（动作）集K、状态转换函数（转换表）T、初始态S和终止态F。对于上面的例子有穷状态机的各个部分如下：

- 状态集J：{准备状态，启动状态，前进状态，后退状态}；
- 输入集K：{打火，熄火，挂前进档，挂后退档，挂启动档}；
- 转换函数T：如表13-1所示；
- 初始态S：{准备状态}；
- 终止态F：{准备状态}；

由上面的说明，我们可以将有穷状态机表示为一个5元组（J，K，T，S，F），其中：

- J 是一个有穷的非空状态集；
- K是一个有穷的非空输入集；
- T是一个从（J-F）×K到J的转换函数；
- S ∈J，是唯一的初始态；
- F J，是若干个终止态。

有穷状态机在计算机系统中应用的十分广泛。如操作系统中的作业调度与管理，GUI图形用户界面的驱动等。

2.8 Petri Net 说明

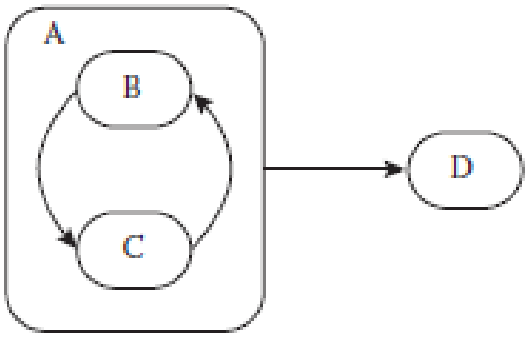
- Petri Net 的介绍与第11讲 交互测试密切关联，因此放在第11讲介绍。

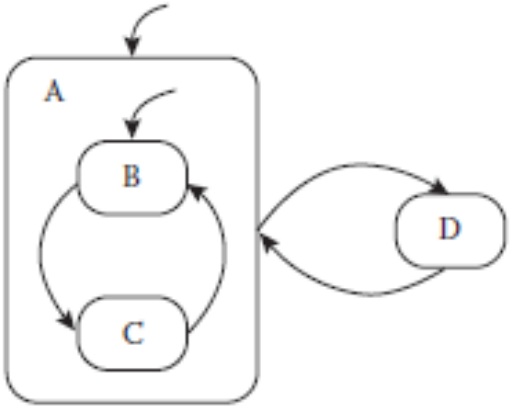
2.9 State Charts

the StateChart notation: to devise a visual notation that combined the ability of Venn diagrams to express hierarchy and the ability of directed graphs to express connectedness (Harel, 1988).

If a state contains substates, as state A does, the edge “refers” to all substates.

Thus, the edge from A to D means that the transition can occur either from state B or from state C.





其它

对于软件测试而言，并不是仅仅需要本节介绍的数学知识。

形式化描述、数学验证、数据结构与算法等都是可能用到的数学。

在本课程中，将以软件测试的实践为主，不再过多介绍数学的知识。

作业说明

- 教材第二章需要自己阅读。
- 下周课程的内容
- Java
- Eclipse
- JUNIT
- EMMA

书上的7个例子程序需要自己熟悉！

Q&A



Chapter 3

Fundamental theory and methods of software test



3.1 Non-execution based Verification

3.2 Execution based Verification

3.3 Formal verification

3.4 Other methods



3.1 Non-execution based Verification

- Not execute the program, tester could use some tools to review and analyze the specification and program code.

- walk through
- Inspection



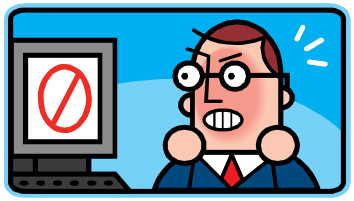
3.1.1 Walk through

- Consist a walk through test group
- Check program code logic
- Generate test case, include input data and expect result.
- Put the data into program code, if the calculate result unequal the expect result, find an error.
- Check the code line by line.

a member of the SQA



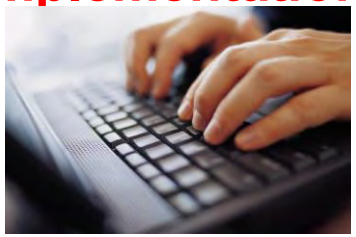
a representative of next step



a client representative



one or more representative of implementation



The walkthrough team should consist of four to six individuals



- The goal of the walkthrough team is to detect faults, not to correct them.
- The person leading the walkthrough guides the other members of the team through the code.
- The walkthrough can be driven by the lists of issues compiled by team members or by the code itself, with team members raising their concerns at the appropriate time.
- In both cases, each issue will be discussed as it comes up and resolved into either a fault that needs to be addressed or a point of confusion that will be cleared up in the discussion

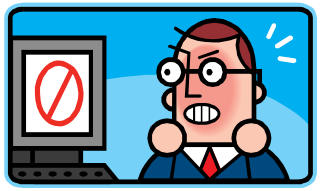
3.1.2 Inspection

- An inspection is a far more formal activity than a code walkthrough.
- It is conducted by a team of three to six people

a moderator



a representative of next step



one or more representative of implementation



a recorder





The review process consists of five formal steps:

- In the overview step, the author of the module gives a presentation to the team.
- In the preparation step, the participants try to understand the code in detail and compile lists of issues, ranked in order of severity(严重性). They are aided in this process by a checklist of potential faults for which to be on the lookout.
- In the inspection step, a thorough walkthrough of the code is performed, aiming for fault detection through complete coverage of the code. Within a day of the inspection, the moderator produces a meticulous（小心翼翼） written report.
- In the rework step, the individual responsible for the code resolves all faults and issues noted in the written report. （**Individual Reviewer Issues Spreadsheet**）
- In the follow-up step, the moderator must make sure that each issue has been resolved by either fixing the code or clarifying confusing points. （**Review Report Spreadsheet**）

Individual Reviewer Issues Spreadsheet

<Work product information>					
<Reviewer Name>					
<Preparation Date>					
<Reviewer Preparation Time>					
	Location		Checklist		
Issue #	Page	Line	Item	Severity	Description
1	1	18	Typo	1	Change "accound" to "account"

Review Report Spreadsheet

<Work Product Information>						
Review Team Members		*Preparation Time*				
Leader						
Recorder						
Reviewer						
Reviewer						
Reviewer						
Reviewer						
	Total prep time					
Meeting date						
<Review Recommendation>						
		Location		Checklist		
Issue #	*Reviewer*	*Page*	*Line*	*Item*	*Severity*	Description
1		1	18	Typo	1	Change "accound" to "account"



- An important product of an inspection is the number and kinds of faults found rated by severity.（Here is a sample outline of a review report）
- If a module comes through an inspection exhibiting a significantly larger number of faults than other modules in the system, it is a good candidate for rewriting.
- If the inspection of two or three modules reveals a large number of errors of specific types, this may warrant (re)checking other modules for similar errors.
- If more than 5 percent of the material inspected must be reworked, the team must reconvene for a full re-inspection.



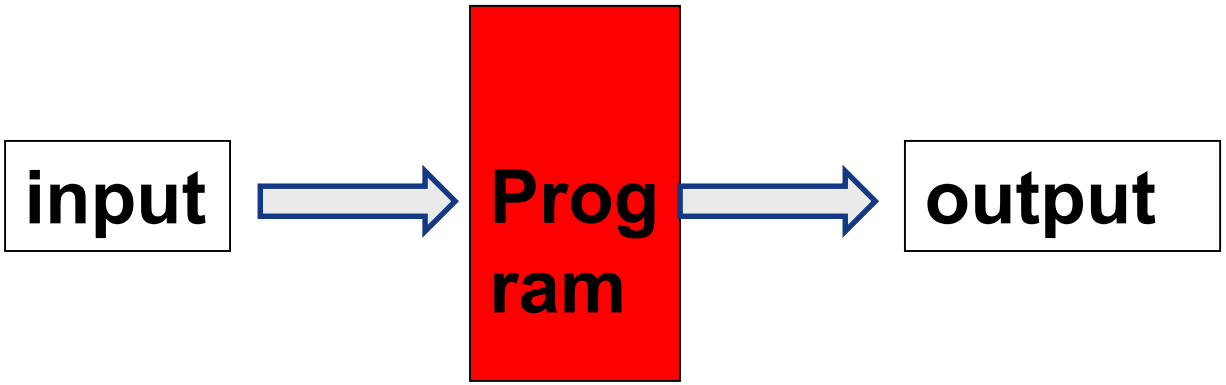
3.2 Execution-based Verification

- Use test cases to execute the program, get all results from the execution.

  There are two basic approaches to testing modules, each with its own weaknesses.

  - **Testing to Specifications, also known as black-box test.**
  - **Testing to Code, also called glass-box, white-box test.**

3.2.1 Black-box test

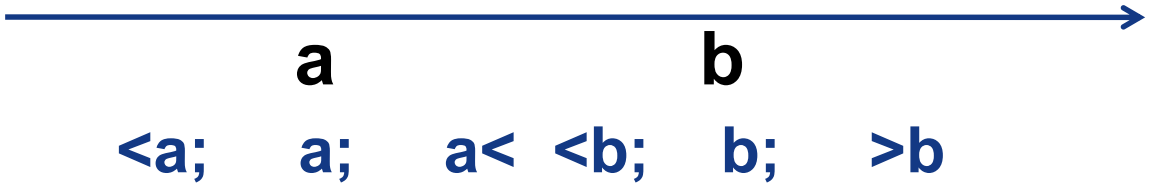


- Seeks to verify that the module conforms to the specified input and output while ignoring the actual code
- It is rarely possible to test all modules for all possible input cases, which may be a huge number of modules



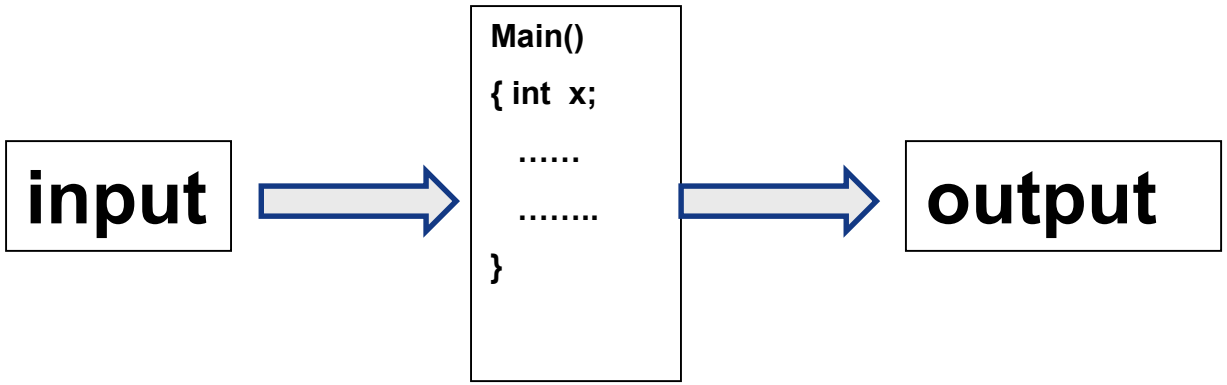
- Equivalence testing **is a technique based on the idea that the input specifications give ranges of values for which the software product should work the same way.**
- Boundary value analysis **seeks to test the product with input values that lie on and just to the side of boundaries between equivalence classes.**

For example the input is (a,b), there are at least 5 equivalence classes:



functional testing, the tester identifies each item of functionality or each function implemented in the module and uses data to test each function.

3.2.2 White-box test



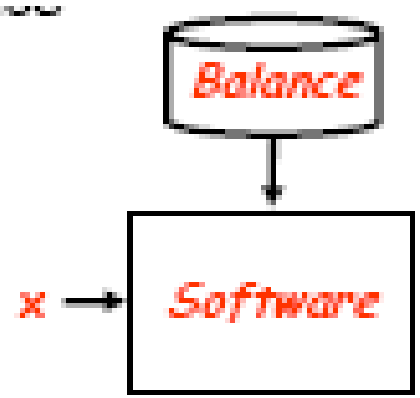
- Also called logic-driven, or path-oriented testing.
- testing each path through the code is generally not feasible, even for simple flowcharts.
- Moreover, it is possible to test every path without finding existing faults, because, for example, the fault lies in the decision criterion for selecting between paths.



- statement coverage **and amounts to running tests in which every statement in the code is executed at least once.**
- branch coverage**, makes sure that each branching point is tested at least once.**
- path coverage, **make sure that every different path is tested at least once.**

Example : black-box testing

- Discussion: MAC/ATM machine
  - Specs
- **Cannot withdraw more than $300**
- **Cannot withdraw more than your account balance**



Test case?

X

1. >300
2. 0~300
3. <0

Balance

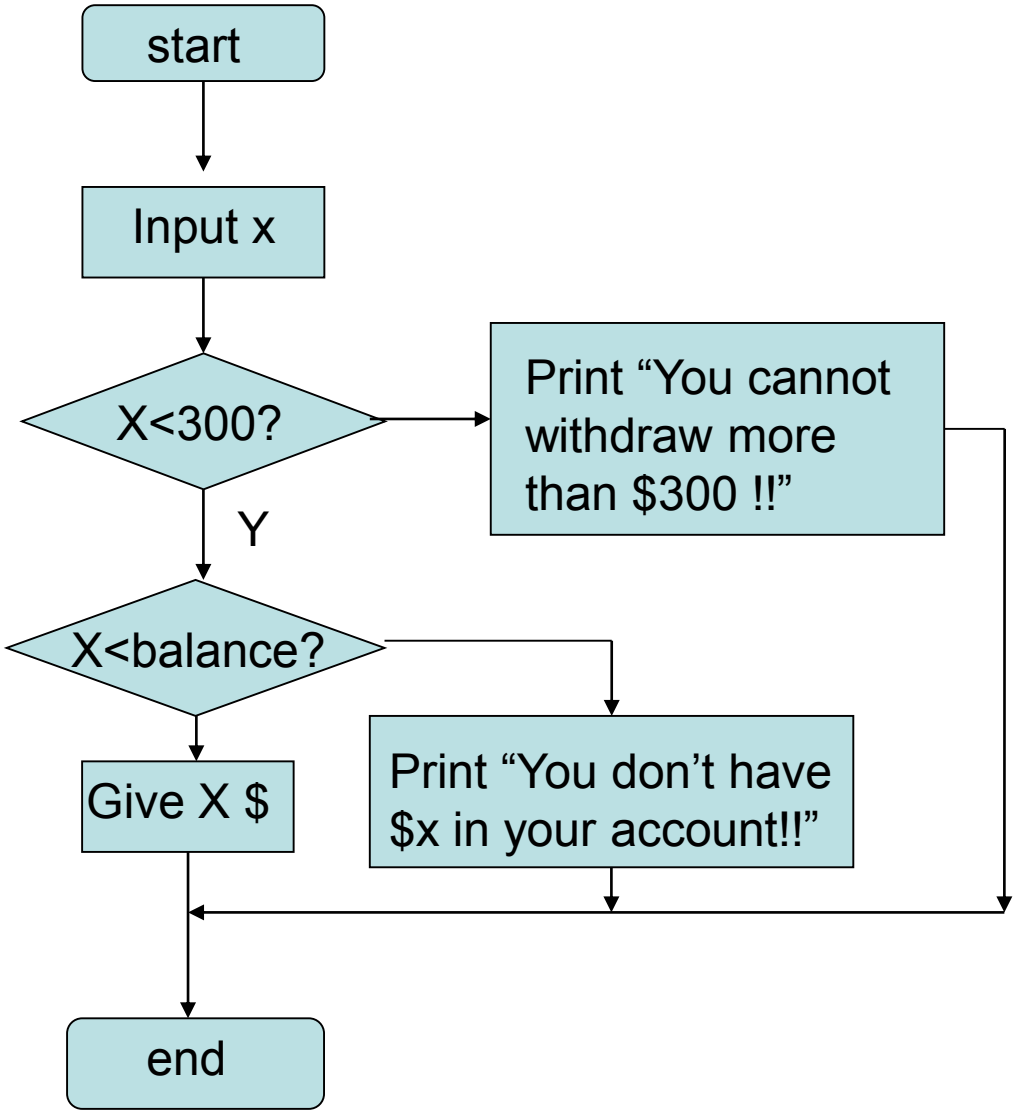
1. = >300,
2. 0~300

White-box Testing

- Example

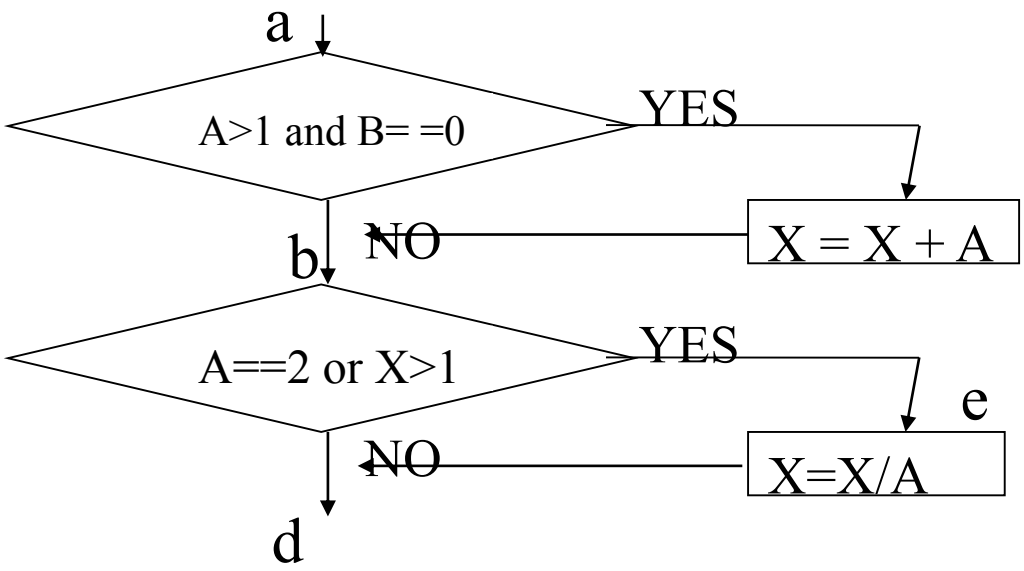
x: 1..1000;

```
1   INPUT-FROM-USER(x);
      If (x <= 300) {
2        INPUT-FROM-FILE(BALANCE);
           If (x <= BALANCE)
3               GiveMoney x;
4          else Print "You don't have $x in your account!!"}
       else
5      Print "You cannot withdraw more than $300";
6    Eject Card;
```


start
Input x
X<300?
Print “You cannot withdraw more than $300 !!”
Y
X<balance?
Print “You don’t have $x in your account!!”
Give X $
end

Test case?

How about example2 ?





For example2, branch coverage is: point A, should have A>1, A≤1, B=0, B≠0, point B, should have A=2, A≠2, X>1, X≤1。

so {2, 0|2, 5} and {0, 1|0, 0} is enough.

How about path coverage?

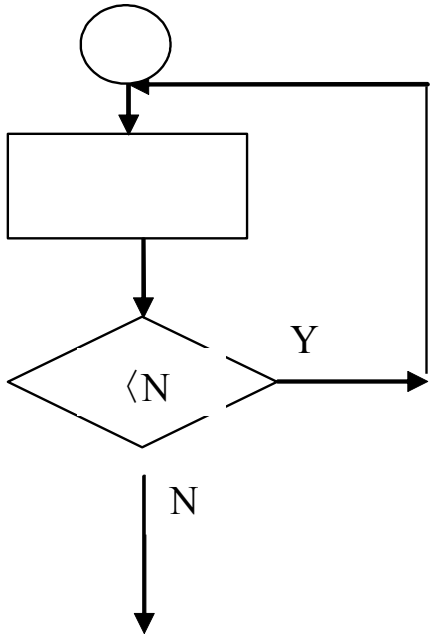
4

Predicates Test case are: 2x2x2x2=16

- How about loop ?
- Branch coverage is two,
- but path coverage is N+1!

Does coverage guarantee absence of faults?

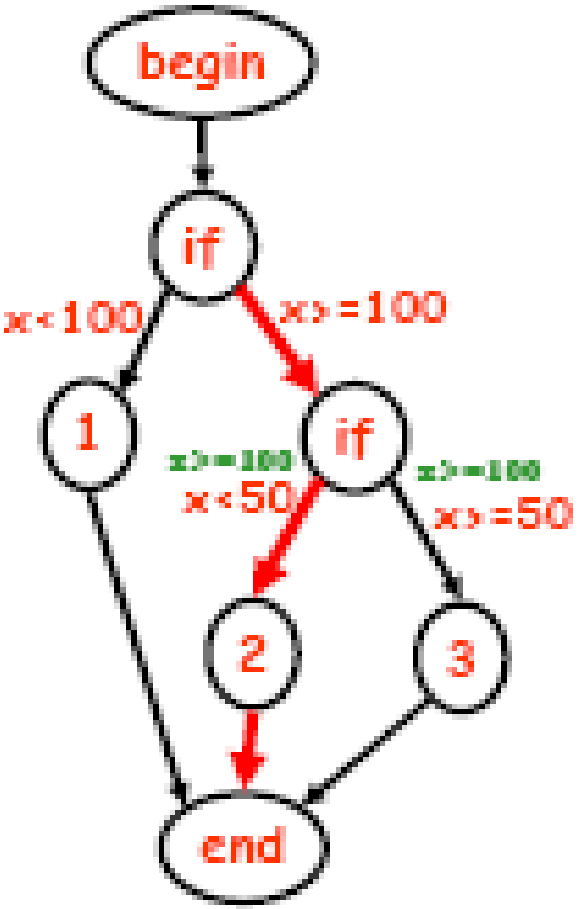
- Can we always get 100% coverage?



Surprise Quiz

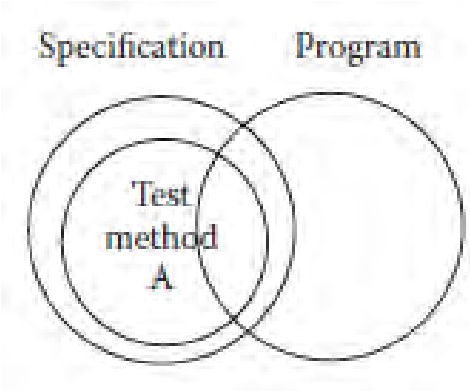
- Determine test cases so that each print statement is executed at least once

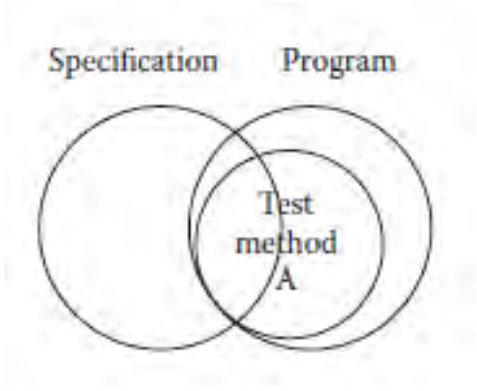
```
input(x);
if (x < 100)
print “Line 1”;
else {
if (x < 50) print “Line 2”
else print “Line 3”;
}
```



Question

- What is the weaknesses of black-box test?
- What is the weakness of white-box test?





3.3 Formal verification

- Correctness proofs

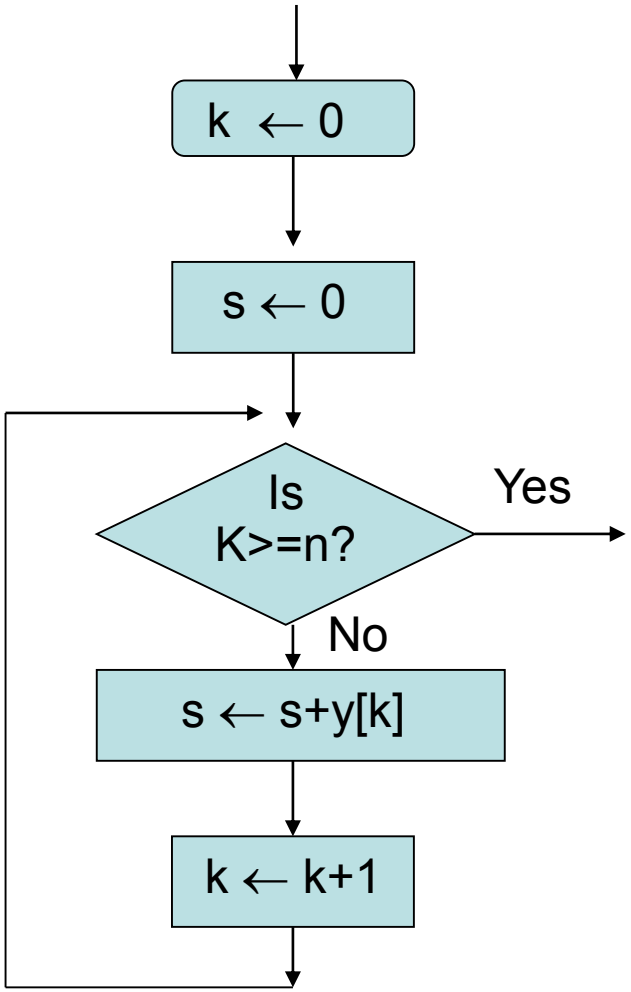
It is a mathematical technique for show that a product is correct, in other word, that it satisfies its specifications.

To see how correctness is proven, consider the fragment (next slide)

We now show that the code fragment is correct- after execute, the variable s will contain the sum of the n elements of array y.

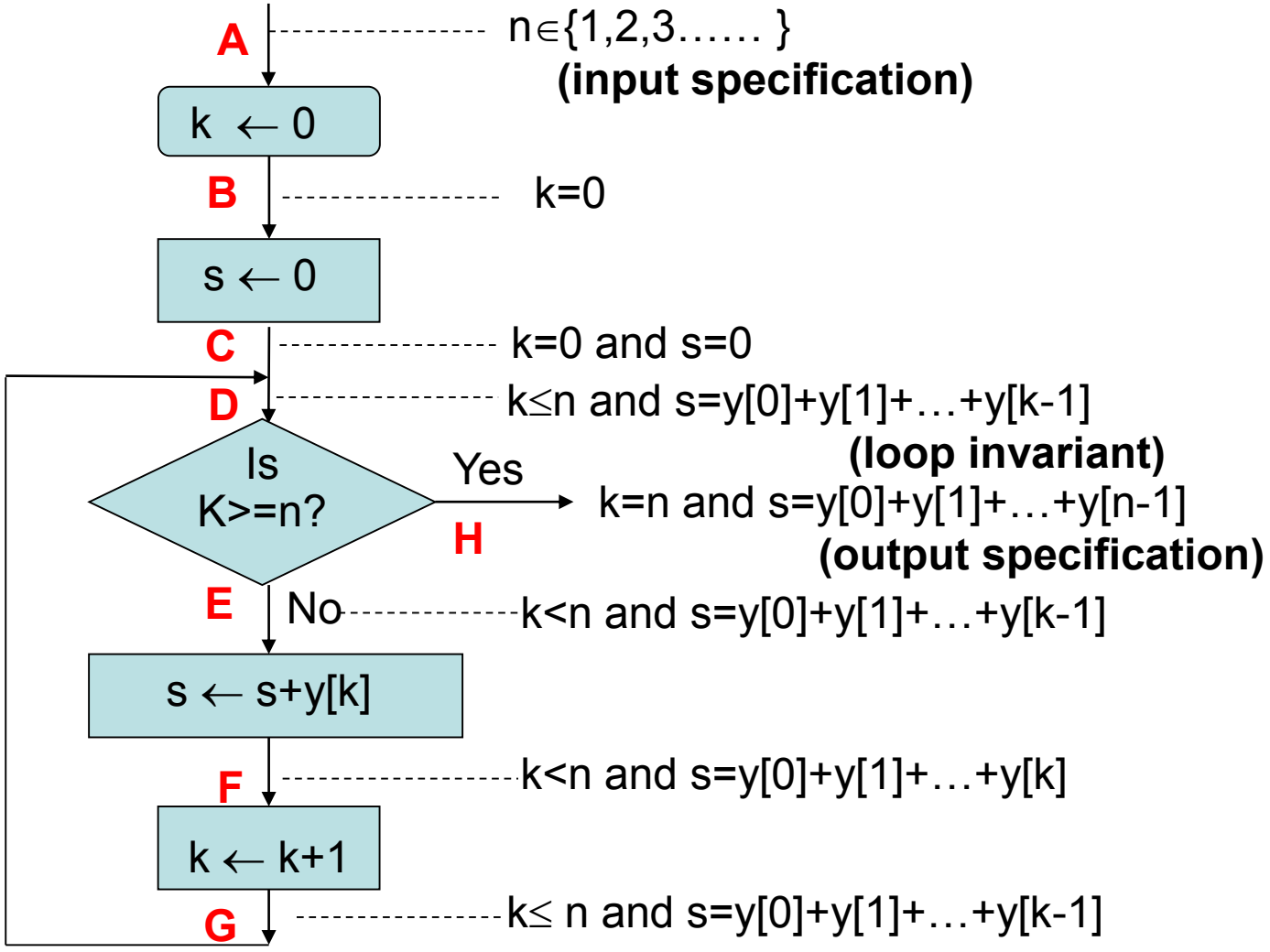
```
Int k,s;
Int y[n];
。 。 。 。 。 。
k=0;
s=0;
While (k<n)
{
   s=s+y[k];
   k=k+1;
 }
```

Code fragment



Flowchart

An assertion is placed before and after each statement, at the place labeled with the letters A through H;



Input specification, n is a positive integer,

A: $n \in \{1,2,3\ldots\ldots\}$ (1)

Output specification is that if control reaches point H, the value of ***s*** contains the sum of ***n*** values stored in array ***y***,

H: $s = y[0]+y[1]+\ldots+y[n-1]$ (2)

A stronger output specification is,

H: $k=n$ and $s = y[0]+y[1]+\ldots+y[n-1]$ (3)

How proven from A to H? key is proven the invariant of the loop, that is,

D: $k \leq n$ and $s=y[0]+y[1]+\ldots+y[k-1]$ (4)

First B: $k=0$ (5)

This is easy to prove, because of assignment statement

$k \leftarrow 0$ is executed.

At point C, the second assignment statement $s \leftarrow 0$ is executed, the follow assertion is true:

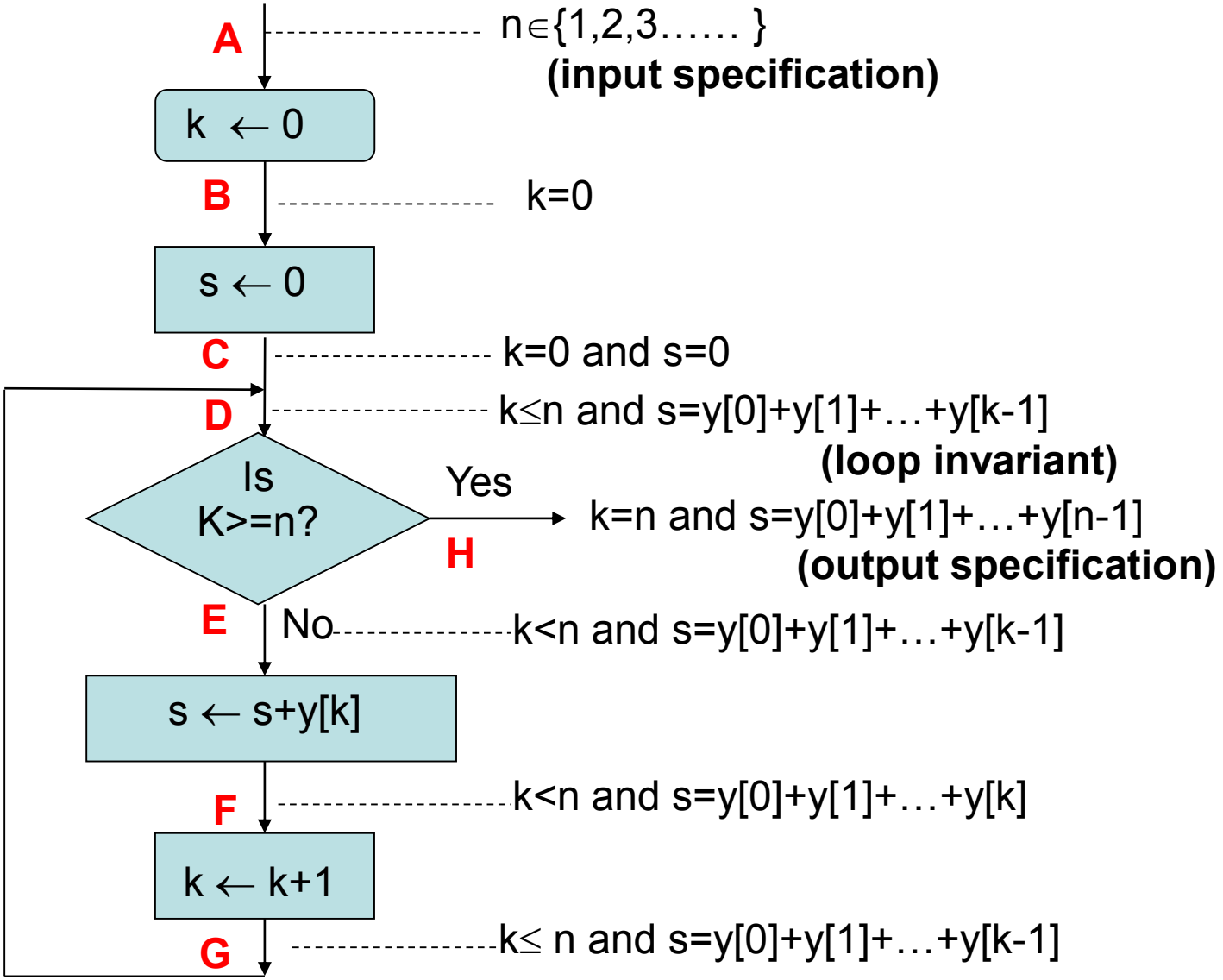
$$k=0 \text{ and } s=0 \qquad (6)$$

Now the loop is entered. Before the loop execute, assertion(6) holds k=0 and s=0. By assertion(1), $n \geq 1$. so it follows that $k \leq n$ is required. Because k=0, it follows k-1=-1, so the sum in (4) is empty s=0. there for loop invariant (4) is true just before the first loop is entered.

Now, assume that, at some stage during the execution of the code fragment, the loop invariant holds. That is k to some value k_0, $0 \leq k_0 \leq n$,

$$D: \quad k_0 \leq n \text{ and } s=y[0]+y[1]+\ldots+y[k_0-1] \qquad (7)$$

We should to prove execute the code fragment, the loop invariant is still true or get the output specification.


A
n∈{1,2,3…… }
(input specification)
k ← 0
B
k=0
s ← 0
C
k=0 and s=0
D
k≤n and s=y[0]+y[1]+…+y[k-1]
(loop invariant)
Is
K>=n?
Yes
k=n and s=y[0]+y[1]+…+y[n-1]
H
(output specification)
E
No
k<n and s=y[0]+y[1]+…+y[k-1]
s ← s+y[k]
F
k<n and s=y[0]+y[1]+…+y[k]
k ← k+1
G
k≤ n and s=y[0]+y[1]+…+y[k-1]

Pass to the test box. If $k_0 \geq n$, then because $k_0 \leq n$ by hypothesis, it follows that $k_0 = n$, this implies that

$$H: K_0 = n \text{ and } s = y[0] + y[1] + \ldots + y[n-1] \quad (8)$$

Just the output specification(3).

On the other hand, $k_0 < n$, and (7), becomes

$$E: \quad k_0 < n \text{ and } s = y[0] + y[1] + \ldots + y[k_0 - 1] \quad (9)$$

Just the assertion of E, then the statement $s \leftarrow s + y[k_0]$ is executed, the point F is

$$F: \quad k_0 < n \text{ and } s = y[0] + y[1] + \ldots + y[k_0 - 1] + y[k_0]$$
$$= y[0] + y[1] + \ldots + y[k_0] \quad (10)$$

The next statement to be executed is $k_0 \leftarrow k_0 + 1$. then because before increasing $k_0 < n$, now $k_0 \leq n$. and (10) becomes

$$G: \quad k_0 \leq n \text{ and } s = y[0] + y[1] + \ldots + y[k_0 - 1] \quad (11)$$

Assertion (11) is equal to assertion (7). That by assumption, holds at point D.

Now we have proved that $0 \leq k \leq n$, the invariant (4) is true .

Last prove is the terminates of the loop. Since the initial k is 0, and each iteration the loop increases the k by 1, surely k will reaches n, k=n. Then by assertion (8), got the output specification (3).

From the example, we could know, that only a 5 lines simple codes, the correctness proofs need 5 pages!

So for a more large program code, it is impossible to use correctness proofs for all codes. Even it is the most reliable method.

3.4 Other methods

- Def-use test
- Mutation test
- Regression Test
- Fault Statistics and Reliability Analysis
- Clean room



3.4.1 Def-use test

– Data-flow based adequacy criteria

- **All definitions criterion**
  - ***Each definition to some reachable use***
- **All uses criterion**
  - ***Definition to each reachable use***
- **All def-use criterion**
  - ***Each definition to each reachable use***

3.4.2 Mutation test

- Error seeding
  - *Introducing artificial faults to estimate the actual number of faults*
- Program mutation testing
  - *Distinguishing between original and mutants*
    - **Competent programmer assumption**
      - **Mutants are close to the program**
    - **Coupling effect assumption**
      - **Simple and complex errors are coupled**



3.4.3 Regression Testing

- Developed first version of software
- Adequately tested the first version
- Modified the software; Version 2 now needs to be tested
- How to test version 2?

- Approaches
  - *Retest entire software from scratch*
  - *Only test the changed parts, ignoring unchanged parts since they have already been tested*
  - *Could modifications have adversely affected unchanged parts of the software?*



Regression Testing

- “Software maintenance task performed on a modified program to instill confidence that changes are correct and have not adversely affected unchanged portions of the program.”

3.4.4 Fault Statistics

- Fault statistics **provide a useful metric for determining whether to continue testing a module of product or whether to recode it.**
- **The number of faults detected via execution- and non-execution-based techniques must be recorded.**
- **Data on different types of faults found via code inspections (faults such as misunderstanding the design, initializing improperly, and using variables inconsistently) can be incorporated into checklists for use during later reviews of the same product and future products.**



Fishing Creel Counts and Fault Insertion

For example, that the Rogue River is stocked with 1000 marked trout(鳟鱼)；and that during the fishing season, the creel report totals are 500 fish caught, of which 300 are marked trout.

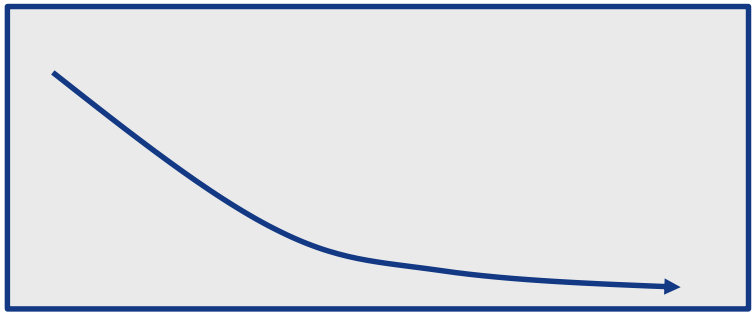
The stream management team would conclude that 60% of the trout in the Rogue River are hatchery (planted) fish. The total populations could also be estimated;

in this case, the 500 fish caught represent 30% of the total population. Total fish population is 1667.

The same idea can be applied to estimate the success of test cases and the number of remaining faults not caught by an existing set of test cases.

3.4.5 Reliability Analysis

- Reliability analysis **uses statistical-based techniques to provide estimates of how many faults are remaining and how much longer it is desirable to keep testing.**
- **It can also be applied in both implementation and integration phases.**





3.4.6 Clean-room

The *Clean-room technique* is a combination of several different software development techniques.

Under this technique, a module isn't compiled until it has passed an inspection, or another kind of non-execution-based review, such as a code walkthrough or inspection.

The relevant metric is testing fault rate**, which is the total number of faults detected per KLOC (thousand lines of code).**



The Cleanroom technique has had considerable success in finding and weeding out faults before execution-based testing.

In one case, all faults were found via correctness-proving techniques, using largely informal proofs, but with a few formal proofs as well. The resulting code was found to be free of errors both at compilation and at execution time.

In a study of 17 other Cleanroom software products, the products did not perform quite as faultlessly, but they still achieved remarkably low fault rates, an average of 2.3 faults per KLOC.

But average there are 25 faults per KLOC!

Summary

1. *Non-execution based Verification*
   - *walkthrough*
   - *inspection*
2. *Execution based Verification*
   - *black-box*
   - *white-box*
2. *Formal verification*
3. *Other methods*
   - **Def-use test**
   - **Mutation test**
   - **Regression Test**
   - **Fault Statistics and Reliability Analysis**
   - **Clean room**

测试计划实例与作业一

XXXX支撑软件与可视化软件

测试计划

目录

1、简介
1.1编写目的
1.2项目背景
1.3测试范围
1.4参考文档
2、测试参考文档和测试提交文档
2.1测试参考文档
2.2测试提交文档
3、测试进度
4、测试资源
4.1人力资源
4.2测试环境
4.3测试工具
5、测试策略



5.1功能测试
5.1.1 XXX系统管理软件功能测试
5.1.2 XXX系统实时通信软件功能测试
5.1.3目标三维动态显示模型及可视化软件功能测试
5.1.4XXX三维动态显示模型及可视化软件功能测试
5.1.5 XXX交会显示软件功能测试
5.1.6XXX系统数据管理软件功能测试
5.1.7 XXX文件与AutoCAD、Creator的转换软件功能测试
5.2性能测试
5.2.1 通讯性能测试
5.2.2 实验性能测试
5.2.3 数据库性能测试
5.2.4 显示性能测试
5.2.5 其它性能测试
5.3故障测试
5.3.1网络故障
5.3.2 服务器故障
5.3.3 管理计算机故障
5.3.4 计算节点机故障
5.3.5 图形工作站故障测试
5.4安全性测试
5.4.1 管理软件安全性测试
5.4.2 数据管理软件安全性测试

1、简介

- 1.1编写目的
- 为了全面、系统地对“XXX支撑软件与可视化软件”进行评估与测试，从而保证系统长期稳定的运行，组织对该软件进行系统的总体综合测试。
- 1.2项目背景
- “XXX验证平台”是中国兵器工业XXX研究所承担的国防基础科研项目。XXX所与XXX大学合作，对该项目的关键技术——XXX支撑技术和可视化技术展开研究。本项目就是在此背景下开展的，通过应用虚拟试验技术、计算机网络技术、可视化技术、数据库技术等，研制“XXX的支撑软件和可视化软件”，并最终集成在“XXX中。
- 本软件系统由以下7个二级软件组成：

  系统管理、实时通讯、三维显示、。。。。

1.3测试范围

本次系统测试将采用开发相关测试软件和利用成熟的测试工具相结合的方式，现场对系统进行测试。按照系统需求任务书的要求，分别进行系统的功能测试、性能测试、故障测试以及安全性测试等相关内容。

本次测试的主要目标包括：

系统软件单元功能测试——根据软件详细设计文档的要求，完成各个软件模块的单元功能测试工作；

系统软件集成功能测试——根据软件需求规格说明的要求，完成软件功能测试工作；

系统软件性能测试——根据技术协议书和软件需求规格说明的要求，完成软件性能测试工作；

系统故障测试——对软件系统在网络故障、数据库故障等异常情况下进行测试工作；

安全性测试——对系统用户登陆、密码保护等安全管理进行测试工作。

- 1.4参考文档
- 《GB 8566-88 计算机软件开发规范》
- 《GJB 438A-97 武器系统软件开发文档》
- Xxxxxxxx

2、测试参考文档和测试提交文档

- 2.1测试参考文档
- 本测试计划参照GB8567——88标准编制，下表列出了制定测试计划时所使用的文档，并标明了各文档的可用性：

2.2测试提交文档

在测试完成后，需要提交给用户相关的测试评估报告与改进报告，并提交详细的测试报告。

3、测试进度

测试活动	计划开始日期	实际开始日期	结束日期
单元功能测试	20XX年 3月16日	20XX年3月16日	20XX年3月26日
集成功能测试	20XX年3月27日	20XX年3月27日	20XX年3月31日
性能测试	20XX年4月2日	20XX年4月3日	20XX年4月5日
故障测试	20XX年4月8日	20XX年4月8日	20XX年4月9日
安全测试	20XX年4月10日	20XX年4月10日	20XX年4月11日
对测试进行评估	20XX年4月15日	20XX年4月16日	20XX年4月18日
用户验收测试	20XX年5月11日		

4.1 测试资源

4.1 人力资源

姓名	角色	具体职责或注释
胡　飞	总体策划人、方案设计人	策划总体规模，测试内容，规划测试方案
A	方案设计人、测试技术设计人	制定测试方案，确定测试深度，测试技术设计
B	计划人、记录人	计划测试进程，记录测试情况
C	计划人、测试人、记录人	计划整个进程以及各个阶段的进度安排、重点任务；测试并记录测试情况
D	计划人、测试人、记录人	测试并记录测试情况
E	测试人、记录人	测试并记录测试情况
F	测试人、记录人	测试并记录测试情况
G	测试人、记录人	测试并记录测试情况

4.2 测试环境

下表列出了测试的系统环境：

机器名 环境	数据库服务器	图形工作站	管理计算机
软件环境（相关软件、操作系统等）	Windows XP Oracle 9i		Windows XP
硬件环境（设备、网络等）	Xeon Mp 2.7×2 CPU/4smp/2GB/DVD/1000M×2	HP Workstation xw6200	HP Workstation xw6200

4.3 测试工具

下表列出此项目测试中使用的工具：

用途	工具名称	生产厂商/自产	版本
压力与性能测试	Jmeter	Jakarta	1.9.1
压力测试网络检测	Webstress	国外软件	6.18
申请一定量的内存	Men	自产	1.0

五、测试策略

5.1 功能测试

由于对测试对象的功能测试应侧重于所有可直接追踪到用例或业务功能和业务规则的测试需求。这种测试的目标是核实数据的接受、计算、处理和输出是否正确，以及业务规则的实施是否恰当，所以此次的功能测试全部为黑盒测试，测试的主要目的是检测以下错误：

1）是否有不正确或遗漏的功能？

2）在接口上，输入是否能正确的接受？能否输出正确的结果？

3）是否有数据错误或外部信息（例如数据文件）访问错误？

4）是否有初始化或终止性错误？

测试方法：

1）等价类划分

XXXXXXXXXXXXX

XXXXXXXXXXXXX

2） 边界值分析

Xxxxxxxxxxxxxx

Xxxxxxxxxxxxxx

3) 确立测试用例

5.1.1 具体功能模块1

5.1.2 具体功能模块2

Xxxxxxxxxxxxxxxxx

5.1.n 具体功能模块n

学生作业样本(e-learning)

作业一

软件功能性测试

寻找具有需求说明的（或熟悉软件功能需求）软件，采用面向功能的测试方法，生成黑盒测试用例，使用测试记录工具，报告所有发现的错误。

提交的报告：

- 测试计划
- 测试报告
- 测试结果分析报告

作业 提交时间：第6周周六以前（10月25日前）



Outline of a review report

1. Introduction
    a. Work product identification
    b. Review team members and roles
2. Preliminary issue list
    a. Potential fault
    b. Severity
3. Prioritized action item list
    a. Identified fault
    b. Severity
4. Summary of individual reports
5. Review statistics
    a. Total hours spent
    b. Faults sorted by severity
    c. Faults sorted by location
6. Review recommendation
7. Appendix with the full review packet

return



Chapter 4

Unit Test environment and tools

Junit and EMMA

1. Java
2. Eclipse
3. JUNIT
4. EMMA

4.1 Java

Why is Java ?

- Easy to get source code and documentation
- Completed test environment
- More free test tools

安装JDK（ Java Development Kit ）

- JDK 是整个Java的核心，包括了Java　行环境，Java工具和Java基础的类库。
- 没有JDK的话，无法安装或者　行java程序。

- JDK共有3个版本，分别为：
  - SE(Jave SE)，standard edition，标准版
  - EE(Jave EE)，enterprise edition，企业版
  - ME(Jave ME)，micro edition，主要用于移动设备、嵌入式设备上的java应用程序

- 目前JDK的最新版本为：Java SE Development Kit 8u51。
- 下　地址：
  - http://www.oracle.com/technetwork/java/javase/downloads/jdk8-downloads-2133151.html

安装与　置

- 下　与安装，假设安装到C:\Program Files\Java\jdk1.8.0_20
- 　置PATH：相当于C++中的GCC　置
  - 我的电脑点右键，选择“属性”，选择“高级”标签，进入“环境变量设置，在系统变量里找到PATH变量，选择→编辑，里面已经有很多的变量值, 变量值的最前面加上C:\Program Files\Java\jdk1.8.0_51\bin;
- 　置CLASSPATH：寻找标准类库位置
  - 我的电脑点右键，选择“属性”，选择“高级”标签，进入环境变量设置，在系统变量那一栏中点新建→ CLASSPATH，并且设置
  - classpath=.; C:\Program Files\Java\jdk1.8.0_51\lib;C:\Program Files\Java\jdk1.8.0_51\lib\tools.jar;

4.2 Eclipse

Eclipse is a kind of universal platform – an open extensible IDE for anything and nothing in particular. It provides a feature-rich development environment that allows the developer to efficiently create tools that integrate seamlessly into the Eclipse Platform.

www.eclipse.org

在Eclipse下开发java程序

- 使用Eclipse编写Java程序步　：
  1. 创建一个Java项目
  2. 创建”source folder”命名为src，创建包
  3. 创建Java源程序
  4. 编译Java源程序
  5. 运行程序

- 示例：HelloJava.java
- Example of Eclipse

4.3 JUNIT

- JUnit是由 Erich Gamma 和 Kent Beck 编写的一个回归测试框架（regression testing framework）。
- Junit测试是程序员开发测试工具，即所谓白盒测试，因为程序员知　被测试的　件如何（How）完成功能和完成什么样（What）的功能。Junit是一套框架，继承TestCase类，就可以用Junit　行自动测试了。
- http://www.junit.org

JUNIT

- For application developers, testing forms an integral part of the development life-cycle.
- As old code is modified for reasons ranging from adding new functionality to speed optimizations, the risk of dependent functionality getting broken increases.
- Tests that are written once, and can be run repeatedly is the only solution to avoid manually QA testing.
- To write these tests, for applications written in java, JUnit provides a solid framework for writing unit tests. Many JUnit integrations also exist to integrate your JUnit tests into build and development tools.

JUNIT安装（省略）

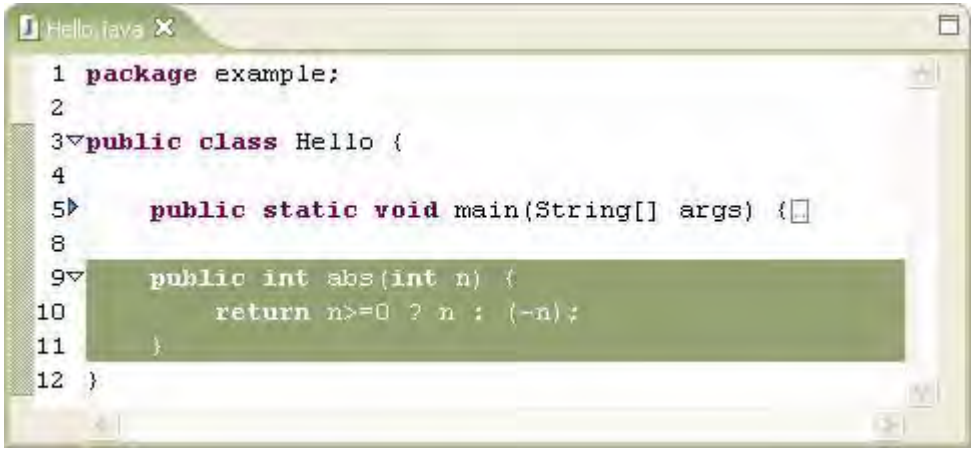
- 1. 从http://www.junit.org/下　JUNIT包JUNIT.JAR
- 2.将JAR文件拷贝到JDK的LIB目录，C:\Java\jdk1.8.0_20\lib.。
- 3. 在CLASSPATH 目录下增加一条 c:\Java\jdk1.8.0_51\lib\junit.jar;

- 新版eclipse 中已经　成了JUnit 3 和JUnit 4。

在Eclipse中使用JUNIT

- 测试对于保证　件开发质　有着　常　要的作用，单元测试更是必不可少，JUnit是一个　常强大的单元测试包，可以对一个/多个类的单个/多个方法测试，　可以将不同的TestCase组合成TestSuit，使测试任务自动化。

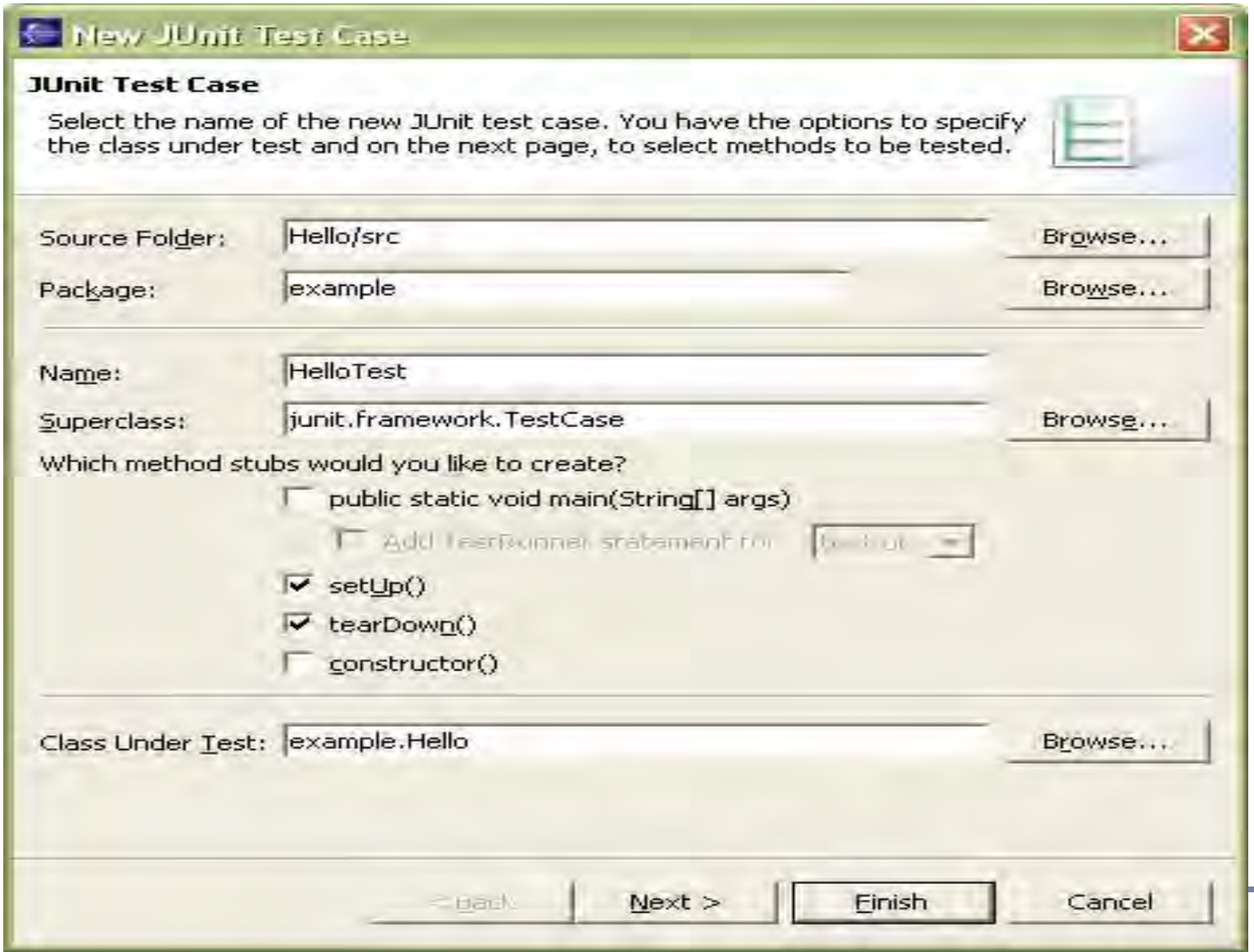
- 我们创建一个Java工程，添加一个example.Hello类，　先我们给Hello类添加一个abs()方法，作用是　回绝对值：


Hello.java
1 package example;
2
3 public class Hello {
4
5 public static void main(String[] args) {
8
9 public int abs(int n) {
10 return n>=0 ? n : (-n);
11 }
12 }

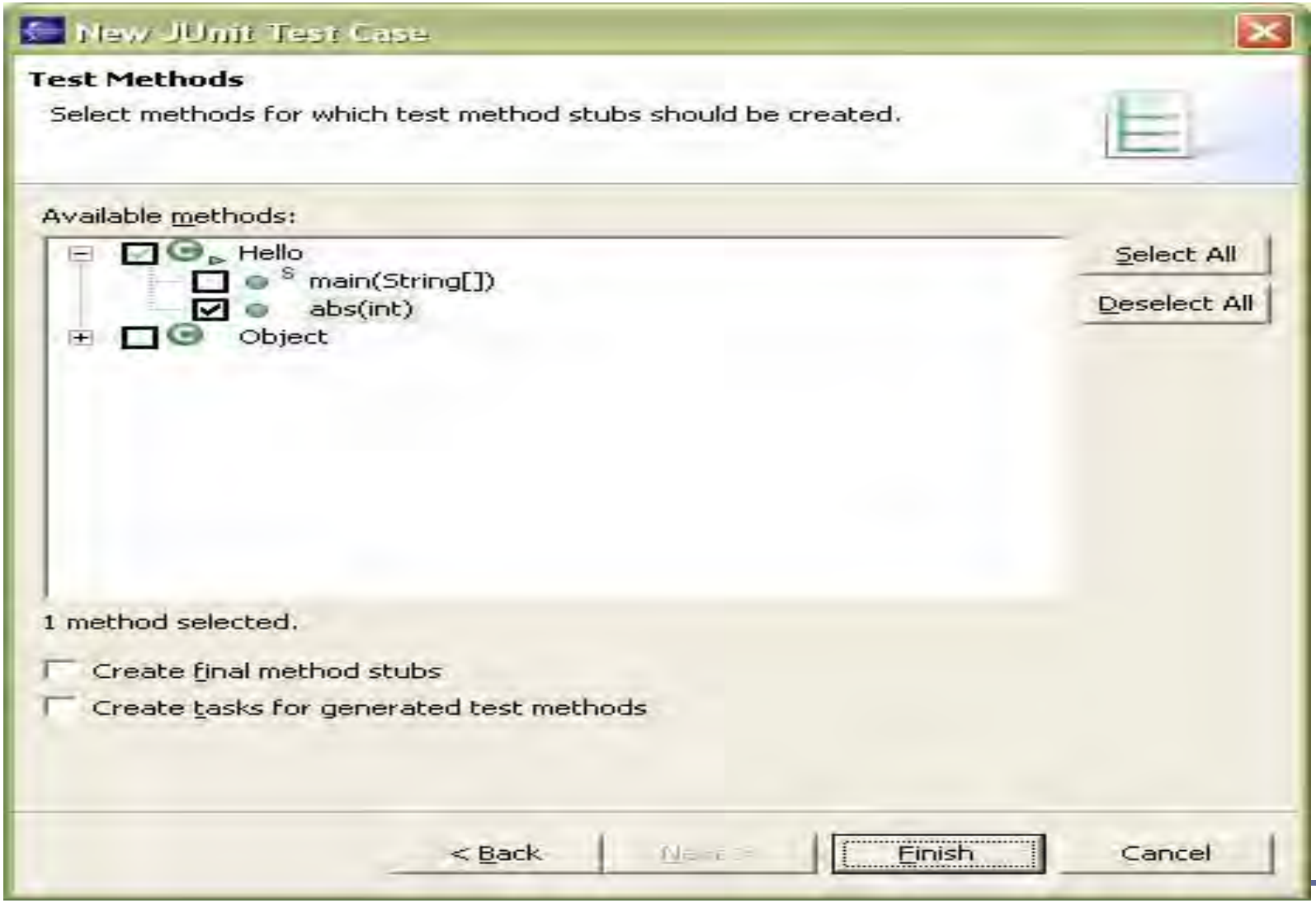
下一步，我们准备对这个方法进行测试，确保功能正常。中Hello.java，右键点击，选择New->JUnit Test Case：



Eclipse会询问是否添加junit.jar包，确定后新建一个HelloTest类，用来测试Hello类。一般自己导入下载的junit包。



选中setUp()和tearDown()，然后点击“Next”：



选择要测试的方法，我们选中abs(int)方法，完成后在HelloTest.java中输入：

```
1  package example;
2  import junit.framework.TestCase;
3
4  public class HelloTest extends TestCase {
5      private Hello hello;
6
7      protected void setUp() throws Exception {
8          super.setUp();
9          hello = new Hello();
10     }
11
12     protected void tearDown() throws Exception
13         super.tearDown();
14     }
15
16     public void testAbs() {
17         assertEquals(hello.abs(14), 14);
18         assertEquals(hello.abs(-5),  5);
19         assertEquals(hello.abs(0) ,  0);
20     }
21 }
```

JUnit会以以下顺序执行测试：（大致的代码）

```
try {
HelloTest test = new HelloTest(); // 建立测试类实例
test.setUp(); // 初始化测试环境
test.testAbs(); // 测试某个方法
test.tearDown(); // 清理资源
}
catch…
```

- setUp()是建立测试环境，　　创建一个Hello类的实例；tearDown()用于清理资源，如　放打开的文件等等。以test开头的方法被认为是测试方法，JUnit会依次执行testXxx()方法。在testAbs()方法中，我们对abs()的测试分别　择正数，负数和0，如果方法　回值与期待结果相同，则assertEquals不会产生异常。
- 如果有多个testXxx方法，JUnit会创建多个XxxTest实例，每次　行一个testXxx方法，setUp()和tearDown()会在testXxx前后被调用，因此，不要在一个testA()中依赖testB()。

直接运行Run->Run As->JUnit Test，就可以看到JUnit测试结果：　绿色表示测试通过，只要有1个测试未通过，就会显示红色并列出未通过测试的方法。可以试图改变abs()的代码，故意返回错误的结果（比如return n+1;），然后再运行JUnit就会报告错误。



- JUnit 单元测试，能在开发 段就找出许多Bug，并且，多个Test Case可以组合成Test Suite，让整个测试自动完成，尤其 合于XP方法。每增加一个小的新功能或者对代码 行了小的修改，就立刻 行一 Test Suite，确保新增和修改的代码不会破坏原有的功能，大大增强 件的可维护性， 免代码 渐“腐烂”。

示例

1. 建立hello（）类
2. 建立abs()方法；
3. 建立单元测试 Test Case——helloTest（）
4. 建立另一个math（）类
5. 建立add（）和 times（）方法
6. 建立单元测试 Test Case——mathTest（）
7. 建立测试　Test Suite

4.4 EMMA

- 测试覆盖率（Code Coverage）
- 测试覆盖率，简单的说，就是评价测试活动覆盖产品代码的指标。测试的目的，是确认产品代码按照　期一样工作，也可以看作是产品代码工作方式的说明文档。　一步考虑，测试覆盖率可以看作是产品代码质　的　接指标－－之所以说是　接指标，因为测试覆盖率评价的是测试代码的质　，并不是产品代码的质　。
- 代码覆盖率是一种白盒测试，因为测试覆盖率是评价产品代码类内　的指标，而不是评价系统接口或规约。测试覆盖率尤其用于评价测试代码是否已经覆盖了产品代码所有的路径。

EMMA

- EMMA 是一种用来衡　java　件覆盖率的　件。它对于不可　代码的检测，以及　证测试　和交互式测试对程序的覆盖情况是很必要的。
- EMMA的设计目标有以下几个，
- 1. 在不增加创建或执行开　的情况下，提供丰富的覆盖分析数据；
- 2. 促　团　开发效率，同时，也能加快单元开发－测试周期；
- 3. 促　小型java　件的开发效率，同时，提　包含大　类的大型企业　件的开发。
- EMMA与其他覆盖测试工具的区别在于它　向极　编程，目的是促使循环的开发－测试　件开发模型。

- 衡　测试覆盖率的指标很多，常用的指标有：
- Statement coverage，也称作Line coverage，用于评价测试的代码语句覆盖率。
- Basic block coverage，是Statement coverage的一个变种，它把没有一个分支的代码区域作为一个计　单位，而不是简单的代码行，用于一个if-else分支代码行数大于另一个的情况，在　种情况下，statement coverage指标并不　用。
- Decision coverage（也称作Branch coverage），用于评价代码分支地测试覆盖率。
- Path coverage，和Decision coverage相似，用于评价代码从开始到结束所有路径的测试覆盖率。
- Function coverage，用于评价代码方法的测试覆盖率。
- EMMA目前支持四种Coverage类型：
- class、method、line和basic block。

安装EclEmma插件

- EclEmma是一个基于EMMA的Java代码覆盖工具。它的目的是让你可以在Eclipse工作平台中使用强大的Java代码覆盖工具EMMA。EclEmma是　侵入式的不　要修改你的　目或执行其它任何安装，它能够在工作平台中启动像　行JUnit测试一样直接对代码覆盖　行分析。
- 安装 EclEmma 插件的　程和大　分Eclipse 插件相同，既可以　　Eclipse 标准的 Update 机制来　程安装 EclEmma 插件，也可以从站点下　 zip 文件并解压到 eclipse 所在的目录中。

安装EclEmma插件

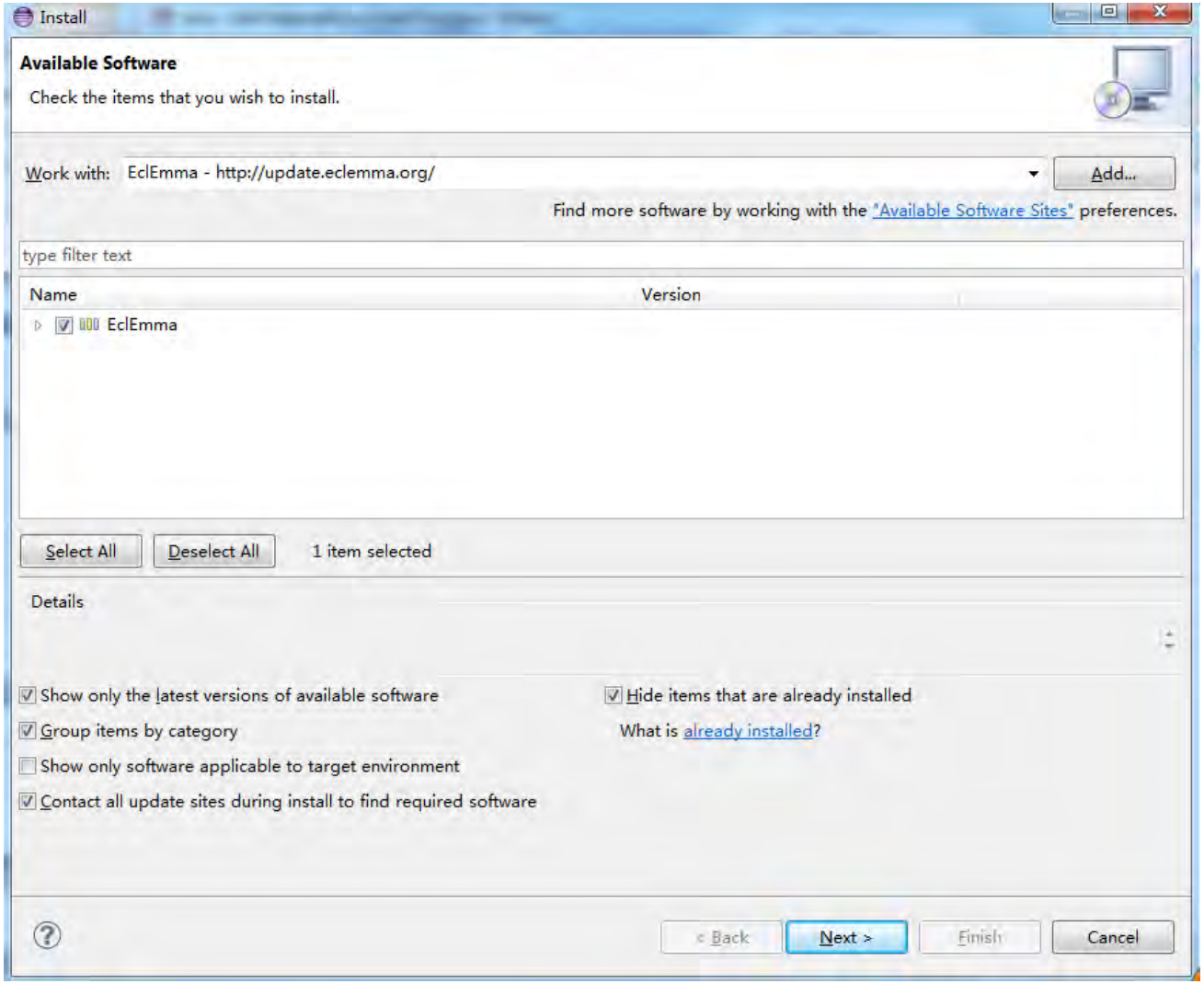
Eclipse的标准 Update 机制　程安装 EclEmma 插件

添加 EclEmma 更新站点

- Help →Install New Software… →Add

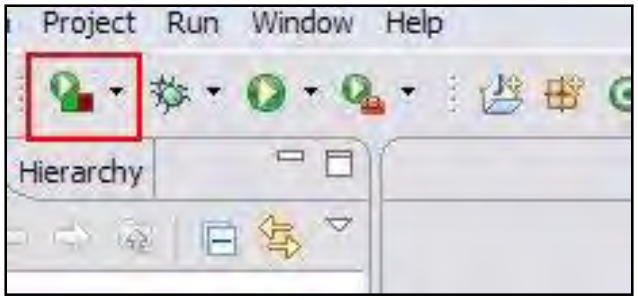


- Name：EclEmma
- URL：http://update.eclemma.org/


Install
Available Software
Check the items that you wish to install.
Work with: EclEmma - http://update.eclemma.org/
Add...
Find more software by working with the "Available Software Sites" preferences.
type filter text
Name
Version
EclEmma
Select All
Deselect All
1 item selected
Details
Show only the latest versions of available software
Group items by category
Show only software applicable to target environment
Contact all update sites during install to find required software
Hide items that are already installed
What is already installed?
< Back
Next >
Finish
Cancel

使用EclEmma插件

- 安装完成并　新启动 Eclipse 之后，工具栏上应该出现一个新的按　。



使用EclEmma 进行测试覆盖率

示例：

- 利用EclEmma测试程序　行覆盖率
- 生成EclEmma测试JUnit覆盖率报告
- 导出EclEmma覆盖率报告

4.5实例

XXXX支撑　件与可视化　件测试计划

目录

1、简介
1.1编写目的
1.2　目背景
1.3测试范围
1.4参考文档
2、测试参考文档和测试提交文档
2.1测试参考文档
2.2测试提交文档
3、测试　度
4、测试资源
4.1人力资源
4.2测试环境
4.3测试工具
5、测试策略

5.1功能测试
5.1.1 XXX系统管理　件功能测试
5.1.2 XXX系统实时　信　件功能测试
5.1.3目标三维动态显示模型及可视化　件功能测试
5.1.4XXX三维动态显示模型及可视化　件功能测试
5.1.5 XXX交会显示　件功能测试
5.1.6XXX系统数据管理　件功能测试
5.1.7 XXX文件与AutoCAD、Creator的转换　件功能测试
5.2性能测试
5.2.1　讯性能测试
5.2.2 实　性能测试
5.2.3 数据库性能测试
5.2.4 显示性能测试
5.2.5 其它性能测试
5.3故　测试
5.3.1网络故
5.3.2 服务器故
5.3.3 管理计算机故
5.3.4 计算节点机故
5.3.5 图形工作站故　测试
5.4安全性测试
5.4.1 管理　件安全性测试
5.4.2 数据管理　件安全性测试

1、简介

- 1.1编写目的
- 为了全　、系统地对“XXX支撑　件与可视化　件”　行评估与测试，从而保证系统　期稳定的　行，组织对该　件　行系统的总体综合测试。
- 1.2　目背景
- “XXX　证平台”是中国兵器工业XXX研究所承担的国基础科研　目。XXX所与XXX大学合作，对该　目的关技术——XXX支撑技术和可视化技术展开研究。本　目就是在此背景下开展的，　应用虚拟试　技术、计算机网络技术、可视化技术、数据库技术等，研制“XXX的支撑　件和可视化　件”，并最终　成在“XXX中。
- 本　件系统由以下7个二级　件组成：

  系统管理、实时　讯、三维显示、。。。。

1.3测试范围

本次系统测试将　用开发相关测试　件和利用成熟的测试工具相结合的方式，现场对系统　行测试。按照系统　求任务书的要求，分别　行系统的功能测试、性能测试、故　测试以及安全性测试等相关内容。

本次测试的主要目标包括：

系统　件单元功能测试——根据　件详细设计文档的要求，完成各个　件模块的单元功能测试工作；

系统　件　成功能测试——根据　件　求规格说明的要求，完成　件功能测试工作；

系统　件性能测试——根据技术协议书和　件　求规格说明的要求，完成　件性能测试工作；

系统故　测试——对　件系统在网络故　、数据库故　等异常情况下　行测试工作；

安全性测试——对系统用户登　、密码保护等安全管理　行测试工作。

- 1.4参考文档
- 《GB 8566-88 计算机　件开发规范》
- 《GJB 438A-97 武器系统　件开发文档》
- Xxxxxxxx

2、测试参考文档和测试提交文档

- 2.1测试参考文档
- 本测试计划参照GB8567——88标准编制，下表列出了制定测试计划时所使用的文档，并标明了各文档的可用性：

2.2测试提交文档

在测试完成后，　要提交给用户相关的测试评估报告与改　报告，并提交详细的测试报告。

3、测试进度

测试活动	计划开始日期	实际开始日期	结束日期
单元功能测试	2006年 3月16日	2006年3月16日	2004年3月26日
集成功能测试	2006年3月27日	2006年3月27日	2006年3月31日
性能测试	2006年4月2日	2006年4月3日	2006年4月5日
故障测试	2006年4月8日	2006年4月8日	2006年4月9日
安全测试	2006年4月10日	2006年4月10日	2006年4月11日
对测试进行评估	2006年4月15日	2006年4月16日	2006年4月18日
用户验收测试	2006年5月11日		

4.1 测试资源

4.1 人力资源

姓名	角色	具体职责或注释
胡　飞	总体策划人、方案设计人	策划总体规模，测试内容，规划测试方案
A	方案设计人、测试技术设计人	制定测试方案，确定测试深度，测试技术设计
B	计划人、记录人	计划测试进程，记录测试情况
C	计划人、测试人、记录人	计划整个进程以及各个阶段的进度安排、重点任务；测试并记录测试情况
D	计划人、测试人、记录人	测试并记录测试情况
E	测试人、记录人	测试并记录测试情况
F	测试人、记录人	测试并记录测试情况
G	测试人、记录人	测试并记录测试情况

4.2 测试环境

下表列出了测试的系统环境：

机器名 环境	数据库服务器	图形工作站	管理计算机
软件环境（相关软件、操作系统等）	Windows XP Oracle 9i		Windows XP
硬件环境（设备、网络等）	Xeon Mp 2.7×2 CPU/4smp/2GB/DVD/1000M×2	HP Workstation xw6200	HP Workstation xw6200

4.3 测试工具

下表列出此　目测试中使用的工具：

用途	工具名称	生产厂商/自产	版本
压力与性能测试	Jmeter	Jakarta	1.9.1
压力测试网络检测	Webstress	国外软件	6.18
申请一定量的内存	Men	自产	1.0

五、测试策略

5.1 功能测试

由于对测试对象的功能测试应侧　于所有可直接　踪到用例或业务功能和业务规则的测试　求。　种测试的目标是核实数据的接受、计算、处理和　出是否正确，以及业务规则的实施是否恰当，所以此次的功能测试全　为盒测试，测试的主要目的是检测以下　误：

1）是否有不正确或　漏的功能？

2）在接口上，　入是否能正确的接受？能否　出正确的结果？

3）是否有数据　误或外　信息（例如数据文件）访　　误？

4）是否有初始化或终止性　误？

测试方法：

1）等价类划分

XXXXXXXXXXXXX

XXXXXXXXXXXXX

2）　界值分析

Xxxxxxxxxxxxxx

Xxxxxxxxxxxxxx

3）确立测试用例

5.1.1 具体功能模块1

5.1.2 具体功能模块2

Xxxxxxxxxxxxxxxxx

5.1.n 具体功能模块n

学生作业样本(e-learning)

说明：P15~P28的6个　　事例是　要大家自己读理解的。

1. 三角形
2. NextDate
3. 佣
4. SATM系统
5. 货币转换
6. 挡　玻璃　刷
7. 车库　控制

？



Chapter 5

Functional Test



1. Boundary Value Testing
2. Equivalence Class Testing
3. Decision Table–Based Testing
4. Cause-and-Effect

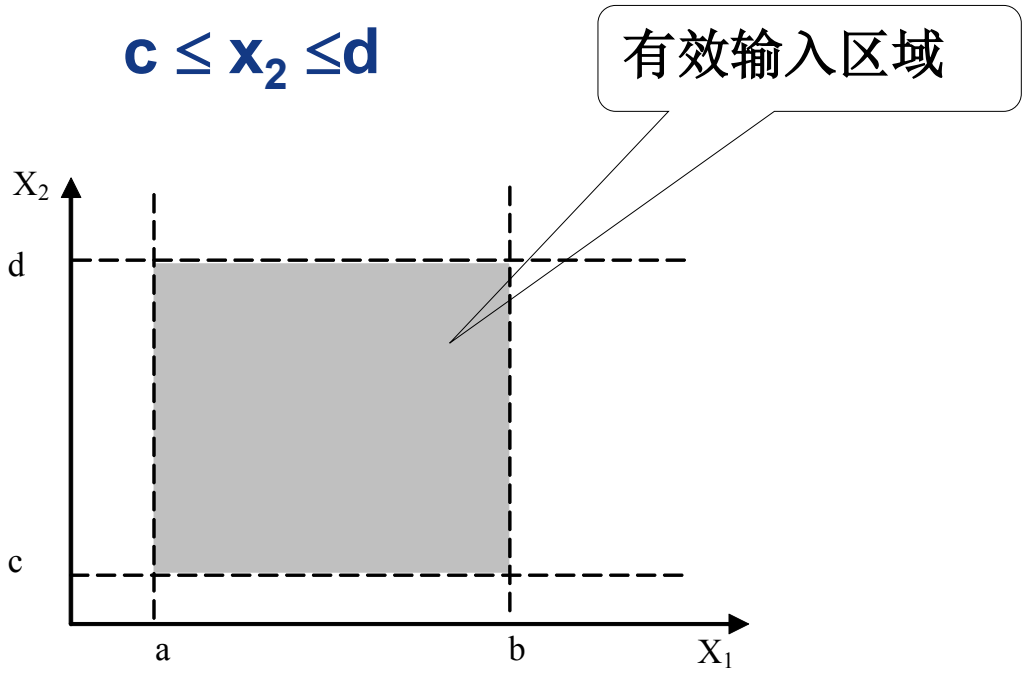
5.1 Boundary Value Testing

The function F: $Y = F(x_1, x_2)$

the input variables x_1 and x_2 will have some (possibly unstated) boundaries:

$$a \le x_1 \le b$$

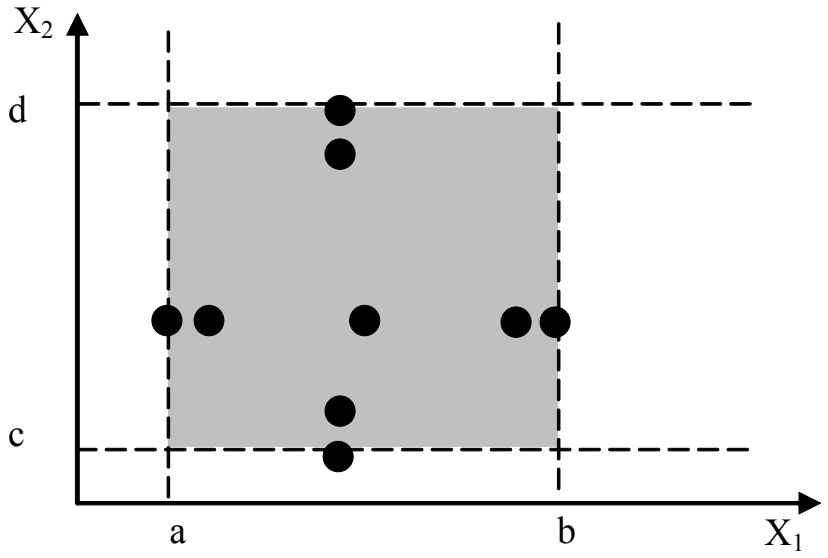
$$c \le x_2 \le d$$



Normal Boundary Value Testing

refers to these values as min, min+, nom, max–, and max.

"single fault" assumption： failures are only rarely the result of the simultaneous occurrence of two (or more) faults.



if we have a function of n variables, boundary value analysis yields **4n + 1** unique test cases.



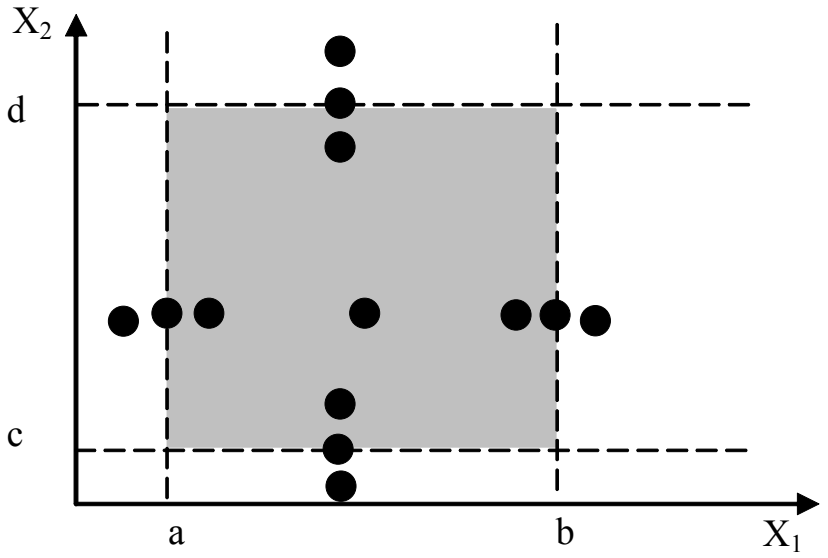
Limitations of Boundary Value Analysis

- Boundary value analysis works well when the program to be tested is a function of several independent variables that represent bounded physical quantities.
- Why?

1. the variables need to be described by a true ordering relation.
2. value analysis presumes the variables to be truly independent.
3. no consideration of the nature of the function, nor of the semantic meaning of the variables.

Robust Boundary Value Testing

in addition to the five boundary value，the extrema are exceeded with a value slightly greater than the maximum (max+) and a value slightly less than the minimum (min–)



For n variables, Robust Boundary Value analysis yields **6n + 1** unique test cases.

Worst-Case Boundary Value Testing

- **Rejecting single-fault assumption**
  - worst-case testing for a function of n variables：generates 5^n test cases.
  - robust worst-case testing for a function of n variables： generates 7^n test cases.
- **Test Cases for the Triangle Problem**
  - 3 sides：a、b、c;
  - each side：（1~200）
  - Normal Boundary Value Testing：4×3+1=13（Table 5.1）
  - Worst-Case Boundary Value testing ：5^3 =125个（Table 5.2）

Table 5.1 Normal Boundary Value Test Cases

Case	a	b	c	*Expected Output*
1	100	100	1	Isosceles
2	100	100	2	Isosceles
3	100	100	100	Equilateral
4	100	100	199	Isosceles
5	100	100	200	Not a triangle
6	100	1	100	Isosceles
7	100	2	100	Isosceles
8	100	100	100	Equilateral
9	100	199	100	Isosceles
10	100	200	100	Not a triangle
11	1	100	100	Isosceles
12	2	100	100	Isosceles
13	100	100	100	Equilateral
14	199	100	100	Isosceles
15	200	100	100	Not a triangle

Table 5.2 (Selected) Worst-Case Boundary Value Test Cases

Case	a	b	c	*Expected Output*
1	1	1	1	Equilateral
2	1	1	2	Not a triangle
3	1	1	100	Not a triangle
4	1	1	199	Not a triangle
5	1	1	200	Not a triangle
6	1	2	1	Not a triangle
7	1	2	2	Isosceles
8	1	2	100	Not a triangle
9	1	2	199	Not a triangle
10	1	2	200	Not a triangle
11	1	100	1	Not a triangle
12	1	100	2	Not a triangle
13	1	100	100	Isosceles
14	1	100	199	Not a triangle
15	1	100	200	Not a triangle
16	1	199	1	Not a triangle
17	1	199	2	Not a triangle
18	1	199	100	Not a triangle
19	1	199	199	Isosceles
20	1	199	200	Not a triangle
21	1	200	1	Not a triangle
22	1	200	2	Not a triangle
23	1	200	100	Not a triangle
24	1	200	199	Not a triangle
25	1	200	200	Isosceles



Random Testing

- Even if all tests of the boundary value are completed, the errors of the program are not found, and some errors exist in the non boundary value.
- rather than always choose the min, min+, nom, max–, and max values of a bounded variable, use a random number generator to pick test case values.
- how many random test cases are sufficient?

5.2 Equivalence Class Testing

1）Weak Normal Equivalence Class Testing

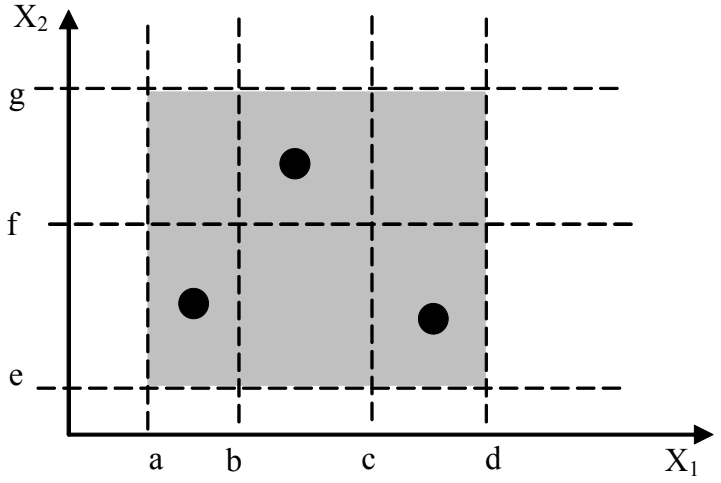
The function: $Y = f(x_1, x_2)$

$a \le x_1 \le d$, with intervals [a, b), [b, c), [c, d], $P_1=3$

$e \le x_2 \le g$, with intervals [e, f), [f, g], $P_2=2$

test cases as classes in the partition with the largest number of subsets

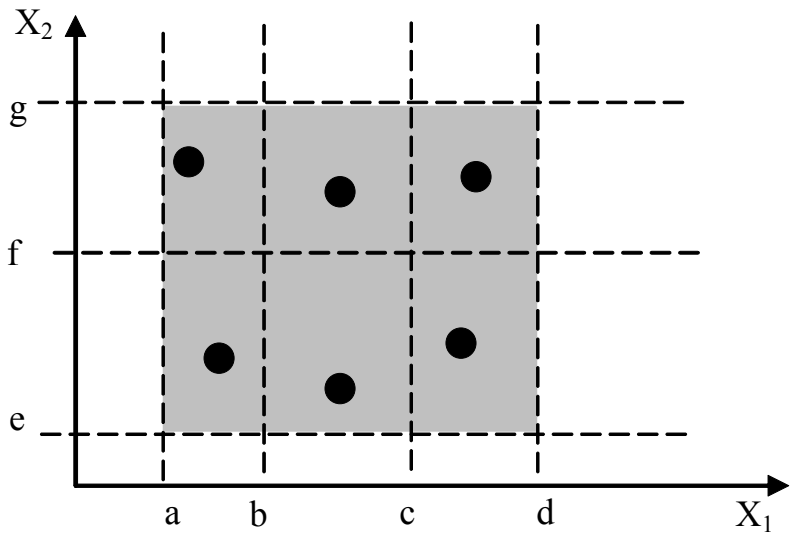
Test case = 3



2) Strong Normal Equivalence Class Testing

based on the multiple fault assumption，test cases from each element of the Cartesian product of the equivalence classes

Test case：N×M。

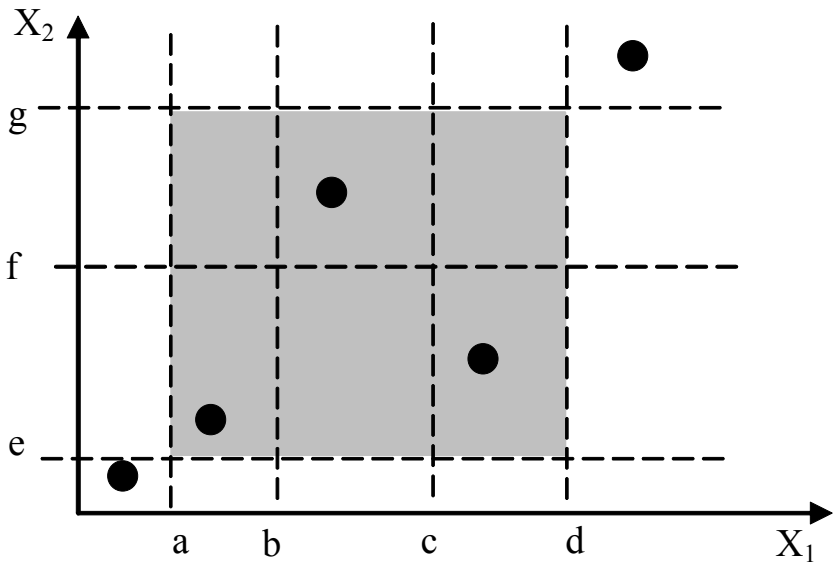


3）Weak Robust Equivalence Class Testing

weak：single fault assumption

robust：invalid values

Test case：the largest number of subsets（3） plus invalid inputs（2）

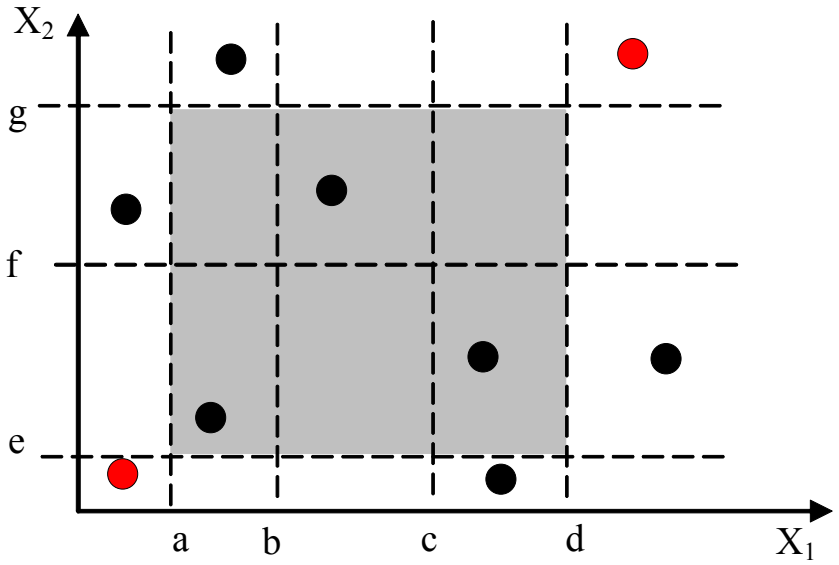


Consider：

weak：single fault assumption

robust：invalid values

Test case：the largest number of subsets（3） plus invalid inputs（2 X input class）

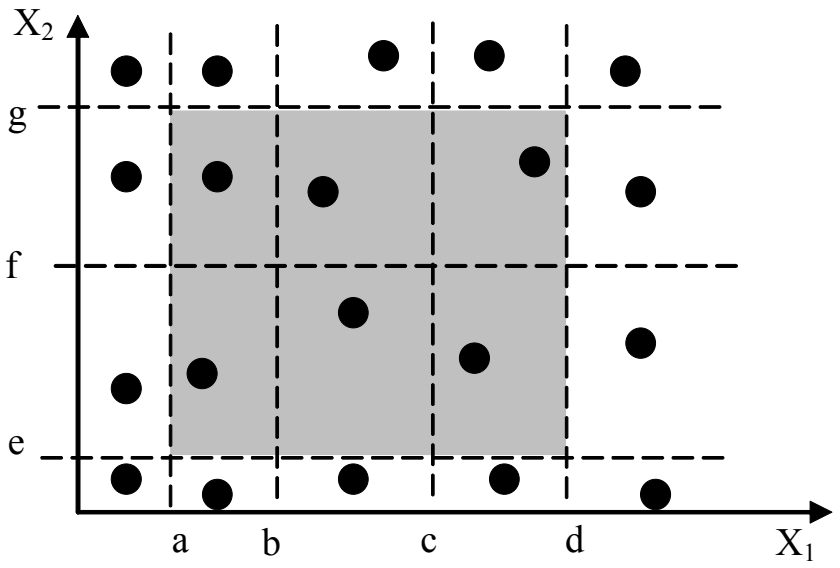


4）Strong Robust Equivalence Class Testing

strong ： multiple fault assumption.

robust：invalid values

Test case：Cartesian product of all the equivalence classes, both valid and invalid. （N+2）×（M+2）





Equivalence Class Test Cases for the Triangle Problem

Triangle Problem

input 3 sides, four possible outputs:

R1 = {<a, b, c>: the triangle with sides a, b, and c is equilateral}

R2 = {<a, b, c>: the triangle with sides a, b, and c is isosceles}

R3 = {<a, b, c>: the triangle with sides a, b, and c is scalene}

R4 = {<a, b, c>: sides a, b, and c do not form a triangle}

Four weak normal equivalence class test cases

Test case	a	b	c	Expected Output
1	**5**	**5**	**5**	**Equilateral**
2	**2**	**2**	**3**	**Isosceles**
3	**3**	**4**	**5**	**Scalene**
4	**4**	**1**	**2**	**Not a triangle**

strong normal equivalence

Test case	a	b	c	Expected Output
1	-1	5	5	a is min-
2	5	-1	5	b is min-
3	5	5	-1	c is min-
4	201	5	5	a is max+
5	5	201	5	b is max+
6	5	5	201	c is max+

strong robust equivalence class test cases: Additional Invalid value combination

Test case	a	b	c	Expected Output
1	-1	-1	5	a、b are min-
2	5	-1	-1	b、c are min-
3	-1	5	-1	a、c are min-
4	-1	-1	-1	a、b、c are min-

Here is only explain for min-, no more explain for max+.



Remember: 7 examples in chapter 2, should be read by yourself.

1. The Triangle Problem
2. The NextDate Function
3. The Commission Problem
4. The SATM System
5. The Currency Converter
6. Saturn Windshield Wiper Controller
7. Garage Door Opener

Equivalence Class Test Cases for the NextDate Function

NextDate

three variables: month, day, and year, valid values defined as follows：

M1（月）=1~12

D1（日）=1~31

Y1（年）=1812~2012

The invalid equivalence classes are:

M2 = {month: month < 1 }, M3 = {month: month > 12 }

D2 = {day: day < 1 }, D3 = {day: day > 31 }

Y2 = {year: year < 1812 }, Y3 = {year: year > 2012 }



weak normal equivalence class test case

Case ID	Month	Day	year	Expected Output
1	**6**	**12**	**2001**	**2001年6月13日**

weak robust test cases:

Case ID	Month	Day	year	Expected Output
1	**6**	**12**	**2001**	**2001-6-13**
2	**-1**	**12**	**2001**	**Invalid month**
3	**13**	**12**	**2001**	**Invalid month**
4	**6**	**-1**	**2001**	**Invalid day**
5	**6**	**31**	**2001**	**Invalid day**
6	**6**	**12**	**1811**	**Invalid year**
7	**6**	**12**	**2013**	**Invalid year**

Strong robust equivalence class test need consider Cartesian product of all the equivalence classes. No more explain here.

More analysis, month include big month(31天)、small month（30天）, February（28天）；end of month day:29、30、31；and year is leap year or not?

M1 = {month has 30 days},M2 = {month has 31 days}

M3 = { month is February};

D1 = {1 ≤ day ≤ 28}, D2 = {day = 29}, D3 = { day = 30}, D4 = {day = 31}

Y1 = { year = 2000}, Y2 = {year is a non-century leap year}, Y3 = {year is a common year}



strong normal equivalence class test cases

Case ID	Month	Day	Year	Expected Output
SN1	6	14	2000	6/15/2000
SN2	6	14	1996	6/15/1996
SN3	6	14	2002	6/15/2002
SN4	6	29	2000	6/30/2000
SN5	6	29	1996	6/30/1996
SN6	6	29	2002	6/30/2002
SN7	6	30	2000	Invalid input date
SN8	6	30	1996	Invalid input date
SN9	6	30	2002	Invalid input date
SN10	6	31	2000	Invalid input date
SN11	6	31	1996	Invalid input date
SN12	6	31	2002	Invalid input date
SN13	7	14	2000	7/15/2000
SN14	7	14	1996	7/15/1996
SN15	7	14	2002	7/15/2002
SN16	7	29	2000	7/30/2000
SN17	7	29	1996	7/30/1996
SN18	7	29	2002	7/30/2002
SN19	7	30	2000	7/31/2000
SN20	7	30	1996	7/31/1996
SN21	7	30	2002	7/31/2002
SN22	7	31	2000	8/1/2000
SN23	7	31	1996	8/1/1996
SN24	7	31	2002	8/1/2002
SN25	2	14	2000	2/15/2000
SN26	2	14	1996	2/15/1996
SN27	2	14	2002	2/15/2002
SN28	2	29	2000	3/1/2000
SN29	2	29	1996	3/1/1996
SN30	2	29	2002	Invalid input date
SN31	2	30	2000	Invalid input date
SN32	2	30	1996	Invalid input date
SN33	2	30	2002	Invalid input date
SN34	2	31	2000	Invalid input date
SN35	2	31	1996	Invalid input date
SN36	2	31	2002	Invalid input date

5.3 Decision Table–Based Testing

A decision table has four portions: the stub portion; the entry portion. the condition portion, and the action portion.

stub portion

entry portion

Stub	*Rule 1*	*Rule 2*	*Rule 3,4*	*Rule 5*	*Rule 6*	*Rule 7,8*
c1	T	T	T	F	F	F
c2	T	T	F	T	T	F
c3	T	F	—	T	F	—
a1	X	X		X		
a2	X				X	
a3		X		X		
a4			X			X

A decision table has four portions: the stub portion; the entry portion. the condition portion, and the action portion.

condition portion

action portion

Stub	*Rule 1*	*Rule 2*	*Rule 3,4*	*Rule 5*	*Rule 6*	*Rule 7,8*
c1	T	T	T	F	F	F
c2	T	T	F	T	T	F
c3	T	F	—	T	F	—
a1	X	X		X		
a2	X				X	
a3		X		X		
a4			X			X



To identify test cases with decision tables, we interpret conditions as inputs and actions as outputs.

Decision Table for Triangle Problem

c1:a,b,c form a triangle?	N	Y	Y	Y	Y	Y	Y	Y	Y
c2: a=b?	—	Y	Y	Y	Y	N	N	N	N
c3: a=c?	—	Y	Y	N	N	Y	Y	N	N
c4: b=c?	—	Y	N	Y	N	Y	N	Y	N
a1: Not a triangle	X								
a2: Scalene									X
a3: Isosceles					X		X	X	
a4: Equilateral		X							
a5: Impossible			X	X		X			

Refined Decision：

c1:a<b+c;

c2:b<a+c;

c3:c<a+b;

c4:a=b;

c5:a=c;

c6:b=c;

Table 7.3 Refined Decision Table for Triangle Problem

c1: a < b + c?	F	T	T	T	T	T	T	T	T	T	T
c2: b < a + c?	–	F	T	T	T	T	T	T	T	T	T
c3: c < a + b?	–	–	F	T	T	T	T	T	T	T	T
c4: a = b?	–	–	–	T	T	T	T	F	F	F	F
c5: a = c?	–	–	–	T	T	F	F	T	T	F	F
c6: b = c?	–	–	–	T	F	T	F	T	F	T	F
a1: Not a triangle	X	X	X								
a2: Scalene											X
a3: Isosceles							X		X	X	
a4: Equilateral				X							
a5: Impossible					X	X		X			

Test Cases for the NextDate Function

First Try，set of equivalence classes close：

M1 = {month: month has 30 days}

M2 = {month: month has 31 days}

M3 = {month: month is February}

D1 = {day: 1 ≤ day ≤ 28}

D2 = {day: day = 29}

D3 = {day: day = 30}

D4 = {day: day = 31}

Y1 = {year: year is a leap year}

Y2 = {year: year is not a leap year}

Conditions		
c1: Month in M1?		
c2: Month in M2?		
c3: Month in M3?		
c4: Day in D1?		
c5: Day in D2?		
c6: Day in D3?		
c7: Day in D4?		
c8: Year in Y1?		
a1: Impossible		
a2： NextDate		

Will have 2^8=256 rules !



Second Try, focus on the leap year aspect of the NextDate function.

M1 = {month: month has 30 days}
M2 = {month: month has 31 days}
M3 = {month: month is February}
D1 = {day: 1 ≤ day ≤ 28}
D2 = {day: day = 29}
D3 = {day: day = 30}
D4 = {day: day = 31}
Y1 = {year: year = 2000}
Y2 = {year: year is a non-century leap year}
Y3 = {year: year is a common year}

These conditions would result in a decision table with 36 rules

	1	2	3	4	5	6	7	8
c1:Month in	M1	M1	M1	M1	M2	M2	M2	M2
c2:Day in	D1	D2	D3	D4	D1	D2	D3	D4
c3:Year in	—	—	—	—	—	—	—	—
Rule count	3	3	3	3	3	3	3	3
a1:Impossible				X				
a2:Increment day	X	X			X	X	X	
a3:Reset day			X					X
a4:Increment month			X					?
a5:Reset month								?
a6:Increment year								?

续表

	9	10	11	12	13	14	15	16
c1:Month in	M3	M3	M3	M3	M3	M3	M3	M3
c2:Day in	D1	D1	D1	D2	D2	D2	D3	D4
c3:Year in	Y1	Y2	Y3	Y1	Y2	Y3	—	—
Rule count	1	1	1	1	1	1	3	3
a1:Impossible						X	X	X
a2:Increment day	X	X						
a3:Reset day			?	X	X			
a4:Increment month			?	X	X			
a5:Reset month								
a6:Increment year								

Third Try，specific about days in February and December

M1 = {month: month has 30 days}
M2 = {month: month has 31 days except December}
M3 = {month: month is December}
M4 = {month: month is February}
D1 = {day: 1 ≤ day ≤ 27}
D2 = {day: day = 28}
D3 = {day: day = 29}
D4 = {day: day = 30}
D5 = {day: day = 31}
Y1 = {year: year is a leap year}
Y2 = {year: year is a common year}

Table 7.14 Decision Table for NextDate Function

	1	2	3	4	5	6	7	8	9	10
c1:Month in	M1	M1	M1	M1	M1	M2	M2	M2	M2	M2
c2:Day in	D1	D2	D3	D4	D5	D1	D2	D3	D4	D5
c3:Year in	—	—	—	—	—	—	—	—	—	—
Actions										
a1:Impossible					X					
a2:Increment day	X	X	X			X	X	X	X	
a3:Reset day				X						X
a4:Increment month				X						X
a5:Reset month										
a6:Increment year										

22 rules

	11	12	13	14	15	16	17	18	19	20	21	22
c1:Month in	M3	M3	M3	M3	M3	M4	M4	M4	M4	M4	M4	M4
c2:Day in	D1	D2	D3	D4	D5	D1	D2	D2	D3	D3	D4	D5
c3:Year in	—	—	—	—	—	—	Y1	Y2	Y1	Y2	—	—
Actions												
a1:Impossible										X	X	X
a2:Increment day	X	X	X	X		X	X					
a3:Reset day					X			X	X			
a4:Increment month								X	X			
a5:Reset month					X							
a6:Increment year					X							

Table 7.15 Reduced Decision Table for NextDate Function

	1~3	4	5	6~9	10	11~14
c1:Month in	M1	M1	M1	M2	M2	M3
c2:Day in	D1,D2,D3	D4	D5	D1,D2,D3,D4	D5	D1,D2,D3,D4
c3:Year in	—	—	—	—	—	—
Actions						
a1:Impossible			X			
a2:Increment day	X			X		X
a3:Reset day		X			X	
a4:Increment month		X			X	
a5:Reset month						
a6:Increment year						

共13个测试规则,具体测试用例(略)

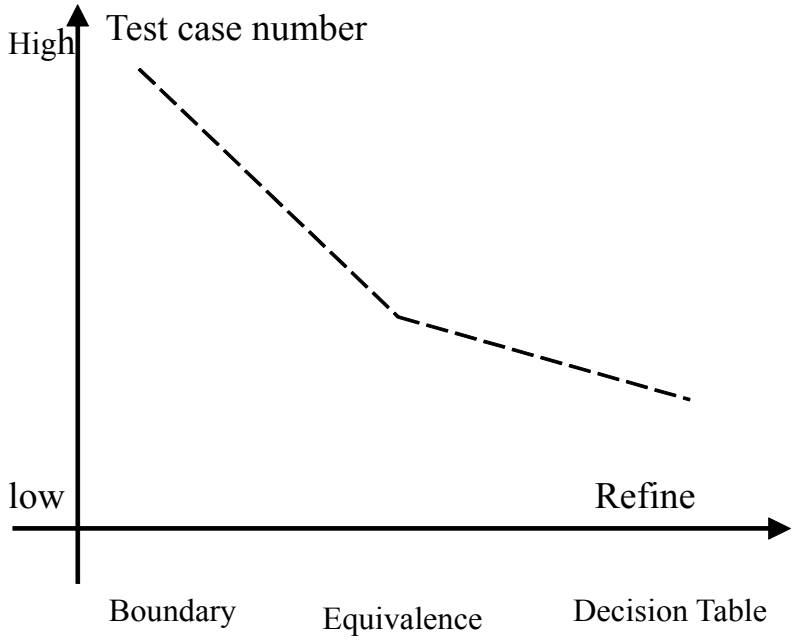
	15	16	17	18	19	20	21,22
c1:Month in	M3	M4	M4	M4	M4	M4	M4
c2:Day in	D5	D1	D2	D2	D3	D3	D4,D5
c3:Year in	—	—	Y1	Y2	Y1	Y2	—
Actions							
a1:Impossible						X	X
a2:Increment day		X	X				
a3:Reset day	X			X	X		
a4:Increment month				X	X		
a5:Reset month	X						
a6:Increment year	X						

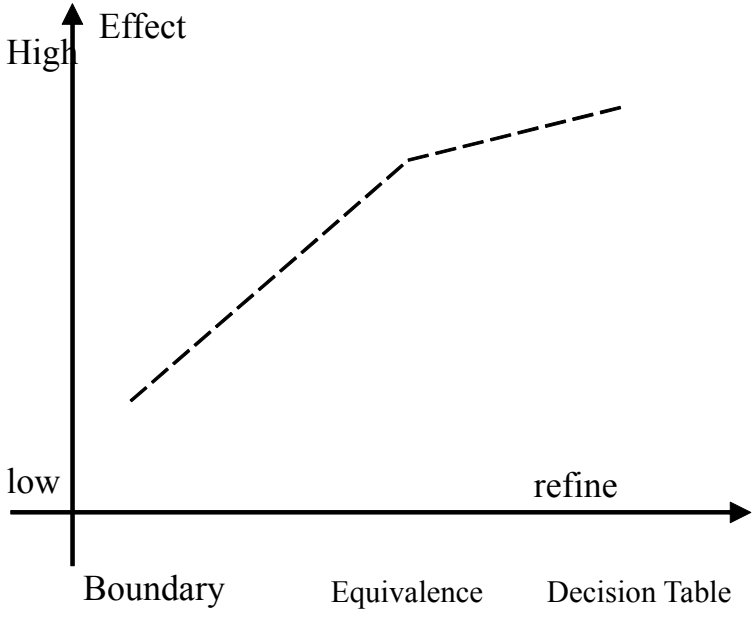
从第一次的256个测试规则到现在的13个测试规则，测试等价类的完备性是一样的。

换言之，从决策表测试方法，我们可以充分地认识到，测试用例越多，并不能增加测试的完备性。而通过分析与合并，使用尽可能少的测试用例来达到测试完备性要求，正是测试技术研究的核心内容和目标。

5.4 Cause-and-Effect

From the introduction before, We can clearly get the following conclusions:





软件测试技术研究的目标:使用尽可能少的测试用例,发现尽可能多的软件错误。提高测试的效率。

有关测试效率的问题，在后续的章节中还要多次提到。

Guidelines and Observations

1. The decision table technique is indicated for applications characterized by any of the following:
    a. Prominent if–then–else logic
    b. Logical relationships among input variables
    c. Calculations involving subsets of the input variables
    d. Cause-and-effect relationships between inputs and outputs
    e. High cyclomatic complexity (see Chapter 9)
2. Decision tables do not scale up very well (a limited entry table with n conditions has 2^n rules).
3. The first set of conditions and actions you identify may be unsatisfactory. Use it as a stepping stone and gradually improve on it until you are satisfied with a decision table.。

Q&A

Chapter 6

Software Standards and Documentation

1. 件工程标准化
2. 件文档
3. 件质 认证

6.1 软件工程标准化

6.1.1 件工程标准化定义

1983年我国 布的国家标准(GB3935.1-83)中对“标准”的定义是：“标准是对 复性事物和概念所做的统一规定。它以科学、技术、实践经 和综合成果为基础，经有关方 协商一致，由主管机构批准，以特定形式发布，作为共同 守的准则和依据。”

1983年国 标准化组织发布的ISO第二号指南(第四版)对“标准”的 新定义是：“由有关各方根据科学技术成就与先 经 ，共同合作起草，一致或基本上同意的技术规范或其他公开文件，其目的在于促 最佳的公众利益，并由标准化团体批准。”

在2000年发布的GB/T1.1-2000“标准化工作导则”中，将标准定义为：“为在一定的范围内获得最佳秩序，对活动或其结果规定共同的和　复使用的规则、导则或特性文件。该文件经协商一致制定，并经一个公认机构的批准。标准应以科学、技术和经　的综合成果为基础，以促　最佳社会效益为目的。”

统一性

科学性

权威性

6.1.2 软件工程标准的范围

整个　件生存期: 从？到　？
跨越　件生存期各个　段的技术管理工作：？
如：　程管理、　置管理、产品管理等
如：产品分析、测试、评审和审计等

- 所有　些方　建立的标准或规范，即是　件工程标准化。

6.1.3 软件工程标准化的作用

(1)可提　　件的可　性、可维护性和可移植性，从而提　　件产品的质　；

(2)可提　　件人员的技术水平和　件的生产率；

(3)可提　　件人员之　的　信效率，减少差　和误解；

(4)为科学地　行　件管理奠定了基础；

(5)有利于　低　件产品的成本和　行维护成本；

(6)有利于缩短　件开发周期；

(7)标准化是　件研究、生产、使用三者之　的桥梁。

6.1.4 软件工程标准的层次

用的范围有所不同，它可分为五个级别，即国　标准、国家标准、行业标准、企业(机构)标准及　目(课　)标准。

1) 国　标准：ISO

2) 国家标准

由政府或国家级的机构制定或批准，　用于全国范围的标准，下列为几个国家的国家标准。

（1）GB

（2）ANSI

（3）FIPS(NBS)

（4）BS

（5）DIN

（6）JIS

3) 行业标准

由行业机构、学术团体或国　机构制定，并　用于某个业务　域的标准。

（1）IEEE

（2）GJB

（3）DOD-STD

（4）MIL-S

4)企业规范

5)　目规范

6.1.4 常见的国际标准化组织

- 国　标准化委员会（ISO）
- 国　电工委员会（IEC）
- 电气与电子工程师协会（IEEE）
- 美国国家标准学会（ANSI）
- 美国　信工程协会（TIA）
- 美国电子工程协会（EIA）
- 欧洲电工标准化委员会CENELEC与欧洲标准化委员会CEN

6.1.5 常用的国家标准化管理委员会批准的软件工程国家标准

中国软件工程标准		
分类	标准名称	标准号
基础标准	信息技术 软件工程术语	GB、T 11457-2006
开发标准	软件开发规范	GB 8566-2001
	软件生成周期过程 用于项目管理的指南	GB/Z 20156-2006
	软件生成周期过程 配置管理	GB/T 20158-2006
	信息技术 软件维护	GB/T 20157-2006
文档标准	软件文档管理指南	GB
	计算机软件产品开发文件编制指南	GB 8567-2006

质量标准	信息技术 软件产品评价-质量特性及其使用指南	GB
	产品质量 第1部分：质量模型	GB/T 16260.1-2006
	产品质量 第2部分：外部度量	GB/T 16260.2-2006
	产品质量 第3部分：内部度量	GB/T 16260.3-2006
	产品质量 第4部分：使用质量的度量	GB/T 16260.4-2006

附件：软件工程国家标准

6.2 软件文档

5.2.1 件文档的含义

件文档(Document)也称文件， 常指的是一些记录的数据和数据媒体，它具有固定不变的形式，可被人和计算机 读。

件 =?

程序 + 文档



6.2.2 软件文档的作用

(1)　件文档是开发管理工作的依据，　件文档是检查　件开发　度和开发质　的标准。

(2)提　联系和开发效率。对各　段的工作有详细掌握，减少工，便于发现　误和不一致性，并及时　行纠正。

(3)　件文档也是开发人员一定　段的工作成果和结束标志。

(4)便于以后　件的开发、使用和维护。

(5)提供　件的　行、维护和培训的有关信息。

(6)能充分展示　件的功能、性能等各　指标，为用户　购符合自己　要的　件提供参照。

可以说，文档是软件开发规范的体现和标准，按规范要求生成一整套文档的过程，就是按照软件开发规范完成一个软件开发的过程。

6.2.3 常用软件文档

国家标准局在1988年1月　布了《计算机　件开发规范》和《　件产品开发文件编制指南》，指出　件全　程应提交的文档为以下13种:

1. 可行性研究报告：

- 技术可行?
- 经济可行?
- 社会因素?

2. 　目开发计划：

- 　度
- 人员
- 经费　算
- 硬件和　件资源

3. 件 求规格说明：

功能、性能

界 、环境

4. 数据要求规格说明：

数据类型、格式、 、存储要求

5. 概要设计规格说明：

功能分 、模块划分

接口设计、 行设计（工作流程）

数据结构、出 处理

6. 详细设计规格说明：

每个模块：实现算法、 流程等。

7. 用户手册：

详细描　　件的功能、性能和用户界　，使用户了解如何使用该　件。

8. 操作手册：

为操作人员提供该　件各种　行情况的有关知识，特别是操作方法细节。

9. 测试计划：

内容、　　范围、　　度、　　条件　、人员、

测试用例的　取原则、测试结果允许的偏差范围等。

10. 测试分析报告：

测试分析

测试结论

11.开发　度月报：

计划与实　　度差

成果、

计划调整

12.　目开发总结报告：

度、 成果、 成本、 资源

经　与教训

13.维护修改建议：

修正、更改建议

必不可少的报告:

1. 目开发计划
2. 件 求规格说明
3. 系统设计报告(概要）
4. 详细设计报告
5. 用户(操作)手册
6. 测试计划与报告(包含分析报告)
7. 目开发总结报告

文档实例1：软件项目开发计划（学生自学用）

1) 引言

编写目的：　明编写　件计划的目的，指出读者对象。

　目背景：包括　目委托单位、开发单位和主管　　；该系统与其他系统的关系。

定义：列出本文档中用到的专　术语的定义和缩略词的原文。

参考资料：包括　目经核准的计划任务书、合同或上级机关的批文；文档所引用的资料、规范等；列出资料的作者、标　、编号、发表日期、出版单位或资料来源。

2)　目概

简要说明　目的各　主要工作，介绍所开发　件的功能性能等。

条件与　制：　明为完成　目应具备的条件开发单位已具备的条件以及尚　创　的条件。必要时　应说明用户及分合同承包者承担的工作、完成期　及其他条件与　制。



（学生自学用）

产品：

(1)程序：列出应交付的程序名称使用的语言及存储形式。

(2)文档：列出应交付的文档。

(3)　行环境：应包括硬件环境和　件环境。

(4)服务：　明开发单位可向用户提供的服务，如人员培训、安装、维护和其他　行支持。

(5)　收标准。

3) 实施计划

任务分解：任务的划分及各　任务的负责人。

　度：按　段完成的　目，用图表说明开始时　完成时　。

　算：描　本　目的总费用概算。

关　　：说明可能影响　目的关　　，如设备条件技术　点或其他　　因素，并说明对策。

人员组织及分工。

4) 交付期

文档实例2：软件需求规格说明（学生自学用）

1) 引言

(1)编写目的

(2)　目背景

(3)定义

(4)参考资料

2) 任务概

(1)目标

(2)　行环境

(3)条件与　制

(4)数据描　，包括：　态数据；动态数据(包括　入数据和　出数据)；数据库描　(给出使用数据库的名称和类型)；数据字典；数据　　。

（学生自学用）

(5)功能　求，包括：功能划分；功能描　。

(6)性能　求，包括：数据精确度；时　特性(如响应时　、更新处理时　、数据转化与传时　、　行时　等)；　应性(在操作方式、行环境与其他　件的接口以及开发计划等发生变化时，应具有的　应能力)。

(7)　行　求，包括：用户界　(如屏幕格式、报表格式、菜单格式、　入　出时　等)；硬件接口；　件接口；故　处理。

(8)其他　求：如可使用性、安全保密性、可维护性、可移植性等。

文档实例3：测试计划（学生自学用）

一、简介
1.1编写目的
1.2　目背景
1.3测试范围
1.4参考文档
二、测试参考文档和测试提交文档
2.1测试参考文档
2.2测试提交文档
三、测试　度
四、测试资源
4.1人力资源
4.2测试环境
4.3测试工具
五、测试策略

6.2.4 软件文档的标准化检查

件文档

正确性

完整性

一致性（文档与文档、文档与程序）

及时性

文档中的章节编号
（学生自学用）

要求使用：

1

1.1

1.1.1

不可以使用：

一　　（一）　　（1）　　1）

文档中关于图的说明
（学生自学用）

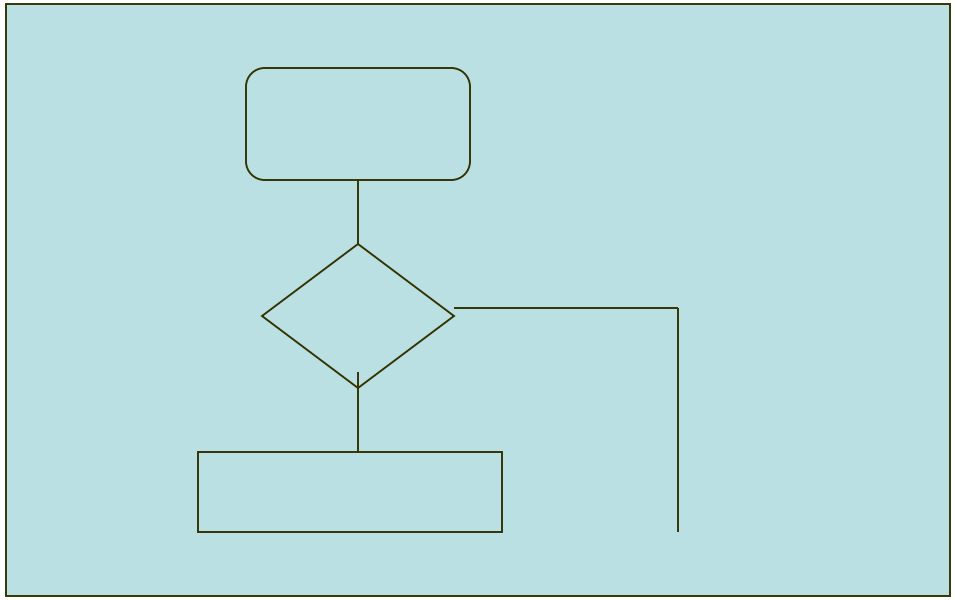


图 2　流程图实例

文档中关于表的说明
（学生自学用）

表 5　成绩单

6.3 软件质量认证

1. 质　认证与质　保证的区别？
2. 你　知　有哪些质　认证？

- ISO9000（1987）
- CMM（1990）、CMMI（2001）
- ISO27001（2005）
- 其它？

6.3.1 ISO 9000 标准概述

ISO9000

- 名称：质　管理和质　保证标准系列
- 时　：1987年3月
- 目的：为了满足国　贸易的　要。

- 供方：
- 核心：对供方的质　要求和管理方法

中国获得ISO认证承认：　　1998年1月

发展历程

ISO9000：1987	ISO9000：1994	ISO9000：2000
●历史性综合 ●6项标准 ●主要针对制造业由其是较大企业	●战术性换版 ●22项标准 ●补充内容、扩大范围 ●开始考虑服务业应用 ●按产品导向建立20个要素	●战略性换版 ●5项加技术报告书 ●充分考虑一切组织,包括小企业、服务业 ●按过程导向建立PDCA循环

- 2000年12月15日，2000版的ISO9000族标准正式发布实施，2000版ISO9000族国　标准的核心标准共有四个：

1、ISO9000:2000 质　管理体系 ——基础和术语；

2、ISO9001:2000 质　管理体系——要求；

3、ISO9004:2000 质　管理体系——业绩改　指南；

4、ISO19011:2000质　和环境管理体系审核指南。

我国等同　用ISO9000系列标准，于2001年12月28日发布，**2001年6月30日实施，标准代号**

GB/T19000

GB/T19001

GB/T19004

ISO 9000标准的特点

(1) 国　性

120多个国家和地区　用为国家标准

(2) 完整性

20个标准

(3) 兼容性

设计、生产、管理、　售、服务

(4) 主动性

生产者推动

(5) 可信性

独立性

标准化

(6) 指导性

质　管理和质　保证的原则

(7) 科学性

现代质　管理的科学原理

(8) 实践性

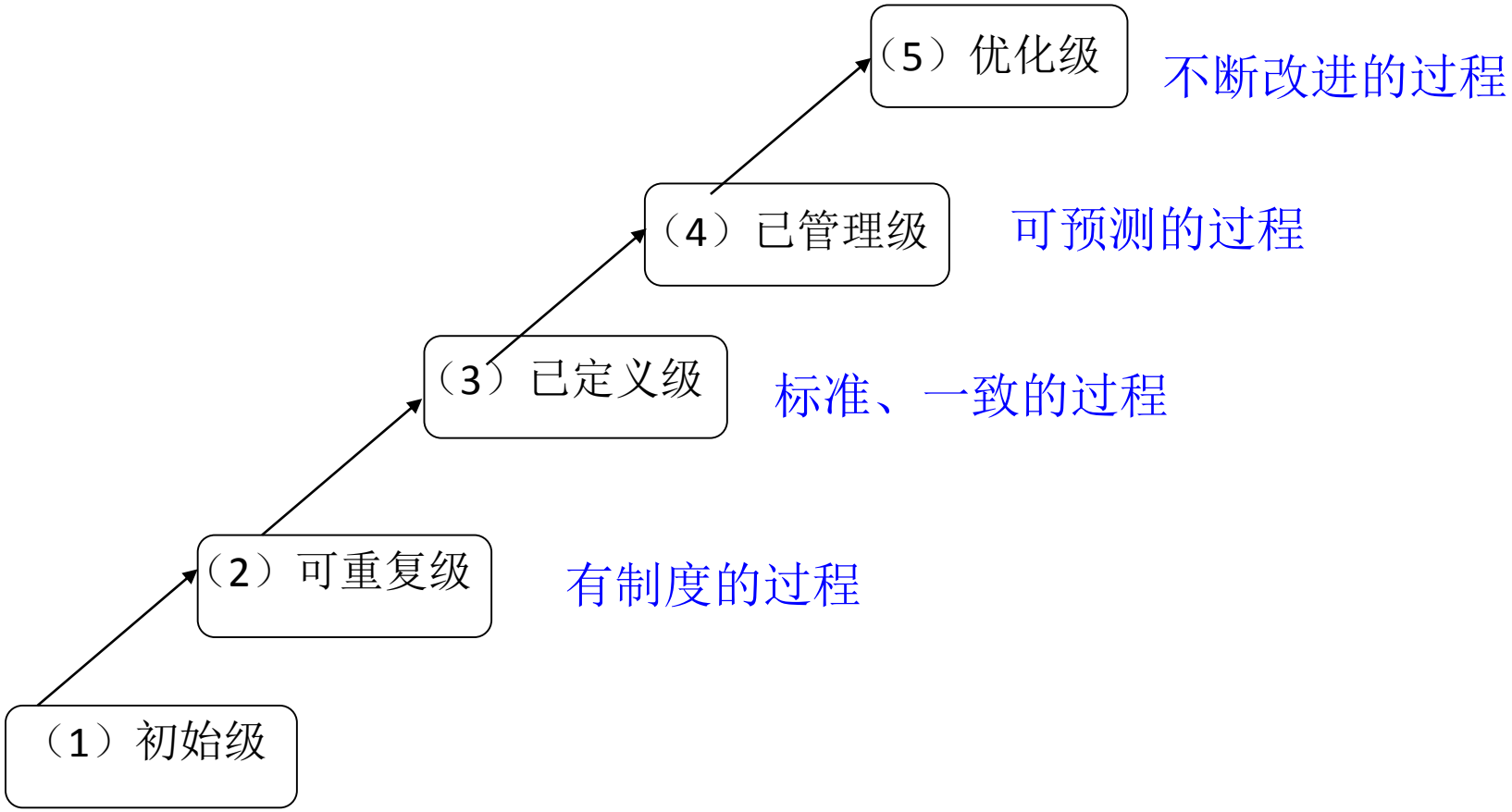
基于国外大　的质　管理实践

有关ISO的资料

6.3.2 CMM/CMMI 概述

① 什么是CMM?

CMM(Capability Maturity Model)即能力成熟度模型，是美国卡内基•梅　大学　件工程研究所(SEI)在美国国（DoD）资助下，于20世纪80年代末建立的，用于评价件机构的　件　程能力成熟度。


（1）初始级
（2）可重复级
有制度的过程
（3）已定义级
标准、一致的过程
（4）已管理级
可预测的过程
（5）优化级
不断改进的过程

第一级（初始　）：无序

第二级（可　复级）：　件　目的策划和跟踪是稳定的，提供了可　复以前成功实践的　目环境。

第三级（已定义级）：定义了完整的　件　程(　程模型)，并在　件　程工程活动和　件　目工程活动中，实现了标准化和文档化。

第四级（已管理级）：建立了定　的质　目标，所有　目的　要　程活动　是可度　的。

第五级（优化级）：　件机构能够　中精力　行不断的　程改　。

从CMM到CMMI

- CMMI在CMM的基础上增加了　成的产品和　程开发（IPPD）等专业　域，不再局　于　件。
- 增加的有：系统工程能力成熟度模型（SE-CMM），件　购能力成熟度模型（SA-CMM），　成产品开发能力成熟度模型（IPD-CMM）

也分为5级

- 1. 级别1（初始级）
  2. 级别2（已管理级）
  3. 级别3（严格定义级）
  4. 级别4（定　管理级）
  5. 级别5（优化级）

6.3.3 ISO27001

- 什么是ISO27001

信息安全管理标准

它来源于英国标准协会（British Standards Institute，BSI）于1995年2月制定的信息安全管理标准--BS7799，BS7799分两个　分，

BS7799-1，信息安全管理实施规则

BS7799-2，信息安全管理体系规范。

其第一　分于2000年被ISO组织　纳，正式成为ISO/IEC 17799标准。该标准2005年经　最新改版，发展成为ISO/IEC 17799:2005标准。

BS7799标准的第二　分经　　时　讨论修订，也于2005年成为正式的ISO标准，即ISO/IEC 27001:2005。

- 是建立信息安全管理体系（ISMS）的一套规范（Specification for Information Security Management Systems），其中详细说明了建立、实施和维护信息安全管理体系的要求，指出实施机构应该　循的　　评估标准。

是信息与　件企业最　要的资质之一


SHANGHAI JIAO TONG UNIVERSITY
1896

?

- 1、 P（Plan）-计划，确定方　和目标，确定活动计划；
  2、 D（Do）-执行，实地去做，实现计划中的内容；
  3、 C（Check）-检查，总结执行计划的结果，注意效果，找出　　；
  4、 A（Action）-行动，对总结检查的结果　行处理，成功的经　加以肯定并　当推广、标准化；失败的教训加以总结，以免　现，未解决的　　放到下一个PDCA循环。
- 在质　管理中，PDCA循环得到了广泛的应用，也称PDCA循环是质　管理的基本方法。

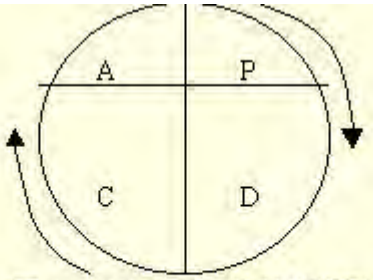


图 1　PDCA 循环的基本模型

Return



1. GB/T1526: 1989 信息处理—数据流程图、程序流程图、系统流程图、程序网络图和系统资源图的文件编制符号及约定 有效 ISO 5807 1985
2. GB/T8566: 2001 信息技术　件生存周期　程 修订 ISO/IEC 12207 - 1995
3. GB/T8567: 1988 计算机　件产品开发文件编制指南 修订
4. GB/T9385: 1988 计算机　件　求说明编制指南 修订 ANSI/IEEE 830 - 1984
5. GB/T9386: 1988 计算机　件测试文件编制规范 修订 ANSI/IEEE 829 - 1983
6. GB/T11457: 1995　件工程术语 修订 IEEE 729
7. GB/T12504: 1990 计算机　件质　保证计划规范 废止 IEEE 730
8. GB/T12505: 1990 计算机　件　置管理计划规范 废止 IEEE 828
9. GB/T13502: 1992 信息处理 程序构　及其表示的约定 有效 ISO 8631 - 1989

10. GB/T14079: 1993 计算机　件维护指南 废止
11. GB/T14085: 1993 信息处理系统 计算机系统　置图符号及其约定 有效 ISO 8790 - 1987
12. GB/T14394: 1993 计算机　件可　性和维护性管理 修订
13. GB/T15189: 1994 DOS中文信息处理系统接口规范
14. GB/T15532: 1995 计算机　件单元测试 修订
15. GB/T15535: 1995 信息处理 单命中判定表规范 有效 ISO 5806 - 1984
16. GB/T15538:　件工程标准分类法 废止
17. GB/T15697: 1995 信息处理 按记录组处理　序文卷的程序流程 废止 ISO 6593 - 1985
18. GB/T15853: 1995　件支持环境 废止
19. GB/T16260: 1996 信息技术　件产品评价 质　特性及其使用指南 修订 ISO/IEC 9126 - 1991

20. GB/T16680: 1996　件文档管理指南 有效 ISO/IEC TR 9294 - 1990
21. GB/T17544: 1998 信息技术　件包 质　要求和测试 有效 ISO/IEC 12119 - 1994
22. GB/T18234: 2000 信息技术 CASE工具的评价与　择指南 有效 ISO/IEC 14102 - 1995
23. GB/T18491.1: 2001 信息技术　件测　功能规模测　第1　分：概念定义 有效 ISO/IEC 14143-1 - 1998
24. GB/T18492: 2001 信息技术 系统及　件完整性级别 有效 ISO/IEC 15026 - 1998
25. GB/T18493: 2001 信息技术　件生存周期　程指南 有效 ISO/IEC 15271 - 1998



26. GB/T18905.1: 2002　件工程 产品评价 第1　分：概　有效 ISO/IEC 14598-1 - 1999
27. GB/T18905.2: 2002　件工程 产品评价 第2　分：策划和管理 有效 ISO/IEC 14598-2 - 2000
28. 28 GB/T18905.3:2002　件工程 产品评价 第3　分：开发者用的 程 有效 ISO/IEC 14598-3 - 2000
29. 29 GB/T18905.4: 2002　件工程 产品评价 第4　分：　方用的　程 有效 ISO/IEC 14598-4 - 1999
30. 30 GB/T18905.5: 2002　件工程 产品评价 第5　分：评估者用的 程 有效 ISO/IEC 14598-5 - 1998
31. 31 GB/T18905.6: 2002　件工程 产品评价 第6　分：评估模块的文档编制 有效 ISO/IEC 14598-6 - 2001
32. 32 GB/T18914: 2002 信息技术　件工程 CASE工具的　用指南 ISO/IEC 14471 – 1999

return



Chapter 7

Structure Test



7.1 Path Testing

7.2 Data Flow Testing

7.3 Retrospective on Unit Testing



7.1 Path Testing

7.1.1 DD（decision-to decision Path）-Paths

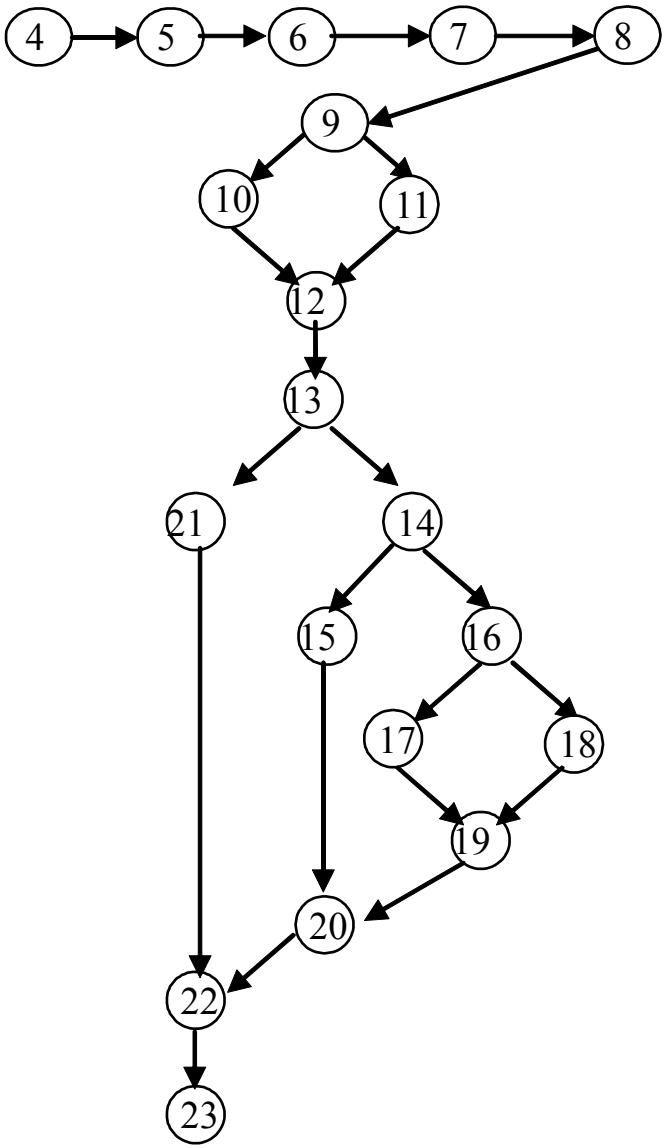
program graph is a directed graph in which nodes are statement fragments, and edges represent flow of control.

for a given program, several distinct program graphs might be used, all of which reduce to a unique DD-path graph.

A program graph of the triangle problem is given in Figure 8.2.

程序伪代码(5分钟程序阅读)

```
1 Program triangle2
2 Dim a,b,c As Integer
3 Dim IsATrinagle As Boolean
4 Output("Enter 3 integers which are sides of a
        triangle")
5 Input(a,b,c)
6 Output("Side A is", a)
7 Output("Side B is", b)
8 Output("Side C is", c)
9 If (a < b + c) AND (b < a + c) AND (c < a + b)
10     Then IsATriangle = True
11      Else IsATriangle = False
12 EndIf
13 If IsATriangle
14     Then If (a = b) AND (b = c)
15           Then Output ("Equilateral")
16           Else If (a ≠ b ) AND (a ≠ c ) AND (b ≠ c )
17                Then Output ("Scalene")
18                Else Output ("Isosceles")
19                EndIf
20           EndIf
21 Else Output("Nota a Triangle")
22 EndIf
23 End triangle2
```





分支覆盖(branch coverage)：？

路径覆盖(path coverage)：？

- **分支覆盖**又称为判定覆盖，它要求设计足够多的测试用例，使得程序中每个判定至少有一次为真值，有一次为假值，即：程序中的每个分支至少执行一次。
- **路径覆盖**,它要求设计足够多的测试用例，使得程序中所有可能的执行路径都被执行一次。

- branch coverage：

1. abd
2. ace

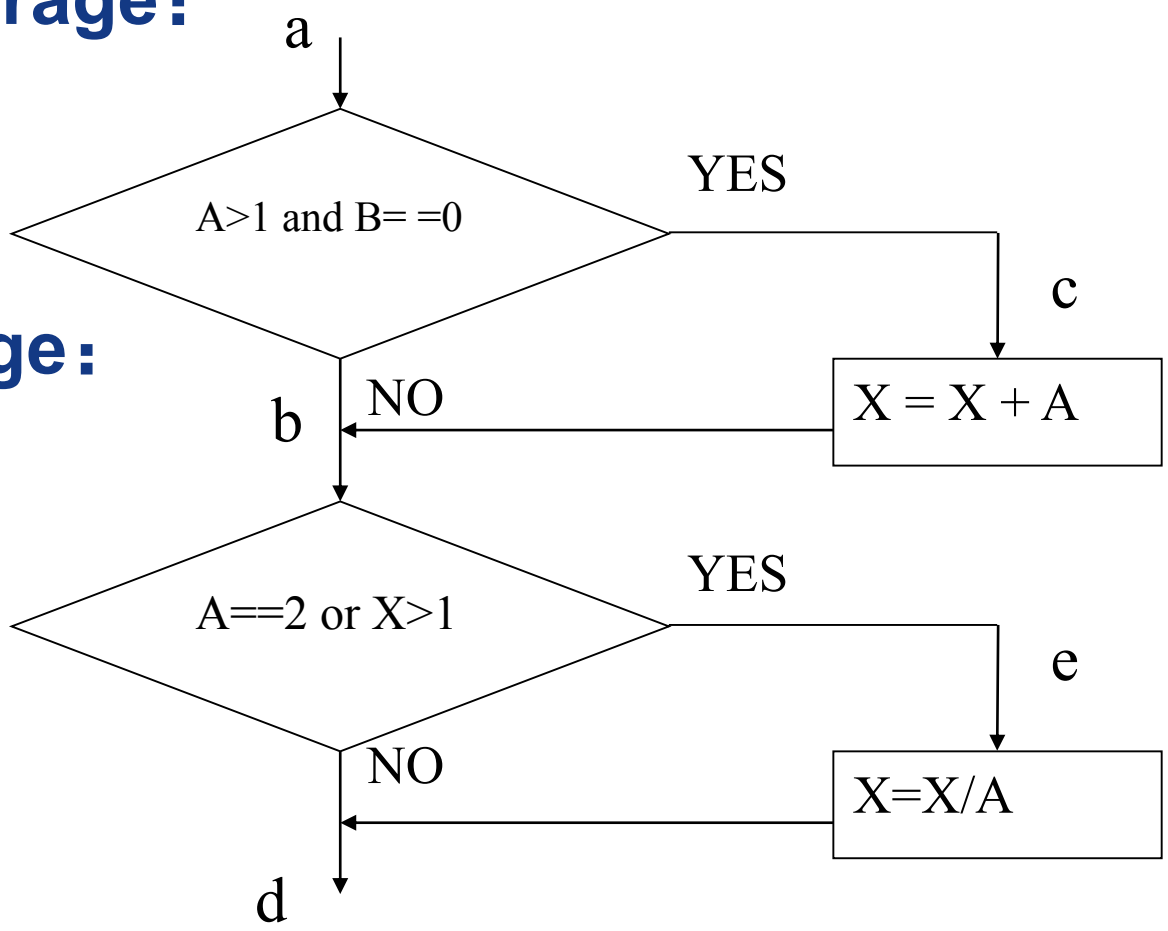
- path coverage：

1. abd、abe
2. acd、ace

进一步考虑：

A>1？、B==0？

A==2?、X>1?

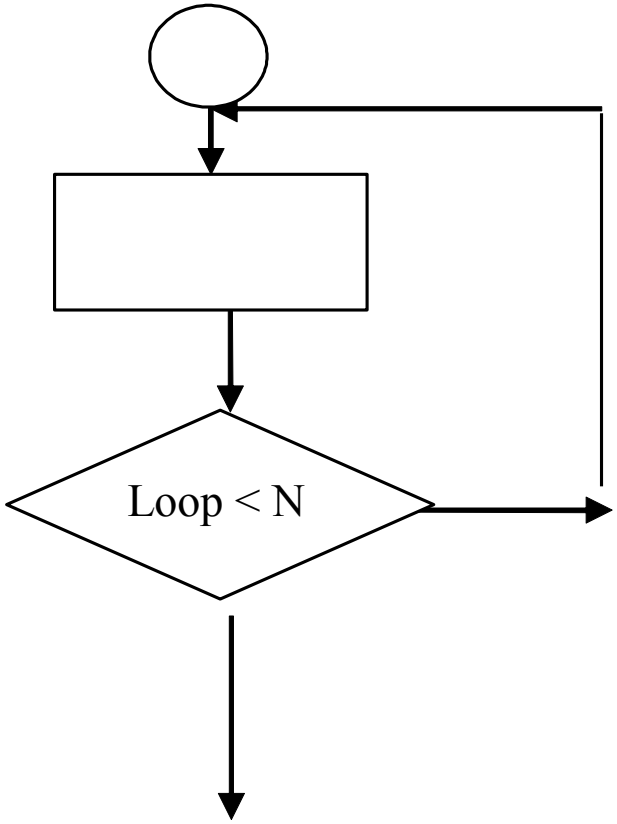


一共有2X2X2X2=16条不同路径！Predicate test

Loop：

branch coverage：2

path coverage：N+1



- Figure 8.3 is a graph of a simple program; it is typical of the kind of example detractors use to show the impossibility of completely testing even simple programs.
- If the loop may have up to 18 repetitions, some 5^{18} = 4.77 trillion distinct program execution paths exist.

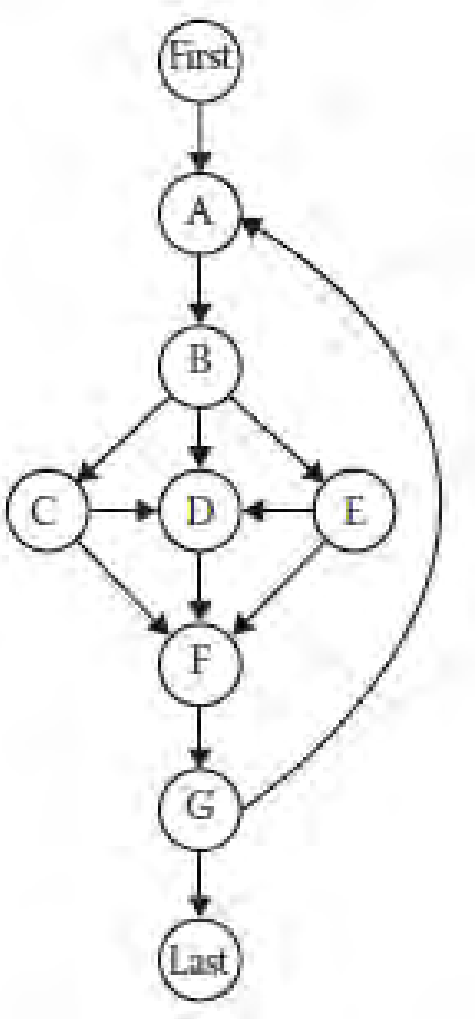


Figure 8.3 Trillions of paths.



DD-Path Definition

- **A DD-path is a sequence of nodes in a program graph such that**

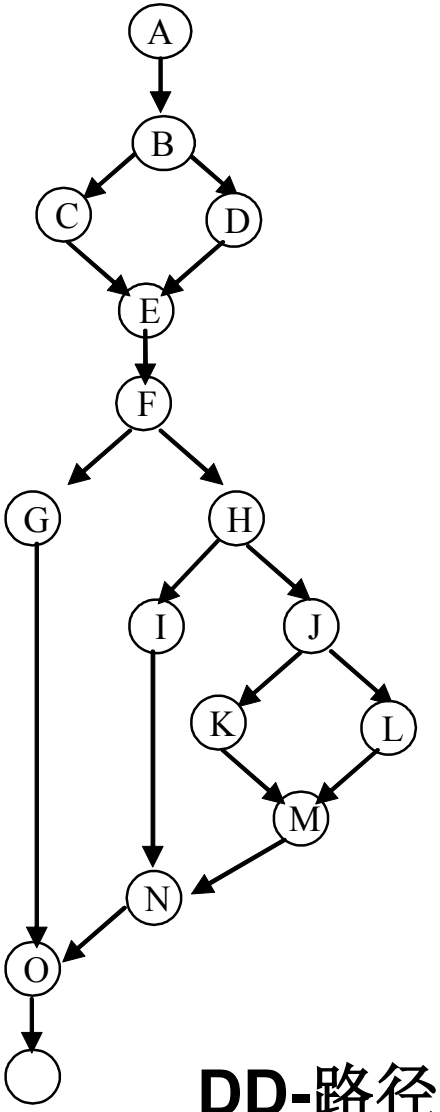
  Case 1: It consists of a single node with indeg = 0.

  Case 2: It consists of a single node with outdeg = 0.

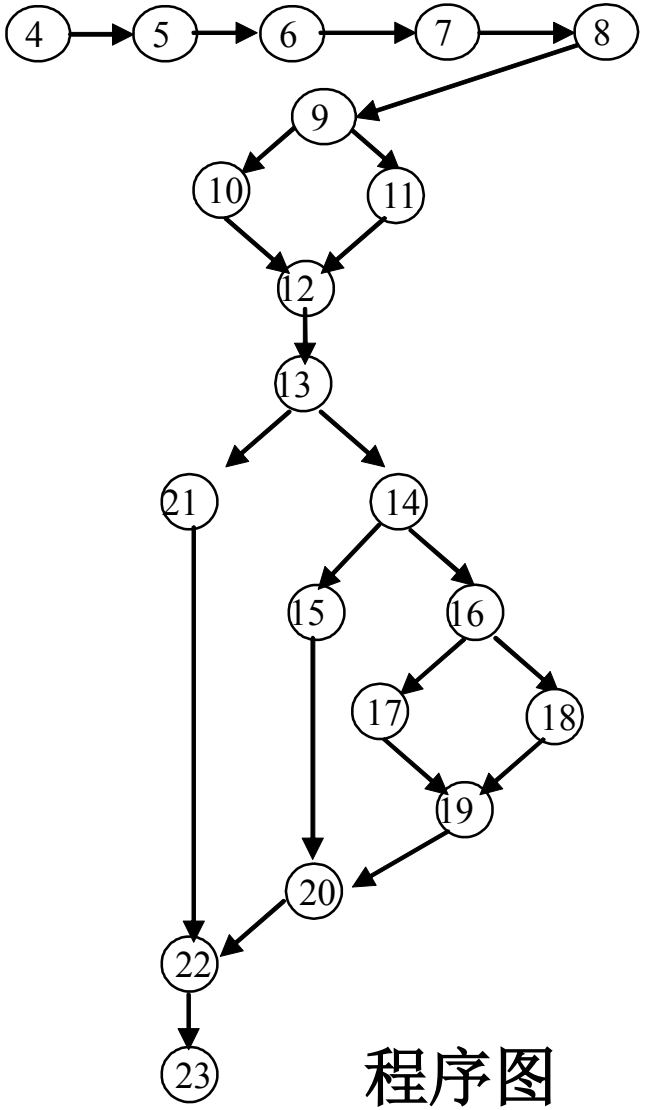
  Case 3: It consists of a single node with indeg ≥ 2 or outdeg ≥ 2.

  Case 4: It consists of a single node with indeg = 1 and outdeg = 1.

  Case 5: It is a maximal chain of length ≥ 1.



DD-路径图



程序图



In practice, it is reasonable to manually createDD-path graphs for programs up to about 100 source lines. Beyond that, most testers look for a tool.

7.1.2 测试覆盖指标

G_{node} : node coverage

G_{edge}: edge coverage

7.1.2 测试覆盖指标

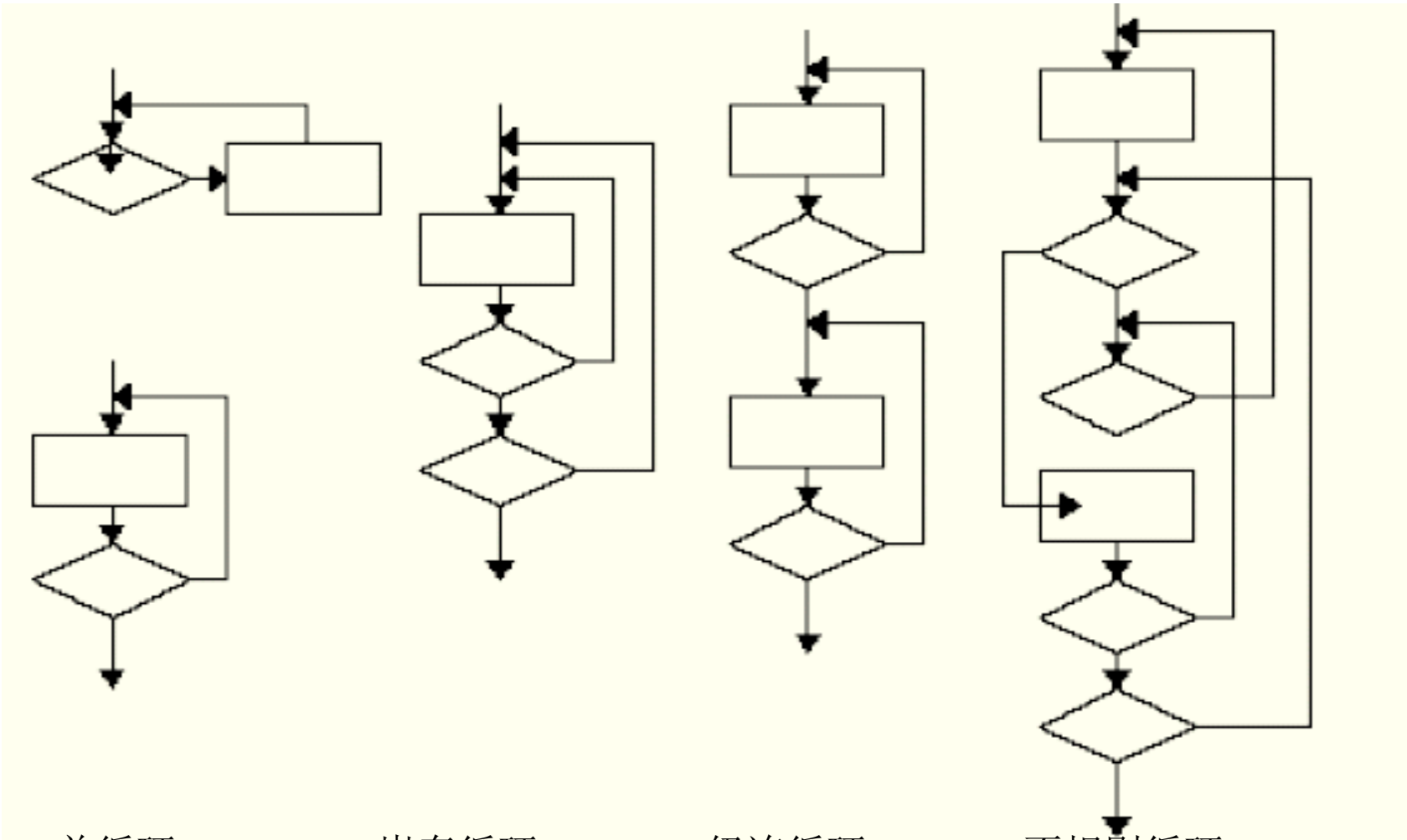
指标	覆盖描述
C_0	Every statement（语句覆盖）
C_1	Every DD-path（分支覆盖）
C_1p	Every predicate to each outcome（条件判断）
C_2	C1 coverage + loop coverage
C_d	C1 coverage + every dependent pair of DD-paths
C_{mcc}	Multiple condition coverage
C_ik	Every program path that contains up to k repetitions of a loop (usually k = 2)
C_{start}	“Statistically significant” fraction of paths
C_∞	All possible execution paths



- DD-路径覆盖：最低要求
- 多条件覆盖：

  组合条件、MCDC 下一讲介绍。
- 循环覆盖：？
- 全路径覆盖：穷尽覆盖

循环测试



单循环　　嵌套循环　　级连循环　　不规则循环

- **单循环测试**

假设循环次数为N

a. 直接跳过循环

b. 循环次数为1

c. 循环次数为2

d. 循环次数为M，M<N

e. 循环次数为N-1，N，N+1

- **嵌套循环测试**

a. 先测试最内部循环，其它循环次数为1

b. 测试第二层循环，其它循环次数为1，

c. 直到最外部循环完成测试

- **级连循环测试**

分别采用单循环测试方法进行测试

- **不规则循环测试　无法测试——重新设计！**

7.1.3 McCabe's Basis Path Method

- 基路径的概念与方法

“基”：向量空间的概念，向量空间的基是相互独立的一组向量，基“覆盖”整个向量空间。使得该空间中的任何其它向量都可以用基向量表示。

基路径：程序图中相互独立的一组路径，使得该程序中的所有路径都可以用基路径表示。

Cyclomatic number (圈数) of a strongly connected graph is the number of linearly independent circuits in the graph.

McCabe圈数V（G）计算方法：

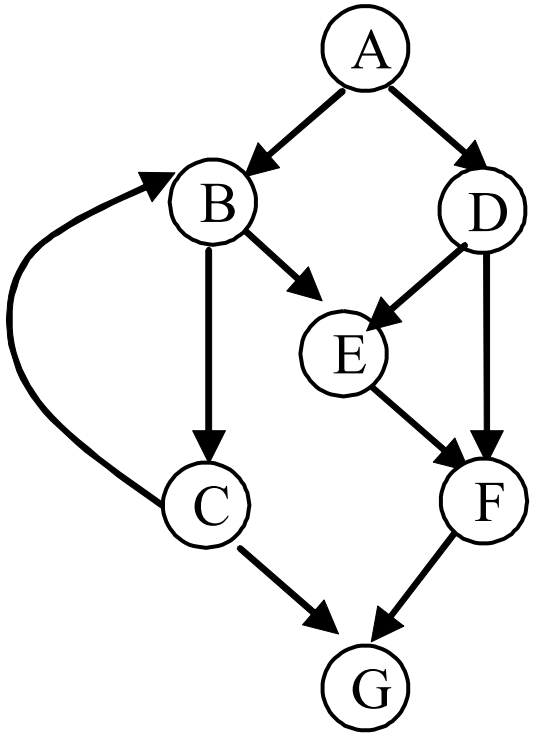
V（G）=e-n+2p

e：边数；

n：节点数

p：连接区域数

对于右图： V（G）=10-7+2p=5



为避免混乱，不再说明强连接有向图的计算公式(不满足单入口、单出口条件)。

- 5 independent paths：

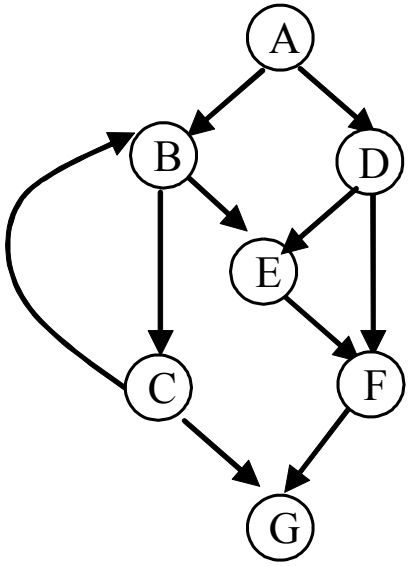
P1：A，B，C，G；

P2：A，B，C，B，C，G；

P3：A，B，E，F，G；

P4：A，D，E，F，G；

P5：A，D，F，G。



Any path could be a combination of 5 independent paths.

Example ：A、B、C、B、E、F、G = P2+P3-P1

其中，加法是一条路径后接一条路径；乘法是路径的重复，而减法则只有数学上去除边的含义，而缺少实际意义。

=A，B，C，B，C，G+ A，B，E，F，G- A，B，C，G ；

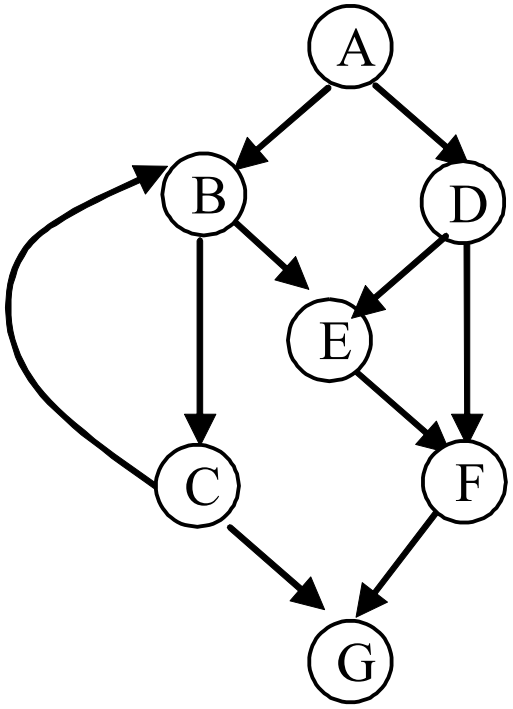
AB+BC+CB+BC+CG+AB+BE+EF+FG-AB-BC-CG

= A、B、C、B、E、F、G

如何寻找McCabe路径？

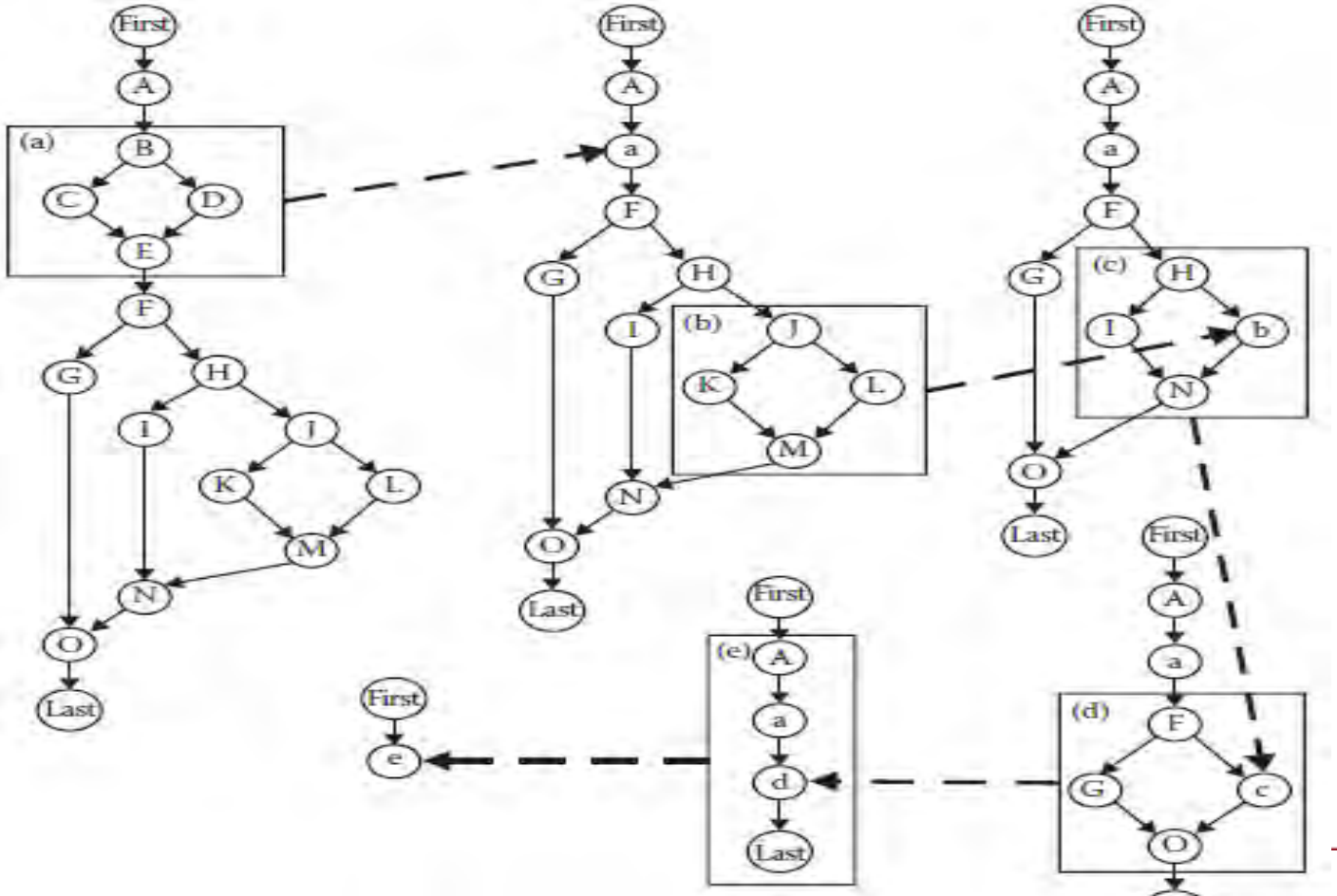
图的搜索与遍历算法！

- 深度优先算法
- 广度优先算法



Essential Complexity

In general, when aprogram is well structured (i.e., is composed solely of the structured programming constructs), it can always be reduced to a graph with one path.



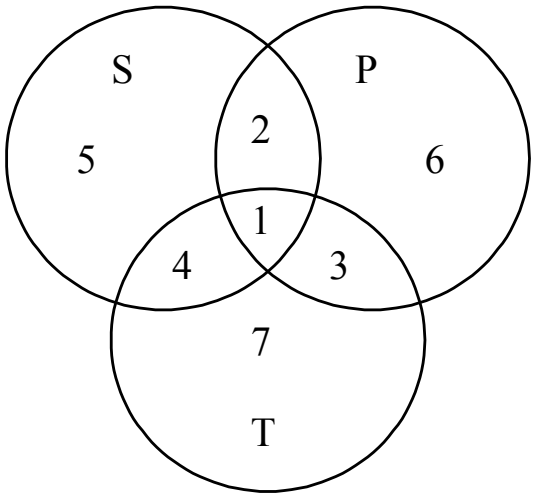
7.1.4 Insights from a Venn Diagram

基于路径的测试，从另一个角度去考虑程序测试的完备性，但是没有考虑数据功能，即使满足DD-路径覆盖，和考虑圈复杂度指标，也仅是测试的下限指标。

如左图：S：已说明行为的路径集合；

P：已编程实现的路径集合；

T：拓扑可达的路径集合；



区域：1：测试的重点，可测；

2、6：已实现的，就应该是拓扑可行的，

3：已实现和可达的，应该是有用的！

4：已描述的，拓扑可达，却没有实现的；要增加，可测；

5：没有实现的已说明路径，不可测，

7：未说明，未实现，拓扑可达，测试无意义！

可行的和在拓扑结构上可能的路径



7.2 Data Flow Testing

Data flow testing refers to forms of structural testing that focus on the points at which variables receive values and the points at which these values are used.

7.2.1 Define/Use Testing

1. The program graph G(P) is constructed as in Chapter 4, with statement fragments as nodes and edges that represent node sequences.
2. Node n ∈ G(P) is a defining node of the variable v ∈ V, written as DEF(v, n), if and only if the value of variable v is defined as the statement fragment corresponding to node n.



3. Node n ∈ G(P) is a usage node of the variable v ∈ V, written as USE(v, n), if and only if the value of the variable v is used as the statement fragment corresponding to node n.
4. A usage node USE(v, n) is a predicate use (denoted as P-use) if and only if the statement n is a predicate statement; otherwise, USE(v, n) is a computation use (denoted C-use).
5. A definition/use path with respect to a variable v (denoted du-path) is a path in PATHS(P) such that, for some v ∈ V, there are define and usage nodes DEF(v, m) and USE(v, n) such that m and n are the initial and final nodes of the path.



6. A definition-clear path with respect to a variable v (denoted dc-path) is a definition/use path in PATHS(P) with initial and final nodes DEF(v, m) and USE(v, n) such that no other node in the path is a defining node of v.

Example the commission problem
（P162 Figure 9.1 8分钟阅读）

```
7.  lockPrice=45.0
8.  stockPrice=30.0
9.  barrelPrice=25.0
10. totalLocks=0
11. totalStocks=0
12. totalBarrels=0

13. Input(locks)
14. While NOT(locks==-1)
15. Input(stocks,barrels)
16. totalLocks=totalLocks+Locks
17. totalStocks=totalStocks+stocks
18. totalBarrels=totalBarrels+barrels
19. Input(locks)
20. ENDWhile
```



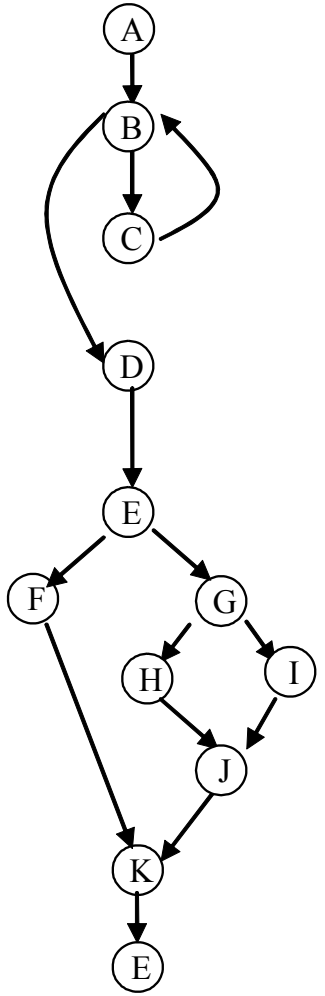
```
21. Output("Locks sold:",totalLocks)
22. Output("Stocks sold:",totalStocks)
23. Output("Barrels sold:",totalBarrels)
24. lockSales=lockPrice*totalLocks
25. stockSales=stockPrice*totalStocks
26. barrelSales=barrelPrice*totalBarrels
27. Sales=lockSales+stockSales+barrelDSales
28. Output("Total Sales: ",Sales)

29. If (sales>1800)
30. Then
31.   commission=0.10*1000.0
32.   commission=commission+0.15*800.0
33. Commission=commission+0.20*(sales-1800.0)
34. Elseif(sales>1000.0)
```

```
35.    Then
36.       commission=0.10*1000.0
37.       commission=commission+0.15*(sales-1000.0)
38.      Else     commission=0.10*sales
39.      Endif
40. Endif
41. Output("commission is $",commission)
42. End
```

Variable name	defining nodes	usage nodes
lockPrice	7	24
stockPrice	8	25
barrelPric	9	26
totalLocks	10,16	16,21,24
totalStock	11,17	17,22,25
totalBarrels	12,18	18,23,26
locks	13,19	14,16
stocks	15	17
barrels	15	18
locksales	24	27
stocksales	25	27
barrelsales	26	27
sales	27	28,29,33,34,37,38
commission	31,32,33,36,37,38	32,33,37,41

DD-Path



Du-paths

1.stocks: have DEF(stocks,15) and USE(stocks,17), the path <15,17> is a du-path with respect to stocks, this path is also definition clear.

2. locks:haveDEF(locks,13)、DEF（locks，19）USE（locks，14）、USE（locks，16）。

These yield four du-paths：

p1=<13,14>

p2=<13,14,15,16>

p3=<19,20,14>

p4=<19,20,14,15,16>

3.totalLocks:With two defining nodes, DEF(totalLocks,10) and DEF(totalLocks,16); and three usage nodes,USE(totalLocks,16)、USE(totalLocks，21) and USE(totalLocks，24).

six du-paths：

p1=<10,11,12,13,14,15,16> is definition clear.

p2=<10,11,12,13,14,15,16,17,18,19,20,21> since the possible repetition of the while loop, totalLocks will be redefined, the path is not definition clear.

p3=<10,11,12,13,14,15,16,17,18,19,20,21,22,23,24>

=<p2,22,23,24> , is not definition clear.

p4=<16,16> is not du path



p5=<16,17,18,19,20,21>

p6=<16,17,18,19,20,21,22,23,24>

Both have the loop iteration problem, so both are not definition clear.

Other Variables analysis are read by students yourselves.



Define/Use Test Coverage Metrics

- All-Defs criterion: for the program P if and only if for every variable v ∈ V, T contains definition-clear paths from every defining node of v to a use of v.
- All-Uses criterion: for the program P if and only if for every variable v ∈ V, T contains definition-clear paths from some defining node of v to every use of v, and to the successor node of each USE(v, n).
- All-P-Uses/Some C-Uses criterion: for the program P if and only if for every variable v ∈ V, T contains definition-clear paths from every defining node of v **to every predicate use of v**; and if a definition of v has no P-uses, a definition-clear path leads to at least one computation use.
- All-C-Uses/Some P-Uses criterion: for the program P if and only if for every variable v ∈ V, T contains definition clear paths from every defining node of v **to every computation use of v;** and if a definition of v has no C-uses, a definition-clear path leads to at least one predicate use.

- All-DU-paths criterion: for the program P if and only if for every variable v ∈ V, T contains definition-clear paths **from every defining node of v to every use of v** and to the successor node of each USE(v, n), and that these paths are either single loop traversals or they are cycle free.

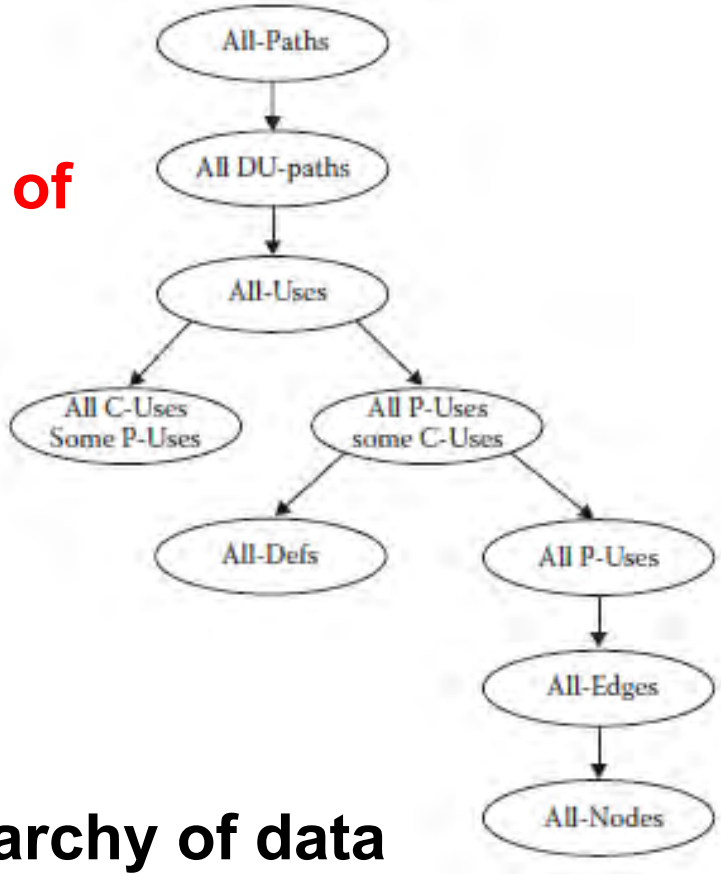


Figure 9.5 hierarchy of data flow coverage metrics.



7.2.2 Slice-Based Testing

Definition:Given a program P and a set V of variables in P, a slice on the variable set V at statement n, written S(V, n), is the set of all statement fragments in P that contribute to the values of variables in V at node n.

five forms of usage:

- **P-use used in a predicate (decision)**
- **C-use used in computation**
- **O-use used for output**
- **L-use used for location (pointers, subscripts)**
- **I-use iteration (internal counters, loop indices)**

Example

- Variable locks have 2 uses node 14、16，2 defining node 13、19，then：
  - S1：S（locks,13）={13}
  - S2：S（locks,14）={13,14,19,20}
  - S3：S（locks,16）={13,14,16,19,20}
  - S4：S（locks,19）={19}
- stocks and barrels variables to be influenced by the loop variable locks, so
  - S5:S(stocks,15) ={15}?
  - S5:S(stocks,15)={13,14,15,19,20}
  - S6:S(stocks,17) ={13,14,15,17,19,20}
  - S7:S(barrels,15) ={13,14,15,19,20}
  - S8:S(barrels,18) ={13,14,15,18,19,20}

- totalLocks: node 10 is defining node, node 16 is defining and uses node, also to be influenced by the loop variable locks. node 21 and 24 are output and computation node. Then：

  S9：S（totalLocks,10）={10}

  S10：S(totalLocks,16) ={10,13,14,16,19,20}

  S11:S(totalLocks,21)=S(totalLocks,24)=S10

- Variables totalStocks and totalBarrels are similar to totalLocks.

- lockPrice，stockPrice、barrelPrice are 1 defining node and 1 uses node，no loop，no more explain.
- Variable sales have 1 defining node 27，28、29、33、34、37、38 total 6 uses nodes，but it is influenced by all variables except commission. So,

  Variable sales have the same Slice on all it's uses nodes.

$S_{24\sim30}$:
={7,8,9,10,11,12,13,14,15,16,17,18,19,20,24,25,26,27}

- Variable commission have 6 defining nodes，4 uses nodes. So,

  S_{31}：S（commission，31）={31}

  S_{32}：S（commission，32）={31，32}

  S_{33}：since include variable sales, so, S(commission,33）={7~33}

  S_{34}：S（commission，36）={36}

  S_{35}：S（commission，37）={7~37}

  S_{36}：S（commission，38）={7~38}

  S_{37}：S（commission，41）={7~41}

Looking at slices as sets of fragment numbers is correct in terms of our definition, but it is also helpful to see how slices are composed of sets of previous slices.

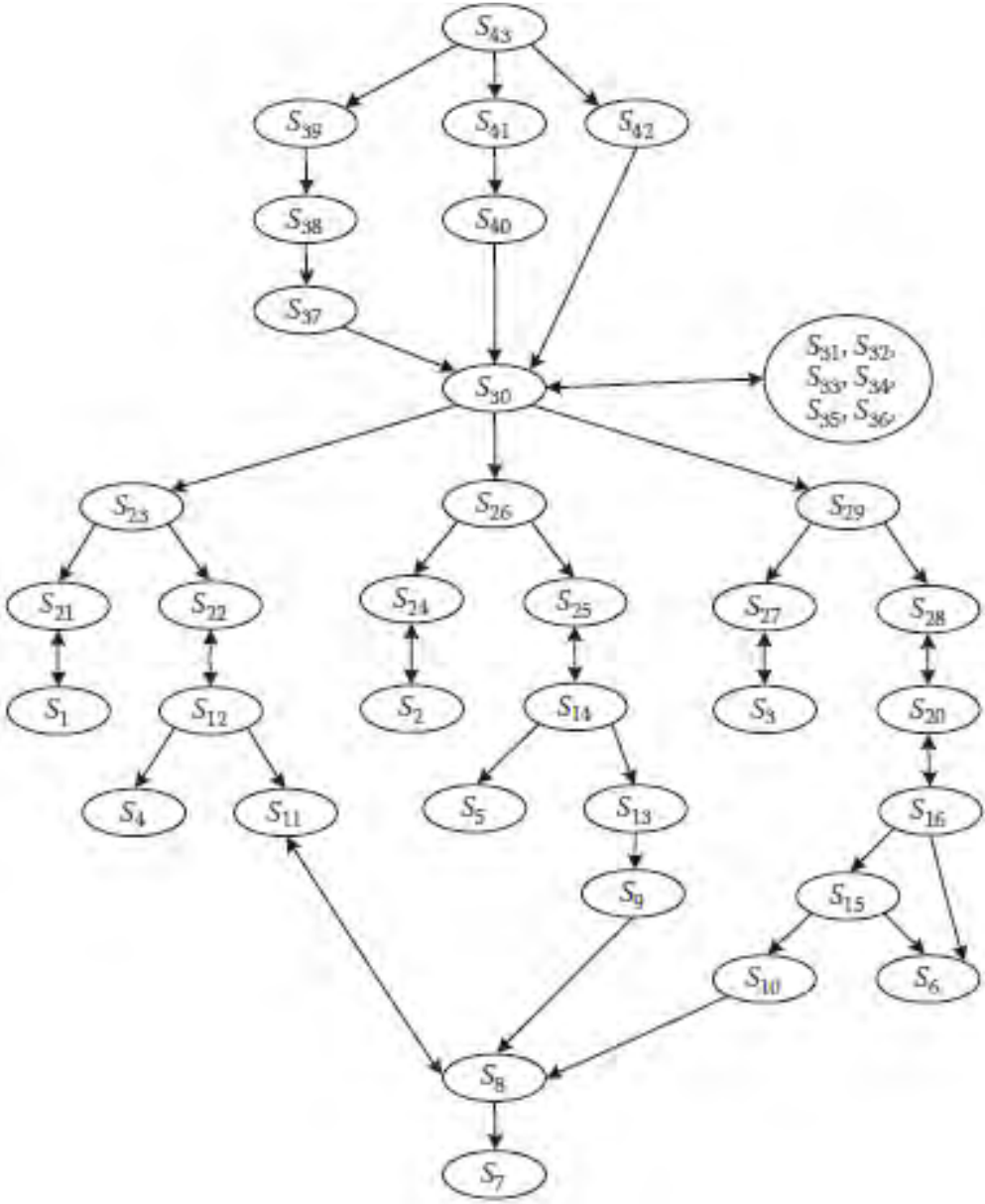


Figure 9.9 Full lattice on commission.

- 不同变量程序片的补集，为变量的分段划分提供了方法。

如：S（commission，41）-S（sales，28）

={29~41}；

将commission的计算分为sales计算和之后的计算，为错误的定位提供了依据。



7.3 Retrospective on Unit Testing

When should unit testing stop? Here are some possible answers:

1. When you run out of time
2. When continued testing causes no new failures
3. When continued testing reveals no new faults

基于经验的有效方法

4. When you cannot think of any new test cases—reason?
5. When you reach a point of diminishing returns-基于分析的方法
6. When mandated coverage has been attained-good
7. When all faults have been removed-excellent

7.3.1 Traversing the Pendulum

The triangle program

There are 88 topologically possible paths, but only 11 of these are feasible.

P1:1-2-3-4-5-6-7-13-16-18-20

P2:

P3:

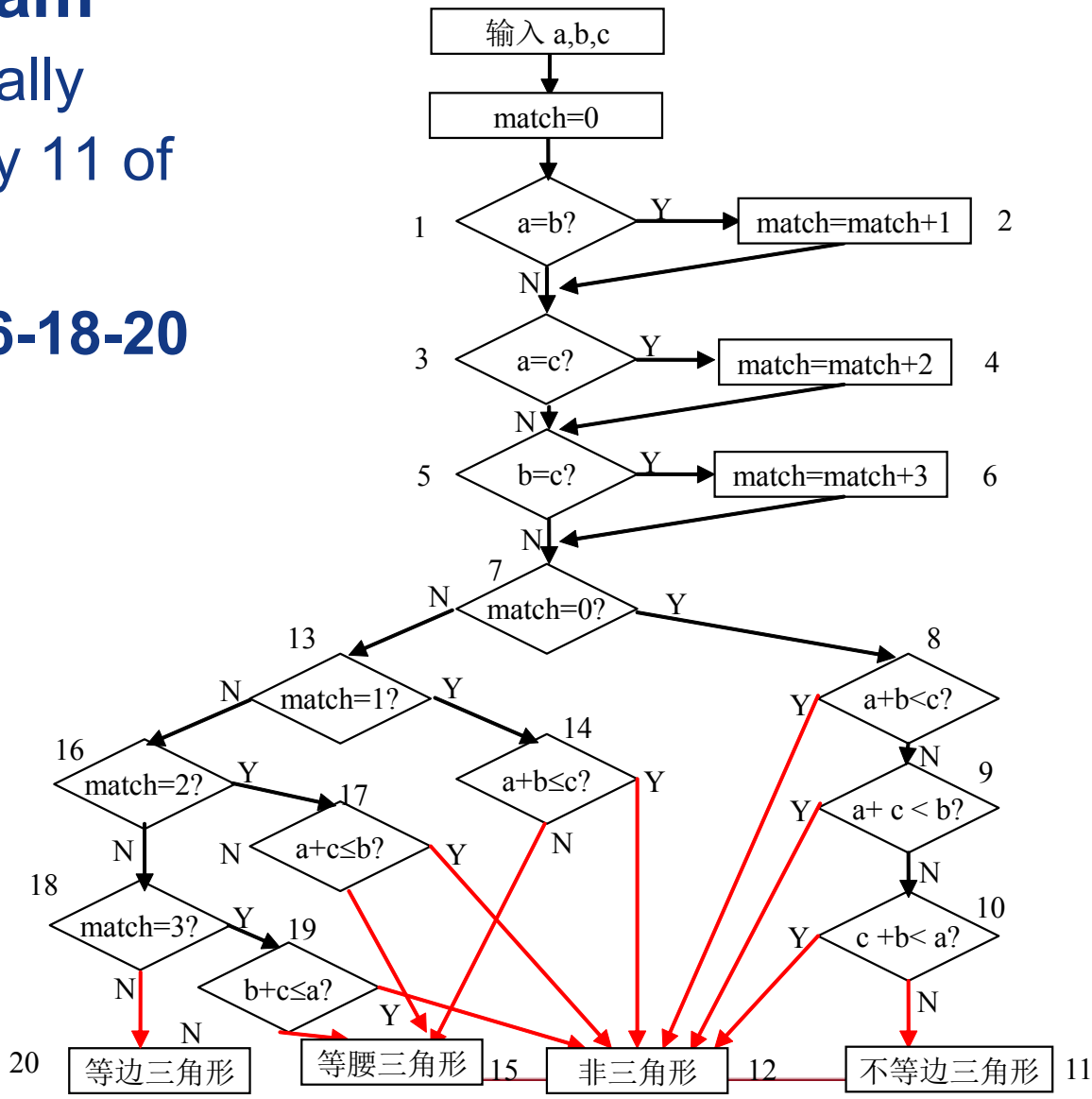
P4:

……

……

P10:1-3-5-7-8-9-10-12

P11:1-3-5-7-8-9-10-11





Suppose we use boundary value testing to define test cases, how many test cases? here are some.

Table 10.2 Path Coverage of Nominal Boundary Values

测试用例	a	b	c	预期输出	路径
1	100	100	1	等腰三角形	P6
2	100	100	2	等腰三角形	P6
3	100	100	100	等边三角形	P1
4	100	100	199	等腰三角形	P6
5	100	100	200	非三角形	P7
…	…	…	…	…	…

- the test cases generated using the nominal boundary value, the paths are covered: p1, p2, p3, p4, p5, p6, p7;
- worst-case boundary value testing, test cases: $5^3=125$, provide full path coverage, but the redundancy is onerous.

	p1	p2	p3	p4	p5	p6	p7	p8	p9	p10	p11
弱一般	1	3	1	3	1	3	1	0	0	0	0
最坏一般	5	12	6	11	6	16	7	17	18	19	12

Decision Table Triangle Program

C1 Match =	0				1				2			
c2. a + b < c?	T	F	F	F	T	F	F	F	T	F	F	F
c3. a + c < b?	F	T	F	F	F	T	F	F	F	T	F	F
c4. b + c < a?	F	F	T	F	F	F	T	F	F	F	T	F
a2. Not a triangle	X	X	X		X					X		
a1. Scalene				X								X
a3.Isosceles								X				
a4. Equilateral												
a5. Impossible						X	X		X		X	



C1 Match =	3				4	5	6			
c2. a + b < c?	T	F	F	F	—	—	T	F	F	F
c3. a + c < b?	F	T	F	F	—	—	F	T	F	F
c4. b + c < a?	F	F	T	F	—	—	F	F	T	F
a2. Not a triangle			X							
a1. Scalene				X						X
a3.Isosceles										
a4. Equilateral										
a5. Impossible	X	X			X	X	X	X	X	

we will have just 11 test cases, and we know we will have full coverage of the 11 feasible paths



7.3.2 Evaluating Test Methods

a specification-based testing technique M generates m test cases, and that these test cases are tracked with respect to a code-based metric S that identifies s elements in the unit under test. When the m test cases are executed, they traverse n of the s structural elements.

The coverage of a methodology M with respect to a metric S is *the ratio of n to s*. We denote it as C(M,S).

The redundancy of a methodology M with respect to a metric S is *the ratio of m to s*. We denote it as R(M,S).

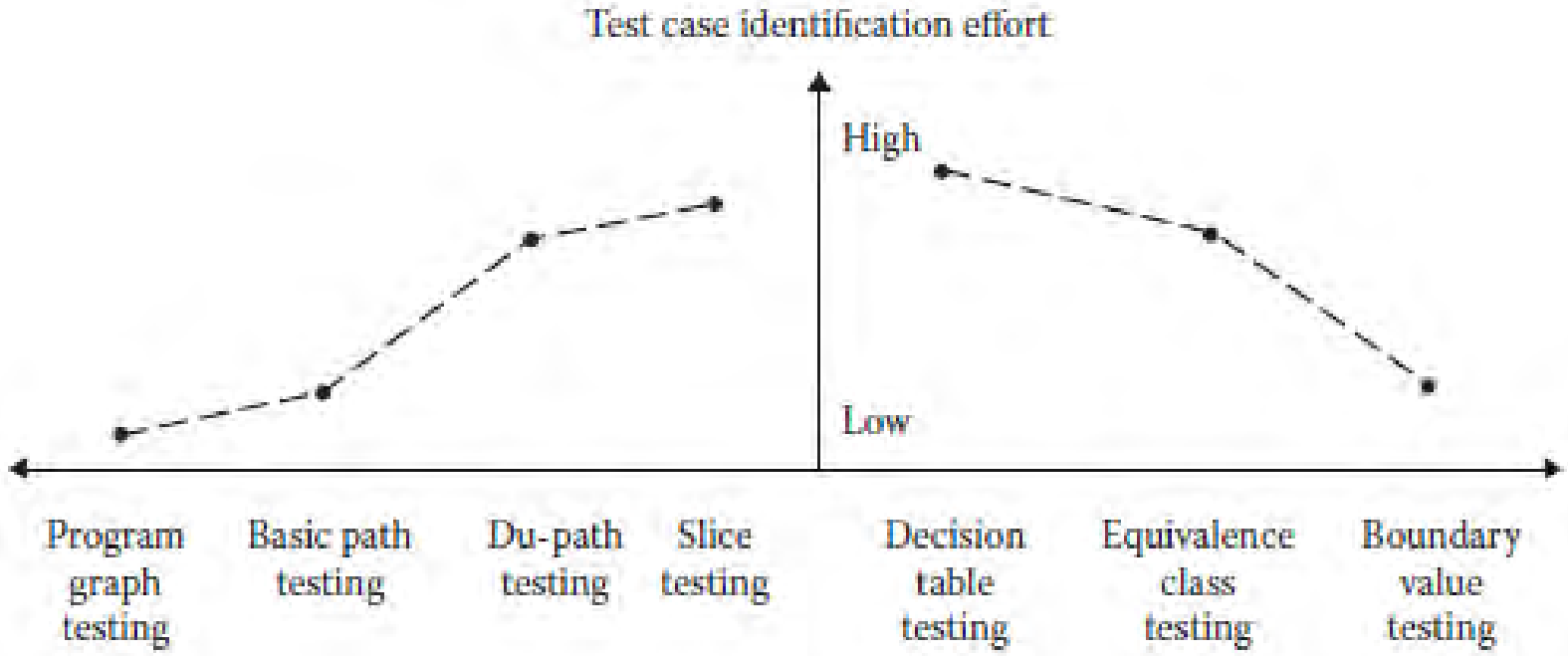
The net redundancy of a methodology M with respect to a metric S is *the ratio of m to n*. We denote it as NR(M,S).

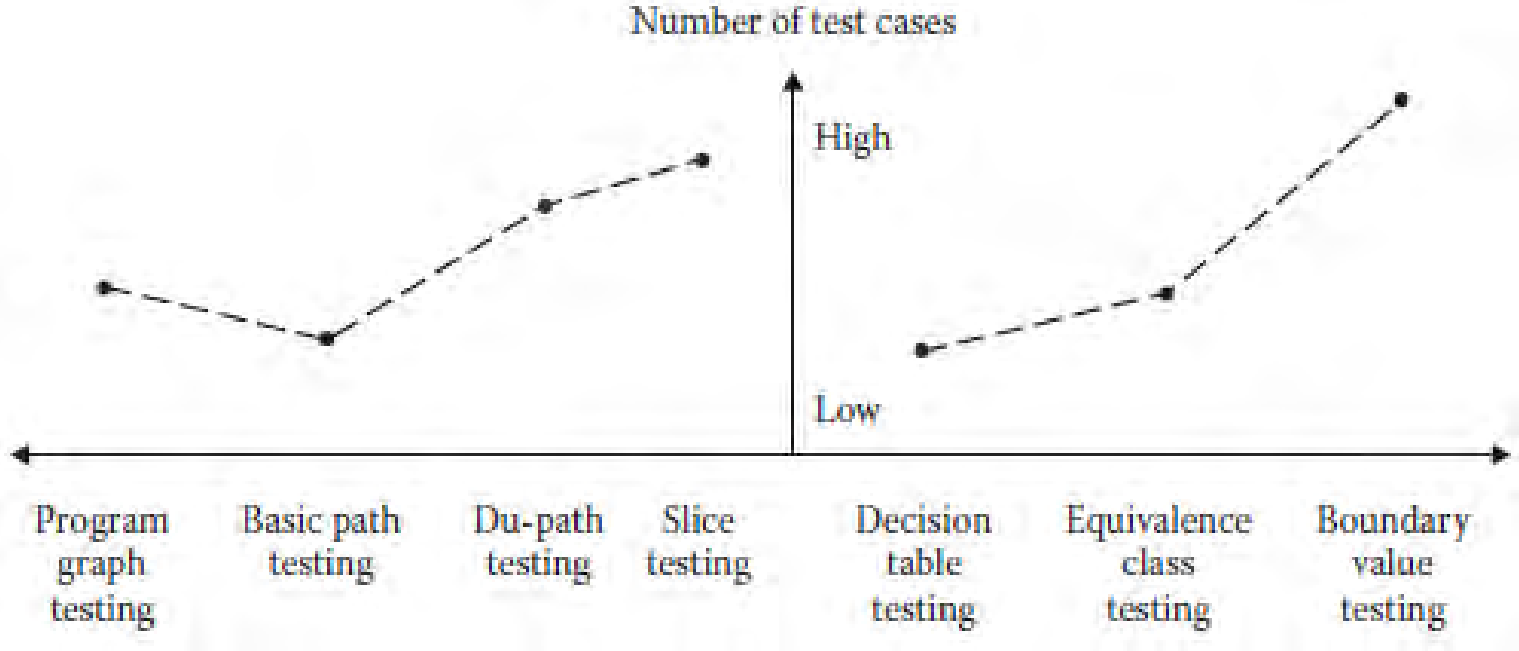
Table 10.5 Metrics for Triangle Program

Method	m	n	s	C(M,S)=n/s	R(M,S)=m/s	NR(M,S)=m/n
Nominal	13	7	11	0.64	1.18	1.86
Worst-case	125	11	11	1.0	11.36	11.36
Goal	s	s	s	1.0	1.0	1.0

Table 10.6 Metrics for Commission Problem

Method	m	n	s	C(M,S)=n/s	R(M,S)=m/s
Boundary	25	11	11	1	2.27
Decision table	3	11	11	1	0.27
DD-path	25	11	11	1	2.27
du-Path	25	33	33	1	0.76
Slice	25	40	40	1	0.63


Test case identification effort
High
Low
Program graph testing
Basic path testing
Du-path testing
Slice testing
Decision table testing
Equivalence class testing
Boundary value testing


Number of test cases
High
Low
Program graph testing
Basic path testing
Du-path testing
Slice testing
Decision table testing
Equivalence class testing
Boundary value testing

Insurance Premium Case Study
（learn by yourself）

问题

- 除了结构化的覆盖率指标外,还有其它的覆盖率指标吗?

Project 2

学生自选或自编程一个程序，程序在200行代码以上，采用白盒测试方法，利用软件测试工具（JUnit）设计测试用例，在eclipse平台上执行测试、测试记录和结果分析。并给出测试的语句覆盖率和分支覆盖率，测试的语句覆盖率达到95%以上，分支覆盖率达到90%以上。

可选择的题目：电子日历、通讯录、简单计算器、单位换算器、电子相册等。

提交：测试计划、测试统计分析报告；

例如：通讯录

内容：姓、名、职位、单位、地址、办公电话、家庭电话、传真、手机、电子邮箱、照片等；

功能：新建、查询、修改、删除、导出、导入（可选）等；

二级功能

查询：按某一属性查询、列表；

导出：输出为文本文件；

导入：从文本文件导入；